



硕士学位论文

(专业学位)



Windows 平台下基于 Proxmox 虚拟化环境的云桌面

系统设计与应用

Design and application of cloud desktop system based on
Proxmox virtualization environment on Windows platform

作者：游钊栗

类别（领域）：电子信息（控制工程）

指导教师：熊卫华 副教授

所在学院：信息科学与工程学院

完成日期：二〇二三年五月

摘 要

桌面虚拟化作为一种可以集中控制用户桌面的方法，由于其拥有较低的 IT 资源开销优势，被广泛应用于教育培训、医疗服务、政务办公等诸多领域。除此之外，它还适用于远程办公、代码开发、实验教学以及其他各种业务场合。随着虚拟化技术和计算机硬件的发展，越来越多的企业、政府和教育机构为了提高办公效率并降低运营成本，逐步进行虚拟化技术的创新与实践。“集中管理，集中运行，分布显示”则是桌面虚拟化系统的核心思想。

本文在对虚拟化和云桌面相关技术进行深入研究后，对各云桌面平台现有研究成果和现实困境进行了总结与分析。针对现有云桌面方案在软硬件捆绑销售和部署、系统客户端资源占用率过高等问题，提出了一种基于 Proxmox VE 虚拟化环境和 qemu-kvm 虚拟化底层技术，使用 VDI 架构的云桌面系统。本文设计的云桌面系统具有较好的通用性，不依赖特定的固件设备和硬件终端。服务端程序运行在安装 Proxmox VE 环境的物理服务器上，客户端程序运行于 Windows 平台的迷你电脑或瘦客户机上即可。本文的主要工作内容如下：

（1）确定本文所设计的云桌面系统的功能和性能需求。确定使用场景为高校云实验室和云办公，主要功能是为学生和教师用户提供一个桌面环境，满足其实验以及办公需求，并完成了云桌面系统的总体架构设计和各功能模块设计。

（2）弥补了国内云桌面厂商在线上直播教学功能上的空白，同时更好地发挥了网络教育在信息传递和资源传递中的优势。在本文设计的云桌面系统中加入了线上教育直播功能，实现校园内智慧云办公和云教学，创新了国内课堂进行线上教育的模式。此外，鉴于 H.264 视频编解码标准常用于实时流媒体视频直播，所以在云桌面系统的线上直播模块中采用 H.264 视频编解码标准对推流端的音视频原始数据进行编码，收流端再将数据解码并通过设备进行播放。本文通过优化 x264 编码器以改进 FFmpeg 音视频框架，以此降低直播过程中的延迟时间。

（3）设计并实现了系统用户组管理、云桌面生成和运行管理、虚拟存储管理等连接个人云桌面功能。在服务器端 Server 程序中对 Proxmox VE 虚拟化环境的部分组件进行重写和优化，通过暴露的接口与底层 Proxmox VE 进行通信。涉及的功能组件包括桌面环境的创建、QEMU 侧 CPU 的创建、KVM 侧 vCPU 的创建、内存虚拟化和设备虚拟化组件等。

(4) 对云桌面系统进行部署和验证, 对用户管理、桌面创建与运行、线上直播教学等核心功能进行了功能和性能上的验证。验证结果表明, 本文设计的云桌面系统各模块工作正常, 系统的功能需求和性能需求达到了设计要求。能够满足校园内智慧云办公和云教学的目标。

关键词: 桌面虚拟化; 虚拟化技术; qemu-kvm; 高校云实验室; 线上直播教学

Abstract

As a method to centrally control users' desktops, desktop virtualization is widely used in many fields, such as education and training, medical services, and government offices, due to its low IT resource overhead. In addition, it is suitable for telecommuting, code development, laboratory teaching, and a variety of other business situations. With the development of virtualization technology and computer hardware, more and more enterprises, governments and educational institutions are gradually innovating and practicing virtualization technology in order to improve office efficiency and reduce operating costs. "Centralized management, centralized operation, distributed display" is the core idea of desktop virtualization system.

After an in-depth study of virtualization and cloud desktop related technologies, this paper summarizes and analyzes the existing research results and practical difficulties of each cloud desktop platform. Aiming at the problems of software and hardware bundle sales and deployment of the existing cloud desktop solutions and high resource usage of the system client, a cloud desktop system based on Proxmox VE virtualization environment and qemu-kvm virtualization underlying technology using VDI architecture is proposed. The cloud desktop system designed in this paper has better versatility, independent of specific firmware devices and hardware terminals. The server program runs on the physical server where the Proxmox VE environment is installed, and the client program runs on a Windows mini computer or thin client. The main contents of this paper are as follows:

(1) Determine the functional and performance requirements of the cloud desktop system designed in this article. The main function is to provide a desktop environment for students and teachers to meet their experimental and office needs. The overall architecture design and functional module design of the cloud desktop system are completed.

(2) It makes up for the gap in the online live teaching function of domestic cloud desktop manufacturers, and gives better play to the advantages of network education in the transmission of information and resources. In the cloud desktop system designed in this paper, the online education live broadcast function is added to achieve intelligent

cloud office and cloud teaching on campus, which innovates the online education mode in domestic classrooms. In addition, in view of the fact that H.264 video codec standard is often used for real-time streaming video broadcast, the online live broadcast module of the cloud desktop system uses H.264 video codec standard to encode the original audio and video data of the streaming end, and then the receiving end decodes the data and plays it through the device. In this paper, the x264 encoder is optimized to improve the FFmpeg audio and video framework, so as to reduce the delay time in the process of live broadcasting.

(3) The system user group management, cloud desktop generation and operation management, virtual storage management and other functions of connecting personal cloud desktop are designed and implemented. Some components of the Proxmox VE virtualization environment are rewritten and optimized in the server-side Server program to communicate with the underlying Proxmox VE through the exposed interface. The functional components involved include creation of desktop environment, QEMU side CPU creation, KVM side vCPU creation, memory virtualization and device virtualization components, among others.

(4) The cloud desktop system was deployed and verified, and the core functions such as user management, desktop creation and operation, and online live teaching were verified in terms of function and performance. The verification results show that each module of the cloud desktop system designed in this paper works normally, and the functional requirements and performance requirements of the system meet the design requirements. It can meet the goal of smart cloud office and cloud teaching on campus.

Keywords: desktop virtualisation; virtualization technology; qemu-kvm; cloud lab for universities; online live teaching

目 录

第 1 章 引言.....	1
1.1 课题研究背景与意义.....	1
1.2 国内外研究现状.....	2
1.2.1 国外研究现状.....	2
1.2.2 国内研究现状.....	3
1.3 本文的主要工作和章节安排	4
第 2 章 云桌面相关技术简述及系统总体方案	6
2.1 虚拟化技术简述.....	6
2.1.1 虚拟化的概念及定义.....	6
2.1.2 操作系统虚拟化的实现方式.....	7
2.1.3 虚拟化层 Hypervisor 的分类.....	9
2.2 kvm-qemu 虚拟化实现原理	10
2.2.1 KVM 虚拟化解决方案的体系结构	10
2.2.2 libvirt 与 kvm-qemu 的关系	11
2.3 桌面传输协议 SPICE.....	11
2.4 VDI 虚拟桌面基础架构	12
2.5 FFmpeg 音视频处理框架	13
2.6 系统的业务场景分析.....	14
2.7 系统在功能和性能上的需求	15
2.7.1 学生用户对功能的需求.....	15
2.7.2 教师用户对功能的需求.....	16
2.7.3 管理员用户的权限需求.....	17
2.7.4 系统在性能上的需求.....	19
2.8 系统总体设计.....	20
2.8.1 系统逻辑架构设计	20
2.8.2 系统物理架构设计	22
2.8.3 系统各模块的分类.....	24
2.9 本章小结.....	24
第 3 章 系统线上直播教学功能的设计与应用	26
3.1 H.264 编码技术.....	27
3.2 x264 编码器系统优化.....	28
3.2.1 编码器的编码流程.....	28
3.2.2 编码器的算法可定制模块.....	30
3.2.3 编码器的参数优化.....	31

3.3 编译 FFmpeg 时安装 x264 编码器	35
3.4 本章小结	36
第 4 章 系统连接个人桌面功能的设计与应用	37
4.1 用户管理模块设计与实现	37
4.1.1 用户的创建与删除	37
4.1.2 用户的登陆与注销	40
4.2 桌面管理模块设计与实现	43
4.2.1 桌面环境的创建	43
4.2.2 QEMU 侧 CPU 的创建	45
4.2.3 KVM 侧 vCPU 的创建	47
4.2.4 内存虚拟化	49
4.2.5 设备虚拟化	51
4.3 虚拟存储模块维护与管理	54
4.3.1 宿主机的存储资源	54
4.3.2 qemu-img 和 pvesm 命令的使用	55
4.3.3 基础映像和派生映像	57
4.4 本章小结	58
第 5 章 系统的构建、运行和验证	59
5.1 系统的运行环境和部署情况	59
5.1.1 服务端和客户端程序开发环境	59
5.1.2 服务端和客户端设备硬件配置	59
5.1.3 Proxmox VE 虚拟化环境的部署	60
5.1.4 服务端 Server 程序和客户端 Client 程序的部署	61
5.2 系统的功能验证	62
5.2.1 用户创建、删除等管理功能	62
5.2.2 云桌面的创建和运行	64
5.2.3 直播教学功能验证	66
5.3 系统的性能验证	67
5.3.1 云桌面运行时性能验证	67
5.3.2 直播教学性能验证	70
5.4 本章小结	71
第 6 章 总结与展望	72
6.1 工作总结	72
6.2 后续展望	73
参考文献	75

第1章 引言

1.1 课题研究背景与意义

随着虚拟化技术的不断发展,现如今已经可以将其运用到桌面端设备,即桌面虚拟化技术,也称其云桌面。这是一种以用户为中心的计算模式,通过消除用户桌面和特定硬件之间的绑定关系,使用户从设备中解脱出来^[1,2]。因桌面虚拟化技术带来的便捷性,使得这项技术在越来越多的领域内受到关注。目前,在企业办公、高校计算机实验室等诸多领域得到广泛应用^[3-5]。

使用硬件虚拟化技术的物理主机可以独立地为每个用户提供服务^[6],提高系统并发处理能力^[7]。再通过虚拟桌面传输协议将物理主机上的资源“发送”到指定用户的客户机上,可以达到与本地主机环境一样的使用效果^[8]。它利用虚拟化技术拓展了现有的物理设备上的资源空间,使得大量的服务器单元可以合并使用。通过虚拟化技术,即使在只有一台物理服务器的情况下,也可以创建出不同的资源分区将一整台物理主机的性能资源进行分区分块。由此,一台物理机器就变成了可以被多个服务或者多个用户同时共享和使用的共享资源。提高了机器的运行效率,也节省了用户因业务需求增多而新购置主机设备的花费^[9,10]。

对于企业用户而言,基于桌面虚拟化技术和超融合技术构建的云平台可以有效提高硬件资源利用率,节约数据中心建设成本,在功能和性能上优于传统模式^[11]。高校还可以利用桌面虚拟化技术实现定制的虚拟实验环境,使用远程桌面虚拟化技术将机房中的服务器资源虚拟化,并且将其分成不同的小块交给实验需求不相同的用户调用,这样的自适应配备可以满足不同课程下实验资源的搭建。每个实验都可以配置多个虚拟互联网主机,学生和老师通过登陆客户端,就可以访问由服务器虚拟出来的各种系统操作环境。还可以对不同用户操作和数据做长期的保存^[12,13]。这样的部署模式有以下优势:

(1) 访问灵活

用户可以在任何时间、任何地点和任何设备上访问云桌面,这是它的主要优点之一^[14]。在云桌面的技术架构下,用户可以通过使用云桌面来访问互联网,实现跨地域的应用。企业可以将不同环境、不同地域和不同地点的用户部署在同一套云桌面系统上进行办公和开发。企业也可以通过对用户数据的统一管理,从而减少重复建设,也增加了信息安全性^[15]。

（2）管理方便

每个桌面只是一个服务器端上的镜像文件，它包含了操作系统、应用程序和用户的个人资料。所以部署新桌面就非常简单，只需克隆或复制该映像并将其作为实例运行即可^[16,17]。在桌面运行过程中，也支持复制快照和备份数据，便于在桌面出错时能快速恢复用户的数据和工作环境。

（3）拓展性强

当机房想为不同的功能需求添置不同的设备时，只需要在原物理服务器上运行性能拓展以安装不同的镜像，就可以达到新增服务的目的，避免了因购置新设备而产生的高昂费用问题^[18]。

（4）成本更低

尽管使用云桌面部署方式不会显著降低初期的部署费用，但可以降低长期机房的运行成本，如增加硬件寿命，减少能源损耗，减少维护复杂性等^[19]。

（5）提高性能和安全性

传统的本地桌面是基于内存计算，因此会造成资源使用效率低的问题。而通过云桌面可以避免 CPU 占用率过高的问题，从而有效地提高资源利用效率^[20]。另外，通过使用虚拟化技术来实现对桌面的集中管理，从而提升网络性能和安全性。对于云桌面的集中管理，可以实现资源统一部署、统一维护^[21-23]。

1.2 国内外研究现状

随着近年来云桌面应用的快速增长，越来越多的国内外云服务提供商开始构建云桌面平台并迅速占据市场地位^[24-26]。本节列出了国内外知名云厂商的云桌面解决方案。

1.2.1 国外研究现状

在国外，VMware、思杰（Citrix）、微软（Microsoft）、亚马逊（Amazon）这几家的云桌面解决方案处于市场主流地位。

VMware 多年来一直是虚拟化技术的先锋，其最新的桌面云产品是 VMware Horizon。VMware Horizon 是一款虚拟桌面基础架构软件，它与 VMware 软件定义的数据中心（包括 vSphere、vSAN 和 NSX）紧密集成，将传统的本地静态桌面转变为通过统一的 VDI 平台提供的虚拟桌面。该虚拟应用平台可在不同的数据中心扩展并快速分配 Windows 和 Linux 资源^[27]。

Citrix Desktops（前身为 Xen Desktop）是 Citrix 公司的桌面云服务，其建立在 VDI 架构和 Citrix HDX 技术之上。Citrix HDX 技术保证了桌面使用过程中对高质

量音视频的支持，并通过 USB 重定向技术获取更多外围设备的接入。Citrix Desktops 使用的远程连接协议是 ICA 协议^[28]，此协议负责客户端与云端桌面之间的消息传输。作为一种按需服务，Citrix Desktops 可以向客户快速提供桌面资源，使他们可以在连接到网络的任何设备上访问云桌面。Citrix Desktops 作为 Xen Desktop 的后代产品，解决了其前作 Xen Desktop 故障率高和操作维护困难的特点，Citrix Desktops 技术架构也更加适应于未来的云计算趋势。

Microsoft 的桌面云解决方案是在 Azure 公共云上提供 Windows 虚拟桌面服务。Azure 虚拟桌面^[29,30]是一个运行在 Azure 云上的桌面和应用程序虚拟化服务，该服务基于 VDI 架构，可以在 Azure 上快速部署和扩展 windows 桌面和应用程序。

致力于云服务研究的亚马逊 Amazon，也为 DaaS 基础设施领域提供了自己的解决方案：Amazon WorkSpaces。Amazon WorkSpaces 提供云桌面托管服务，可将用户从与桌面生命周期管理相关的许多管理任务中解放出来，包括配置、部署、维护和回收桌面。用户需要管理的硬件数目和种类更少，并且避免了不能扩展的复杂虚拟桌面基础设施部署。Amazon WorkSpaces 部署于 Amazon Virtual Private Cloud 内^[31]，同时 Amazon Virtual Private Cloud 也为 Amazon WorkSpaces 部署在自定义的虚拟网络中提供了解决方案。与其他云供应商提供的云桌面解决方案相比，亚马逊的 WorkSpaces 提供了更加简单的管理和交付策略，因此越来越多的企业和个人都选择 Amazon 的云桌面平台。

1.2.2 国内研究现状

从产业发展角度来说，近年云桌面制造商不仅保证了传统客户的产品需求，也满足了国内通信运营商、大型云平台运营商的技术使用需求。各厂商在研发、核心技术上进行合作，形成优势互补、多方合力，在为用户提供灵活便捷的云桌面服务的同时，也创造了新的增长点。鉴于云桌面按需使用、动态灵活等优势，随着网络带宽的提升、硬件计算能力的增强、云桌面协议的不断完善，越来越多的行业开始选择云桌面的解决方案，而不是传统的 PC。数据显示，中国云桌面整体解决方案销量从 2016 年的 108.2 万台增长至 2021 年的 221.3 万台，年均复合增长率占比 19.59%^[32-34]。目前，医疗、教育、政企、金融等多个行业已经实现了云桌面的规模化^[35-37]。据计世资讯发布的《2021 年中国信创桌面云产业研究报告》显示，云桌面产业链上游主要包括瘦客户机、服务器、操作系统等前端和后端软硬件产品，自主创新能力显著提升。产业链中游主要包括云桌面解决方案提供商，服务能力得到显著提升。产业链下游主要包括政府、教育和医疗保健等部门的用户。随着云桌面产品质量的稳步提高，产业链下游覆盖的产业领域也在稳步提升。

在这样的产业发展趋势下，政务、教育、医疗保健等关系民生的活动将得到更好的保障，对于实现资源共享、解决地区差异和人才短缺等制约条件问题具有重要意义。

从技术发展角度来说，云计算技术不断发展，涌现出 VDI（虚拟桌面基础设施）、IDV（智能桌面虚拟化）、VOI（虚拟操作系统基础设施）、RDS（共享云桌面架构）等桌面云技术架构，其中 VDI 和 IDV 是目前主流的技术架构。

在诸多的国内厂商中，华为的 FusionAccess 云桌面是建立在华为云平台上的虚拟桌面应用程序。终端用户可以使用 PC、瘦客户端或移动设备登录云桌面软件，并使用华为专有的云桌面传输 HDP 协议访问平台应用程序和整个虚拟桌面^[38]。华为云桌面 FusionAccess 在很大程度上仍然是基于经典的 VDI 架构。

深信服的 aDesk 云桌面服务同样是建立在 VDI 架构之上。它将服务器虚拟化、桌面虚拟化和存储虚拟化彻底融合在一起。将原来存放在传统 PC 设备上的所有桌面、应用和数据全部集中到数据中心，然后使用 SRAP 桌面交付协议将虚拟桌面环境传输给前端的访问设备。

噢易云公司的云桌面解决方案为武汉职业技术学院打造的全国首家信创桌面云机房，成功部署了 100 套超融合云桌面和多媒体课堂软件，全面适配国产 CPU 和操作系统^[39]。在国产操作系统环境下，保证了视频、屏幕共享等功能的流畅度，软件还支持屏幕广播、视频广播、远程协作、文件传输、收发作业、黑屏肃静等教学互动功能，让教学内容更丰富，教学形式更多元，营造了优质高效的国产化课堂。

1.3 本文的主要工作和章节安排

本文内容一共六章，具体章节安排如下：

第 1 章：引言。该章主要叙述了虚拟化、云计算和云桌面的研究背景以及研究意义。介绍了国内外该领域的研究现状，最后给出本文所设计系统的工作方向与研究内容。

第 2 章：云桌面相关技术简述及总体方案。该章对虚拟化技术进行了整体概述，介绍了虚拟化的概念和分类。针对操作系统虚拟化的实现方式和虚拟化层 Hypervisor 的分类进行了具体的介绍。其次对 kvm-qemu 虚拟化底层体系、桌面传输 SPICE 协议、VDI 虚拟桌面基础框架等技术也进行了阐述。该章对云桌面系统的主要业务场景进行了介绍，提出云桌面系统以高校云办公和云实验室为主要目标。分析了云桌面系统的各类需求，并对系统的业务逻辑进行抽象与分析。同时，

提出基于 Proxmox VE 虚拟化环境的云桌面系统的逻辑架构和物理架构。并对系统做功能上的划分。

第 3 章：系统线上直播教学功能的设计与应用。该章针对云桌面系统的直播教学模块进行了详细设计和实现。主要包括 H264 编解码技术的阐述、编码器可定制模块的分析和云桌面系统线上直播教学功能的一些技术探讨和模块优化。

第 4 章：系统连接个人桌面功能的设计与应用。该章主要介绍了云桌面系统个人桌面功能的详细设计和实现。主要包括管理用户帐户、创建和管理虚拟桌面等。针对每个模块的功能和其实现方式也在本章中做了相关的设计，对于一些重要部分的原理过程，给出图表和流程框图进行梳理。

第 5 章：系统的构建、运行和测试。该章首先介绍了服务端和客户端程序的开发环境、服务端和客户端设备的硬件配置，其次介绍了相关虚拟化平台在服务器上的部署，最后对云桌面平台用户管理、桌面创建与运行和直播教学等相关的功能和性能做了相对应的测试。通过相关实验可以得出结论，系统的功能需求和性能需求达到了要求。

第 6 章：总结与展望。该章首先总结了云桌面系统的设计和研究工作，展望了后续研究工作的方向与内容。

第2章 云桌面相关技术简述及系统总体方案

相对于目前国内云桌面厂商系统软件和硬件设备捆绑销售的情况，本文所设计的云桌面系统更像是一种“应用程序”运行在客户的 Windows 终端中，具有较好的通用性，不依赖特定的固件设备和硬件终端。服务端 Server 程序运行在安装 Proxmox VE 环境的物理机服务器上即可，客户端 Client 程序运行在 Windows 平台的迷你电脑或瘦客户机上即可，且终端设备不要求有高计算和大存储能力。

本节将阐述云桌面用到的相关技术、整个系统的功能需求和性能需求、系统总体架构设计以及各功能模块设计。

2.1 虚拟化技术简述

2.1.1 虚拟化的概念及定义

维基百科对虚拟化的定义是“在计算机技术中，虚拟化或虚拟技术是一种资源管理技术，是将计算机的各种实体资源（CPU、内存、磁盘空间、网路适配器等），予以抽象、转换后呈现出来并可供分割、组合为一个或多个计算机组态环境。由此，打破实体结构间的不可切割的障碍，使用户可以比原本的组态以更好的方式来应用这些计算机硬体资源。这些资源的新虚拟部分是不受现有资源的架设方式、地域或物理组态所限制。一般所指的虚拟化资源包括计算能力和资料存储”^[40]。依据应用的对象不同，虚拟化技术主要分为以下几种：

（1）操作系统虚拟化

将物理资源划分为一个或多个运行环境，在不同的环境中运行不同的操作系统。

（2）存储虚拟化

对硬盘等存储资源进行抽象化。

（3）GPU 虚拟化

将图形处理单元虚拟化之后，可以让运行在数据中心服务器上的虚拟机共享使用同一块或多块 GPU 处理器。

（4）网络虚拟化

将以前基于硬件的网络过渡到基于软件的网络被称为网络虚拟化。网络虚拟化的目的是在实际的硬件和使用该硬件的活动之间设置一个抽象层。它可以起到

合并多个物理网络，或者将一整个物理网络进一步细分，又或者将多个虚拟机网络连接起来的作用。

本文实现云桌面解决方案的方式主要是利用操作系统虚拟化，操作系统虚拟化是对整个计算机操作系统及呈现给用户的桌面环境进行虚拟化，以提高用户桌面访问的安全性、灵活性和高效性。运行在虚拟化平台上的个人桌面可以在任何时间、地点通过终端登陆进行访问，且终端设备可以是不要求有高计算和大存储能力的瘦终端机或迷你主机，从而达到安全、灵活和高效的办公体验。2.1.2 节将对操作系统虚拟化及其实现方式做详细的阐述。

2.1.2 操作系统虚拟化的实现方式

云桌面解决方案中的虚拟化应用，更多指的是操作系统虚拟化。实现操作系统虚拟化主要有模拟器仿真、虚拟化翻译和容器技术三种。

第一种是模拟器仿真技术，也称纯软件仿真。顾名思义就是完全通过软件去模拟一整套硬件环境。由于该环境是由纯软件来完成的，所以效率比较低，但它的优点是可以仿真模拟多种硬件。在 PC 机上运行的游戏仿真软件或游戏模拟器就是采用了该原理，有了这类软件就可以在 PC 机上运行早期的街机游戏和电视游戏机游戏。模拟器仿真的虚拟化产品主要有 QEMU、PearPC 和 Bochs，利用模拟器仿真软件可以模拟 x86、ARM、PowerPC 等多种 CPU 架构。

第二种是虚拟化翻译技术，它的机制与纯软件仿真完全不同。目前大多数的虚拟化都是采用了虚拟化翻译技术，这是由虚拟机监视器 Hypervisor 实现的，如图 2.1 所示。

Hypervisor 是一个虚拟化层，也可以称为子系统或软件层。Hypervisor 可以实现多个虚拟机操作系统（Guest 系统）在同一个宿主机系统（Host 系统）中运行。它为 Guest 操作系统提供硬件的虚拟。虚拟机监视器 Hypervisor 对 Guest 系统的运行情况进行监视和管理，Guest 系统对硬件的访问会被“翻译”成对宿主机物理硬件的访问。同时虚拟机监视器 Hypervisor 还要管理 Guest 系统对资源的利用，协调处理多个 Guest 系统同时访问宿主机物理资源时的冲突。根据 Guest 操作系统如何通过 Hypervisor 对硬件进行访问，又可以将虚拟化翻译技术细分为：无硬件辅助的全虚拟化、半虚拟化和有硬件辅助的虚拟化。现阶段应用最广泛的是有硬件辅助的虚拟化翻译技术。Microsoft Hyper-V、KVM、VirtualBox、VMware Workstation 6.0 以上版本的虚拟化框架都采用的是该方式。

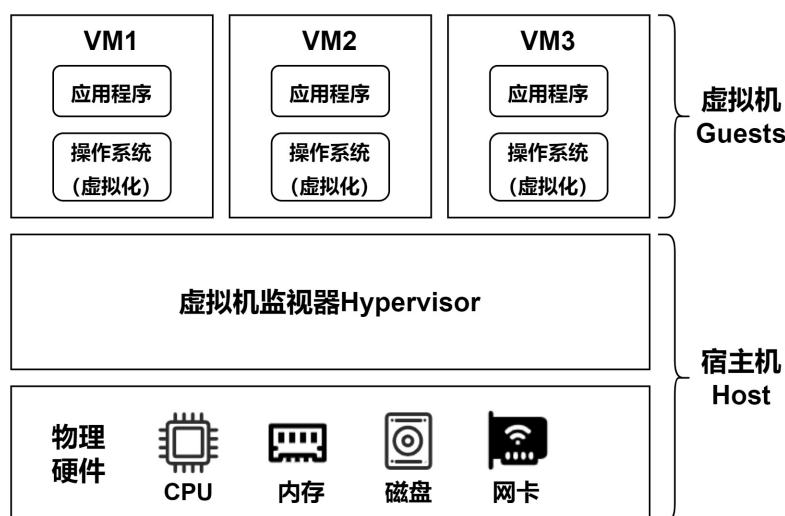


图 2.1 虚拟化翻译技术体系架构图

第三种实现虚拟化的方式是容器技术。容器技术最早出现在 Linux 系统中，如今 Windows 操作系统也可以很好地支持容器技术。“容器”是指在宿主机 Host 操作系统上构建出的一个或多个隔离化的轻型运行环境，用于在 Host 操作系统上运行应用程序，如图 2.2 所示。容器与容器之间是互相隔离的。单个容器不能对 Host 系统内核进行访问，容器与 Host 系统的用户模式环境也是相互隔离的。这种轻型隔离环境使应用更易于开发、部署和管理。容器可以快速地启动、停止，因此适用于不断变化的系统开发需求，这种部署方式将大大减少资源浪费，提高项目部署的效率和基础设施的利用率。

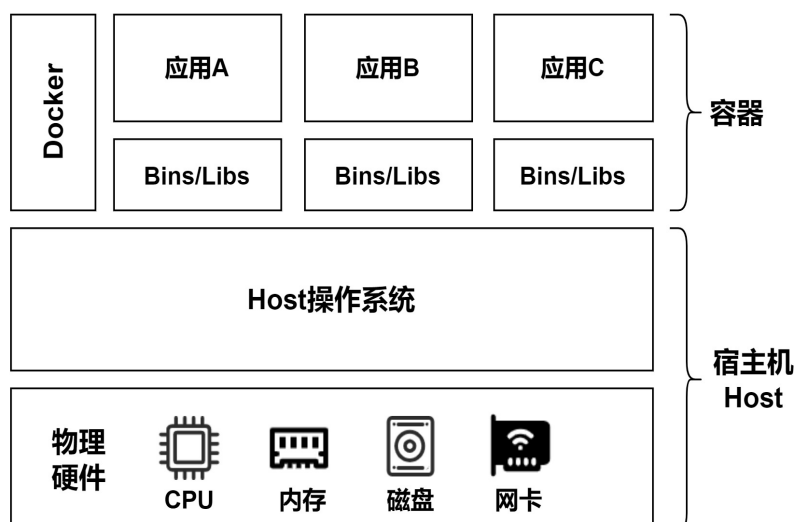


图 2.2 容器技术体系架构图

2.1.3 虚拟化层 Hypervisor 的分类

如果将 2.1.2 小节所述的虚拟化翻译技术中的虚拟机监视器 Hypervisor 细化, 根据 1974 年发表在《美国计算机学会通讯》上的文章《Formal requirements for virtualizable third generation architectures》可将 Hypervisor 分为两种类型^[41]: 裸金属型和宿主型。

裸金属型 Hypervisor 直接运行在宿主机 Host 的硬件上来控制物理硬件和 Guest 操作系统如图 2.3 所示。典型的产品有 Xen Server、VMware ESXi。

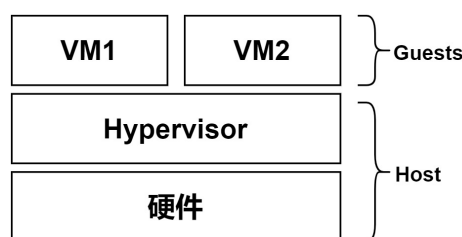


图 2.3 裸金属型 Hypervisor 结构图

对于裸金属型虚拟机监视器 Hypervisor 来说, 当宿主机 Host 通电开机后, 最先加载 Hypervisor。因为它是直接运行在物理硬件上的, 所以从某种意义上来说, 此时的 Hypervisor 可以视为为虚拟化功能而专门裁剪优化过的操作系统。它除了负责虚拟机 Guest 的创建和管理外, 还要负责系统初始化、物理硬件资源管理等通用操作系统功能。其优点是专用、体积小, 但缺点是整个系统可拓展性不强。

宿主型 Hypervisor 则像系统内其他进程一样都运行在通用操作系统(如 Linux、Windows 操作系统)上, 如图 2.4 所示。典型的产品有 VMware Workstation、Windows Server 中的 Hyper-V、Oracle VirtualBox、QEMU 和 KVM。

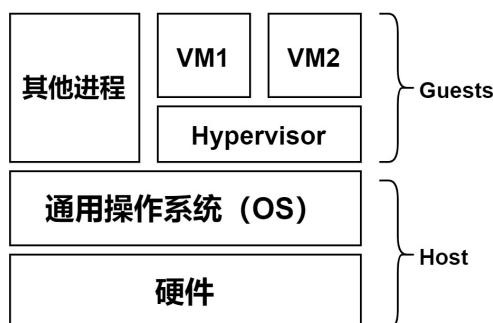


图 2.4 宿主型 Hypervisor 结构图

对于该类型的虚拟机监视器, 当宿主机 Host 通电后, 首先运行宿主机上的通用操作系统。Hypervisor 仅作为一个特殊的、可加载的内核模块嵌入通用操作系统

上，可以当做宿主机通用操作系统的拓展。这样的好处是增强了整个宿主机的拓展性，且实现起来比较简洁。

2.2 kvm-qemu 虚拟化实现原理

KVM 虚拟化解决方案是由 KVM、QEMU、libvirt 这三个项目组合在一起形成的，最初由以色列 Qumranet 公司开发，后被 Red Hat 公司收购。2007 年 2 月，正式被纳入 Linux 内核版本中，从此 Linux 2.6.20 及以上的内核版本都包含 KVM 虚拟化模块。

2.2.1 KVM 虚拟化解决方案的体系结构

KVM 作为可加载的内核模块包含在 Linux 内核中，除了通用模块 `kvm.ko` 外，还针对不同的 CPU 型号，发布了特定的模块，例如针对 AMD 的 `kvm-amd.ko` 和针对 Intel 的 `kvm-intel.ko`。加载 KVM 模块之后，一台 Linux 主机摇身一变成为一台虚拟机监视器 Hypervisor。KVM 虚拟化解决方案的体系结构如图 2.5 所示，其本质上就是一组实现虚拟化功能的 Linux 内核模块，它们的功能就是初始化硬件，打开虚拟化模式以支持虚拟机或容器的运行。

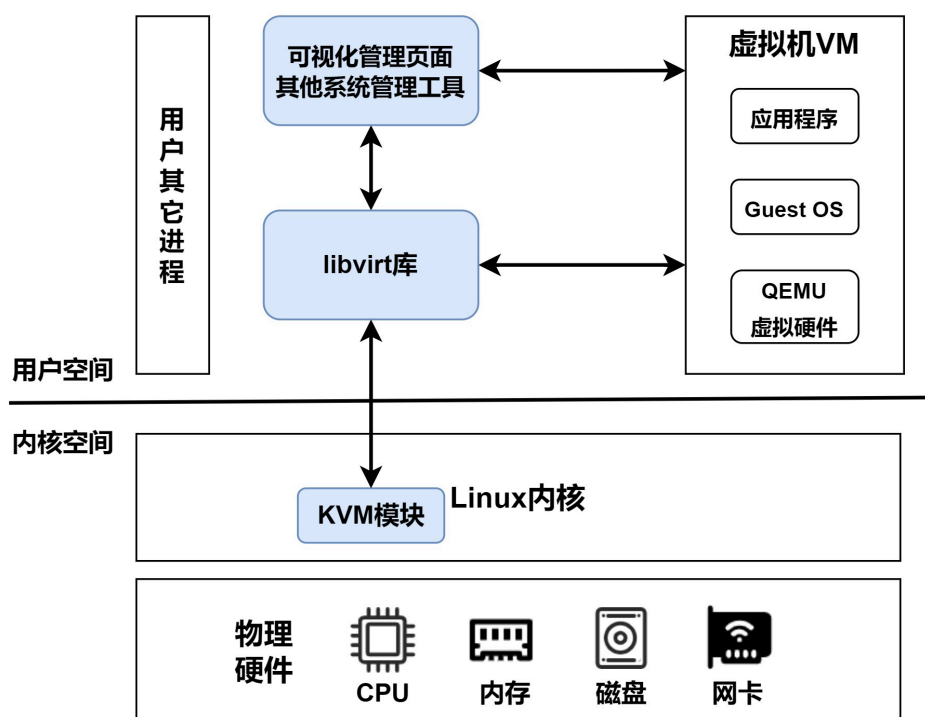


图 2.5 KVM 解决方案体系结构

KVM 只负责 CPU 和内存的虚拟化^[42-44]，它同时还控制着虚拟机系统的中断控制器、时钟控制器等设备。虚拟机其他的硬件设备例如显卡、声卡、网卡、USB 控制器和硬盘等都由 QEMU 来负责模拟^[45-47]。KVM 是内核空间的进程，而 QEMU 则是用户空间的进程，它通过 KVM 创建的/dev/kvm 的接口去管理 Guest 虚拟机操作系统的地址空间，从而向虚拟机提供模拟出来的硬件设备。

2.2.2 libvirt 与 kvm-qemu 的关系

libvirt 是一个软件库，包括 API 库、守护程序 libvirtd 和其他用于管理虚拟机和虚拟化操作的工具。它在云计算和虚拟化解决方案中得到了广泛的应用^[48]。libvirt 软件库在 KVM 解决方案中扮演管理工具的角色，通过 libvirt 提供的接口可以实时监控虚拟机运行和消耗资源情况，还可以通过其接口进行二次开发，将整个解决方案运行、监控和管理情况用 web 和客户端进行可视化显示。

2.3 桌面传输协议 SPICE

桌面传输协议充当用户和服务器后端之间的桥梁。无论云桌面架构如何，桌面环境都必须使用传输协议通过网络发送到云桌面客户端，之后客户端显示屏幕并最终向用户呈现桌面环境。SPICE (Simple Protocol for Independent Computing Environments) 协议是一款开源的桌面传输协议，最早由 Qumranet 开发，后被 Red Hat 公司收购并开源。SPICE 协议的整体架构如图 2.6 所示，搭载 SPICE 协议的客户端与 SPICE 服务端通过虚拟通道实现数据间的交互。

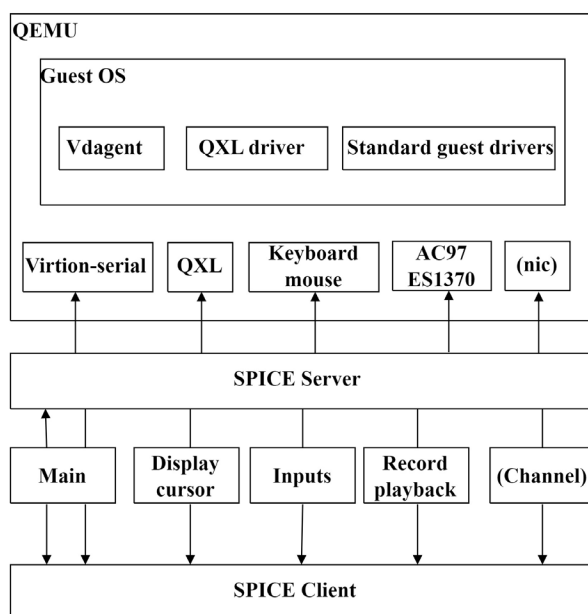


图 2.6 SPICE 协议架构图

SPICE Server 向上层的 QEMU 虚拟机共享管理程序，将 SPICE Server 在装有虚拟化模块物理主机上编译会得到一个供 QEMU 使用的动态链接库。QEMU 就会使用命令行参数启动 SPICE Server。同时 SPICE Server 向下与 SPICE Client 进行通信，其中虚拟通道一共分为六种。主通道负责控制和配置；显示通道负责处理图形命令、图像和视频流播放；输入通道处理鼠标和键盘输入；光标通道处理指针的位置、可见性和形状的显示；音频通道负责接收来自服务器主机的音频数据，并在客户端设备上播放；记录通道负责捕获在客户端产生的音频数据。最上层的 Guest OS 为用户最终使用的云桌面虚拟机，提供 Vdagent、标准驱动以及 QXL 驱动等功能^[49,50]。

本文采用 SPICE 协议作为虚拟桌面传输协议来部署云桌面系统，用户的桌面环境均存放在服务器端。用户使用终端设备时，通过输入用户名和密码等登录信息实现随时随地的接入到虚拟主机上的个人桌面。

2.4 VDI 虚拟桌面基础架构

Virtual Desktop Infrastructure 架构，中文也称为虚拟桌面基础结构，简称 VDI 架构。它是第一个广泛实现的以“集中管理、集中操作”为核心思想的虚拟桌面架构模型^[51]。

如图 2.7 所示，VDI 的主要原理是将终端的操作系统集中部署在数据中心的服务器上，并将桌面进行虚拟化。用户通过各自的终端设备，经由桌面传输协议与虚拟桌面进行连接^[52]，用户通过各自的设备就可以实现安装。

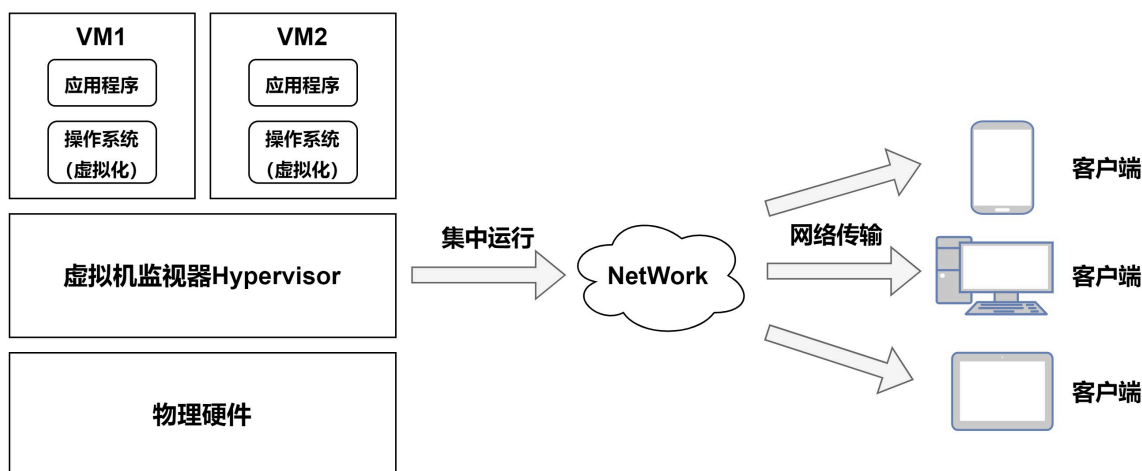


图 2.7 VDI 结构云桌面系统示意图

其特点是：所有数据全部由后端服务器统一运行存储，用户可通过优化后的网络连接协议。在性能上弹性化，不依赖本地终端，且拓展性极高。通过 VDI 架

构的设计,降低了桌面环境的管理复杂性,并实现对云桌面的按需资源分配,包括 CPU、内存、存储和其他资源,提高了桌面环境拓展和部署的灵活性。VDI 架构搭建云桌面系统,它有以下几个方面的优势^[53]:

第一,既定的操作模式不受影响。基于 VDI 的虚拟桌面为用户提供了一个全功能的桌面操作系统环境,与传统的本地计算机的环境极为相似,它不改变一直以来的操作方式。

第二,系统管理更加集中化。在云桌面服务器上,操作系统桌面被统一部署、管理和维护。管理员可以避免传统个人电脑所造成的广泛分散的管理问题,不必前往广泛分布的个人电脑现场进行操作和维护。

第三,VDI 架构使系统非常可靠。它具有动态迁移和自动错误隔离等功能,可以减少服务器或系统错误导致的停机时间。它继承了服务器虚拟化的优点,支持虚拟机的快速传输和复制,并提供方便的恢复解决方案。

第四,系统可拓展性高。当云桌面服务器集群的性能不能满足当前系统内所有注册用户的期望时,可以使用服务器虚拟化技术直接增加硬件资源,扩展虚拟化资源池的资源。当系统内的用户减少或需求减少时,一些设备或硬件资源可以撤回供其他系统使用,从而避免云桌面服务器集群资源大量闲置。

第五,降低终端设备购买和维护的费用。用户的终端设备不要求有很强的算力和硬件配置,其购买和维护的费用较低,物理结构也比标准 PC 设备简单。

基于以上五个方面的原因,本文采用 VDI 架构设计云桌面系统的服务端和客户端。

2.5 FFmpeg 音视频处理框架

FFmpeg 是一个用于音视频数据处理的技术框架,在音视频处理领域具有很大的影响力。因此,它经常被用于音视频有关的开发项目中。FFmpeg 具有许多处理音频和视频的功能,并支持大多数音频和视频编解码器标准^[54-56]。暴风影音、QQ 影音、Mplayer、KMPlayer 等知名播放器的内核均是 FFmpeg^[57]。

FFmpeg 还具有强大的可扩展性,可以手动添加第三方音频和视频编解码器。除了其强大的音频和视频处理能力外,FFmpeg 可以适应多种系统,在跨平台开发领域占有很大的市场。其主要包括以下重要模块^[58]:

AVUtil: 核心工作库,其它模块经常依赖该库做一些基本的音视频处理操作。

AVFormat: 文件格式和协议库,提供了与协议、解封装、封装相关的函数,其在处理音视频过程中,负责编写和读取封装数据的信息。

AVCodec: 编码和解码库。在使用过程中，可以调用 `avcodec_find_decoder` 函数来查找编码器和解码器。

AVFilter: 滤镜库，该模块提供了包括音频特效和视频特效的处理。

FFmpeg 不仅包括上述核心库、编解码库等，还包括设备交互库、音频重采样库和视频像素格式转换库等等^[59,60]。本文采用 **FFmpeg** 作为音视频数据处理的技术框架，并通过对该框架的优化，缩短云桌面直播教学中的延迟。

2.6 系统的业务场景分析

在高校的日常教学中，要求学生进行大量的计算机实验，不同的专业和课程对计算机环境有不同的要求。由于实验室房间内的设备数量通常是固定的，不能根据课程的实际需要快速增加，因此实验室管理人员通常会在房间内的每台计算机上安装多个操作系统和不同的专业软件，以满足不同实验的桌面需求。

在云实验室场景中，每个学生一个接一个地接收与后台匹配的虚拟桌面，系统和用户数据由云桌面服务器后台提供。通过将每个虚拟机环境彼此分离，即使出现故障，也可以快速恢复或直接迁移到新的桌面环境。机房中的物理服务器只需要安装虚拟化环境以及支持云桌面客户端的组件和传输协议，即可作为虚拟桌面的宿主机，并通过桌面传输协议将云桌面环境发送给客户端。按照用户的身份和权限，将系统的业务场景按表 2.1 进行分类。

表 2.1 云桌面系统业务场景分类

用户身份	使用场景	软件环境	系统权限
学生	运行云桌面 或进入教学直播间	云桌面客户端	低
教师	运行云桌面 管理直播间	云桌面客户端	中
系统管理员	对系统资源进行管理 和维护	后台管理系统	高

为了满足各种实验环境对计算机操作系统、软件环境和依赖项的需要，管理员需要为每种实验环境制作基础镜像。基础镜像创建完成后，还可以根据需要将课程相关的操作环境和软件依赖安装在基础镜像中，再将整个桌面环境复制在专用的储存池中，即保存了一个可以复制的云桌面镜像。学生可在存储池中使用复制该云桌面，达到快速部署、快速迁移、快速恢复的目的，大大提高了部署效率和维护成本。

学生在本地登录云桌面客户端并向服务器发送连接请求时，客户端程序从服务器上读取相对应的桌面信息，然后根据配置信息启动虚拟机，最后通过桌面传输协议将存储在服务器后台的云桌面资源发送到指定用户的客户端上。

教师除了和学生一样可以创建自己的云桌面之外，还可以额外拥有一个管理员身份，管理员身份可以通过可视化的管理中心监控物理宿主机和每一台云桌面虚拟机的运行情况，也可以对学生的云桌面进行快速备份或迁移方便同一管理和维护。

与此同时，教育部指出网络教育更应当注重信息传递和资源传递的优势，实现能够在使用云上直播教学时也可以充分地将优质教学资源同现代化教育平台相结合^[61-64]。针对上述要求，在云桌面系统也加入了线上直播教学功能，学生可以实时获取教师授课信息，教师进行课件演示、软件实践教学等。用户使用终端设备时，通过输入用户名和密码等信息接入到虚拟主机上的个人桌面，或者直接输入房间号和密码进入线上直播间。这样的云上拓展方式，激发了学生的学习兴趣，也有效地提高了数据安全性和资源利用率。

2.7 系统在功能和性能上的需求

2.7.1 学生用户对功能的需求

学生用户在功能上的需求如表 2.2 所示：

表 2.2 学生功能需求表

功能	功能描述
登陆客户端	通过用户名和密码登陆客户端软件
运行云桌面	在客户端软件中运行自己的云桌面
管理云桌面	调整桌面显示分辨率、缩放、截屏等
进入线上直播间	在客户端软件在进入直播间
插入 U 盘设备	通过 USB 重定向识别 U 盘

学生用户云桌面客户端用例图如图 2.8 所示：

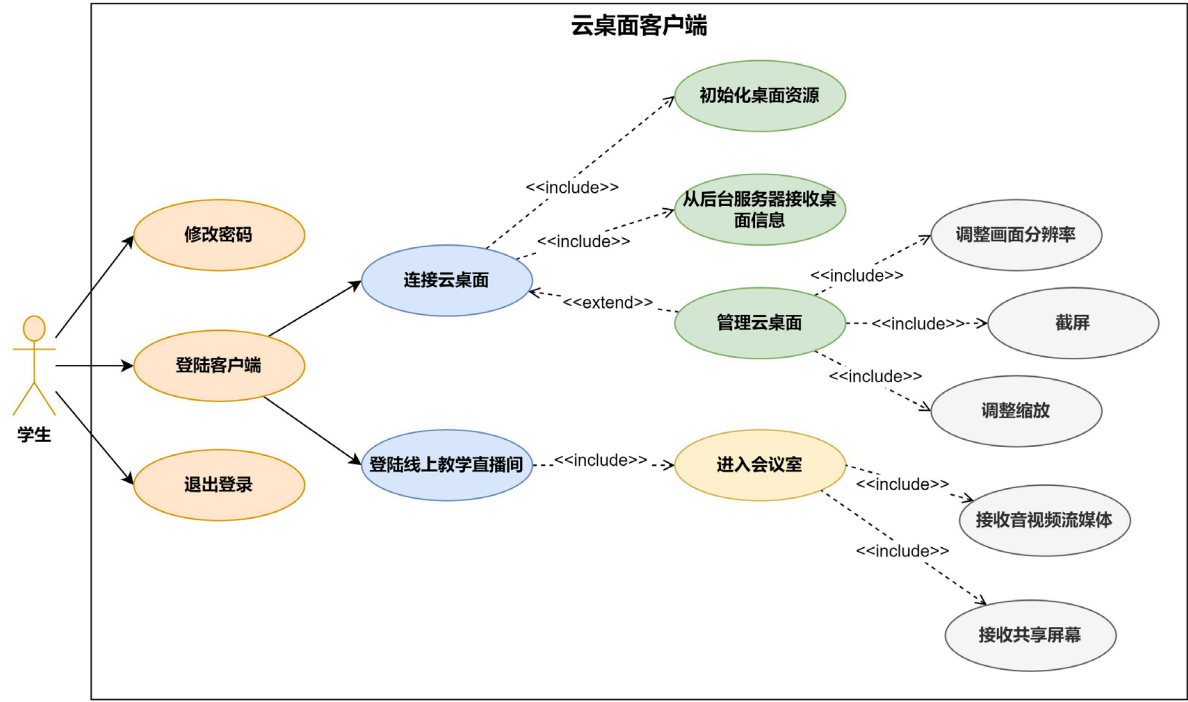


图 2.8 学生用户用例图

2.7.2 教师用户对功能的需求

教师用户在功能上的需求如表 2.3 所示：

表 2.3 教师功能需求表

功能	功能描述
登陆客户端	通过用户名和密码登陆客户端软件
运行云桌面	在客户端软件中运行自己的云桌面
开启教学直播间	通过线上会议室进行授课
教学管理	对授课的班级进行管理，包括学生进入直播间的权限、共享桌面屏幕等
教师端后台管理系统	资源监控、云桌面镜像复制迁移等

教师用户云桌面客户端用例图如图 2.9 所示：

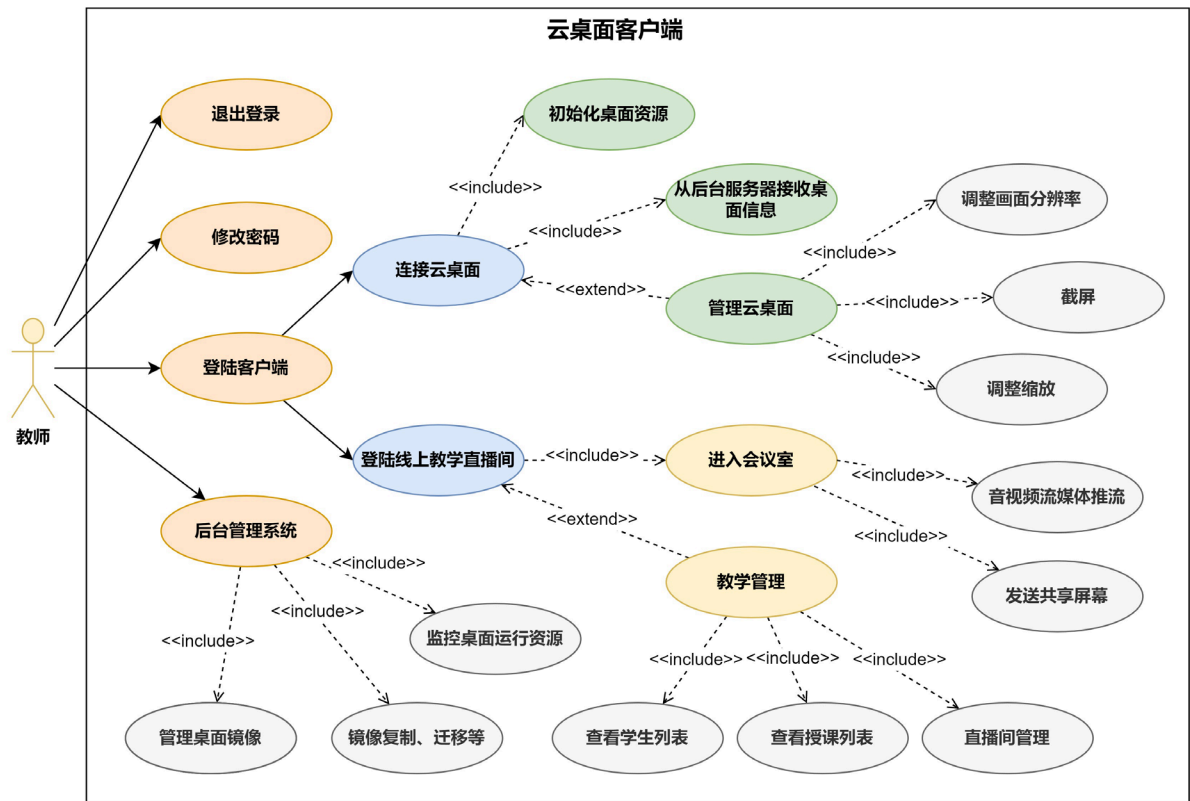


图 2.9 教师用户用例图

2.7.3 管理员用户的权限需求

管理员用户需要拥有整个云桌面系统的管理权限，其在功能上的需求如表 2.4 所示：

表 2.4 管理员功能需求表

功能	功能描述
登陆	管理云桌面系统后台和客户端
修改或重置密码	修改管理员账户密码或重置学生教师用户密码
用户管理	对系统内所有用户进行管理，包括创建新用户、删除用户、更改用户权限等
云桌面资源管理	修改用户云桌面的配置参数，包括 CPU、内存、磁盘等

表 2.4 续 管理员功能需求表

功能	功能描述
云桌面批量管理	对桌面进行统一管理，包括批量创建桌面、批量删除桌面、查看桌面运行状态和更新桌面配置信息。
储存池管理	启动和终止系统存储池，评估可用存储池的容量，并确定每个存储池中存在的镜像数量并管理系统存储池。
镜像管理	对存储池中的系统镜像进行管理，包括上传镜像、删除镜像、检查镜像的使用状态

管理员用户系统控制台用例图如图 2.10 所示：

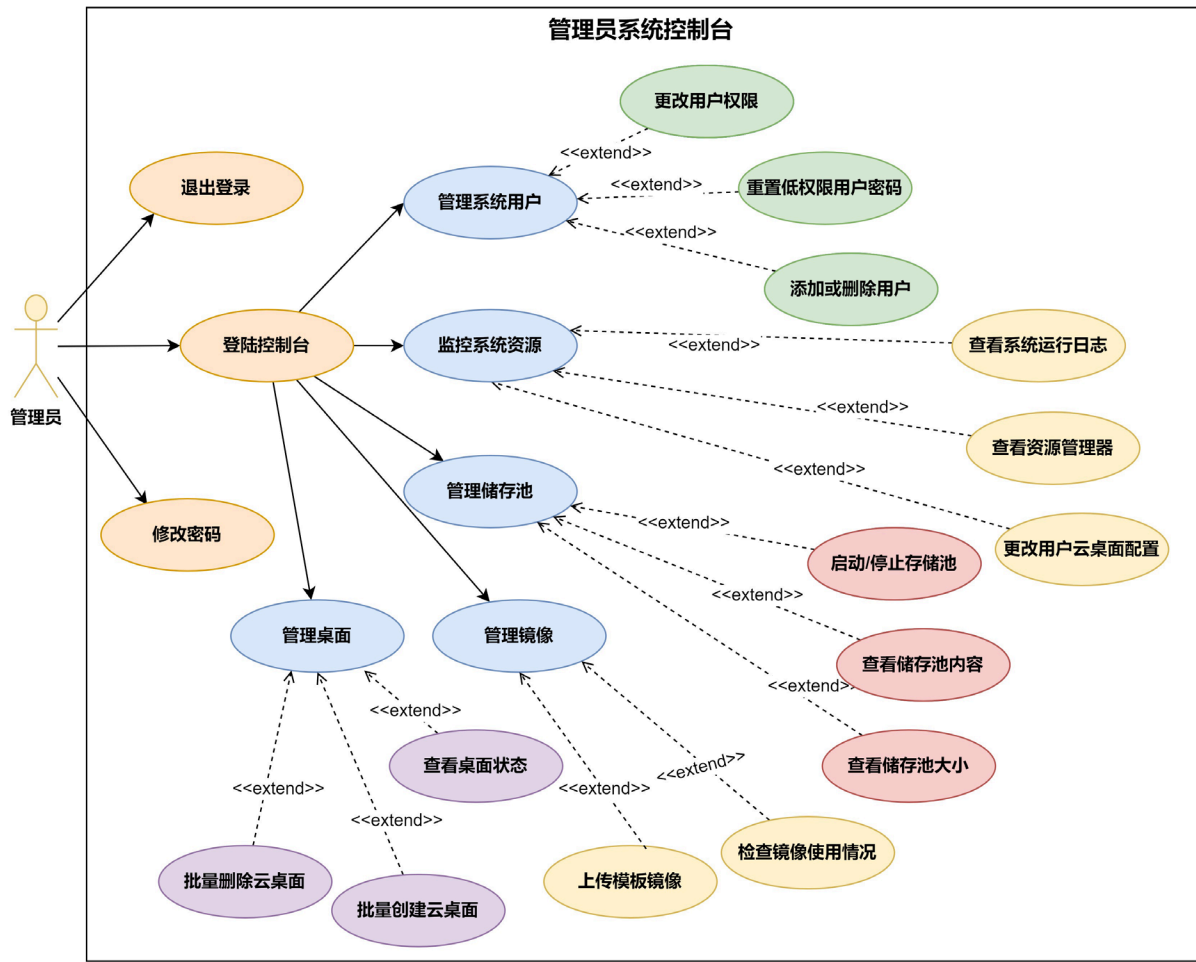


图 2.10 管理员用户用例图

2.7.4 系统在性能上的需求

(1) 高可用性

一些可变因素包括网络环境和服务器性能等，都会对云桌面系统的正常运行有影响，设计时应该考虑上述外在干扰，以提高系统的稳定性。同时云桌面后台数据中心的硬件资源使用情况可以反映出整个云桌面系统的资源利用率。由于实验室的硬件资源有限，系统必须在确保稳定性的同时接受尽可能多的用户，以最大限度地利用资源。

(2) 高响应速度

部署在本地环境上的桌面环境可能需要几分钟的时间来建立并打开操作系统然后再做出应用层上的反应，速度缓慢甚至有时会造成系统崩溃，这是一种很不好的用户体验，因此，高响应速度对于云桌面系统来说尤为关键。由于在云桌面系统的服务端中有很多模块会同时运行，而且它们中的大多数必须通过消息机制进行沟通，因此，为了提高系统的响应速度，确定每个模块之间的响应时间也是至关重要的。其次，当用户在请求云桌面时应将资源池中的实例分配给他们，这减少了创建容器的时间也改善了用户体验。

(3) 易管理性

对于不同的使用角色，云桌面客户端都有一个单独的用户界面与之对应。用户登陆客户端后就可以立即找到多种功能入口。此外，系统还提供一个基于 Web 的云桌面后台管理系统。具有管理员权限的教师或管理人员可以使用浏览器登录云桌面后台管理系统，保证了系统的简单易用。

(4) 易维护性

后台管理系统能够实时记录服务端和客户端的运行日志，记录系统关键时间节点的操作以及反馈结果，帮助维护人员更好地进行系统维护或故障排查。此外，管理员可以很容易地恢复或复制用户的桌面环境，或者在他们当前的桌面出现故障或不能正常工作时，为他们快速设置一个全新的桌面环境。

(5) 强安全性

云桌面系统必须保护桌面数据安全，同时也要为使用者提供与其身份对应的控制权限和外设接入权限。当不需要外部设备通过云桌面终端连入云桌面时，管理员可以禁用 USB 重定向功能以拒绝云桌面的外设连接权限，阻止 U 盘等外部设备的访问。管理员还可以禁止云桌面对外部网络的访问，只使用虚拟局域网进行连接。

2.8 系统总体设计

本文设计的云桌面系统利用 Proxmox 虚拟化环境（Proxmox Virtual Environment，简称 Proxmox VE）搭建虚拟机监视器。Proxmox VE 是一个运行虚拟机和容器的平台环境，它的内核基于 Debian Linux 系统，属于宿主型虚拟机监视器 Hypervisor。其扮演的就是 2.1 节介绍的虚拟机监视器 Hypervisor 以及虚拟机管理工具 libvirt 的角色。

它可以利用 KVM 技术虚拟出 Linux 或 Windows 虚拟机，也可以利用 LXC 容器技术创建 Linux 容器。如图 2.11 所示，从整个虚拟化解方案的逻辑层看 Proxmox VE 处于物理宿主机之上，位于 VM 虚拟机之下。

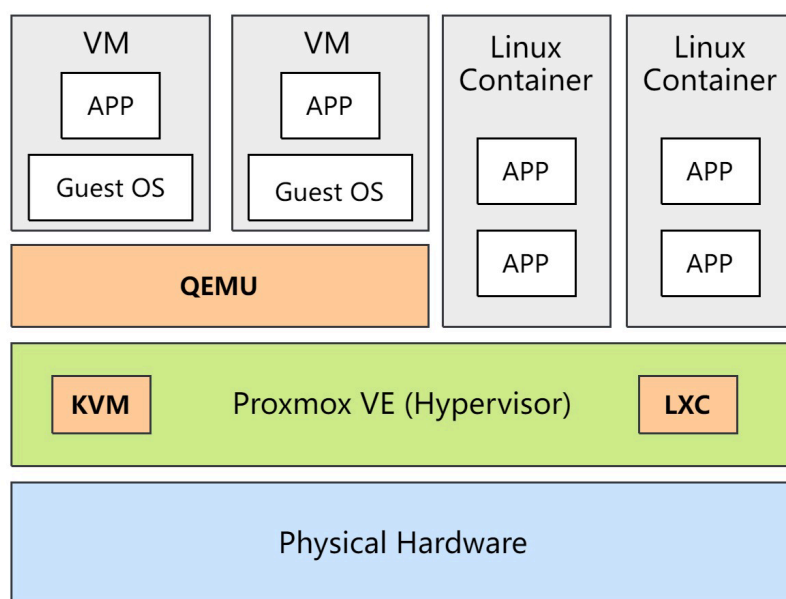


图 2.11 Proxmox VE 虚拟化逻辑架构图

Proxmox VE 的优势是去中心化、超融合、高可用、部署低成本、易于实施管理；其劣势是只有 Proxmox Server Solutions 一家厂商在维护，生态方面稍有欠缺。但由于其开源特性，允许开发人员根据部署情况和功能需求更改对应的组件和性能，对开发和运维人员友好度高。Proxmox VE 主要面向中小型虚拟化场景或私有云场景，因此能够满足本文所述的云桌面系统要以高校云办公和云实验室为主要目标的需求。

2.8.1 系统逻辑架构设计

基于 Proxmox VE 和 VDI 架构的云桌面系统，以 VDI 架构云桌面的“集中管理，集中运行”的基本设计为宗旨，采用了 C/S 架构模式。服务器端部署在物理

宿主机中，可视化 Web 后台控制台、SSH、Shell 和 Proxmox 被用来简化服务器集群的维护，并提供外部调用 API 以处理客户端和 Web 网络端业务需求。

当用户在本地终端上登录云桌面客户端并请求个人云桌面数据时，服务端将存储在物理机中的相关桌面环境数据发送给客户端，并通过 SPICE 协议将数据可视化地“投影”到客户端上。云桌面整体架构如图 2.12 所示。

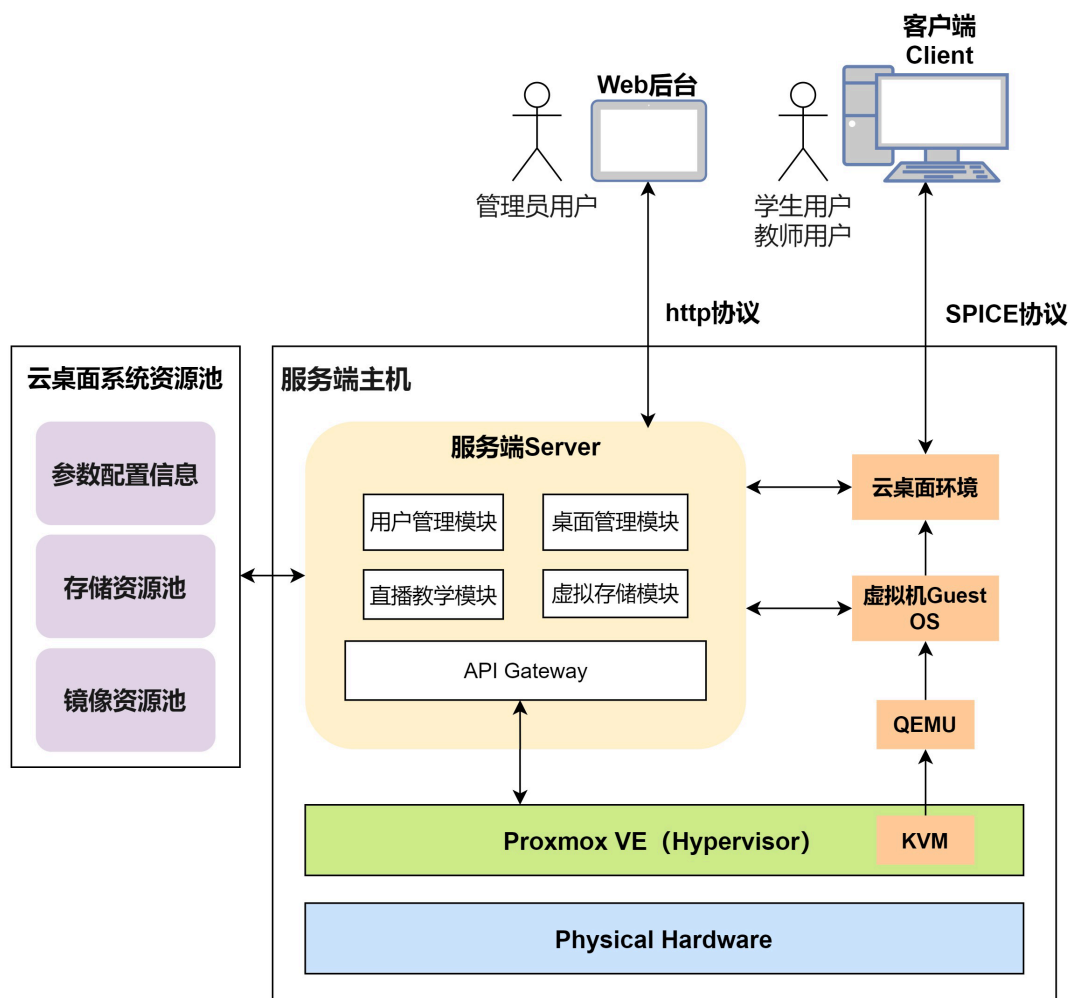


图 2.12 系统逻辑架构图

本文设计的云桌面系统主要有三个部分组成：Server 服务端、Client 客户端、Web 后台控制台。

（1）服务端 Server

服务端 Server 程序是整个云桌面系统的服务端程序，在 Proxmox VE 虚拟化环境中做二次开发，由于其开源特性，允许开发人员根据部署情况和功能需求更改对应的组件和性能。服务端程序部署在云桌面物理宿主机上，其物理机由单节点或多节点的 Ceph 集群组成。开发人员、管理人员可以通过可视化 Web 后台控制

台、SSH、Shell 对物理集群进行访问。此外，服务端还创建了资源池，将计算、存储资源池化，并通过暴露的 API 接口与下层 Proxmox VE 相互通信，以处理客户端和 Web 网络端业务需求。

（2）客户端 Client

客户端 Client 是云桌面系统的客户端程序，该程序运行在用户的终端机之上。云桌面 Client 客户端的功能主要是两大块：首先是连接个人云桌面，利用 Server 服务端提供的 API 接口将用户操作和桌面数据发送到服务器，并从服务端中收集桌面所需的各类数据通过 SPICE 桌面传输协议传输到客户端显示给用户，从而完成云桌面系统不同的业务活动；其次是承接线上直播的功能，教师用教师客户端将自己的音频数据、视频数据或者是共享屏幕向服务端的流媒体服务器进行推流，通过 RTMP 或 RTSP 流媒体协议将这些数据包分发给学生客户端，学生端再通过接收、解封装、解码音视频数据包，得到完整的流媒体数据，从而实现完整的线上直播服务。

（3）Web 后台控制台

基于 Web 的云桌面后台控制台，可以被认为是云桌面系统 Server 服务端的 Web 组件。该组件提供了一个画面简洁且对用户友好的操作界面，可以将管理员基于浏览器的操作转换为对服务器端不同功能的请求，然后服务器端进行响应并显示结果。

2.8.2 系统物理架构设计

如图 2.13 所示，云桌面系统的物理组成包括服务端的存储节点、计算节点、流媒体处理节点、服务端主机，客户端的用户客户机以及其他的网络连接设备。按照服务端设备的用途，将服务端集群主要分为四个部分：

（1）储存节点

存储节点主要用于创建系统的储存中心，创建一个统一的储存资源池，存放系统需要的所有类型的镜像和虚拟桌面磁盘文件。因为储存中心的镜像资源被虚拟化平台池化成一个统一的资源池，所以使得服务端 Server 程序的调用方式和储存中心的实施策略是可以互相脱钩的。这样就可以在建设储存中心时根据实际情况灵活选择不同的技术方案。

（2）计算节点

计算节点基于 Proxmox VE 虚拟化环境，主要作为云桌面系统中运行虚拟机的宿主机。云桌面的计算能力和性能取决于计算节点。

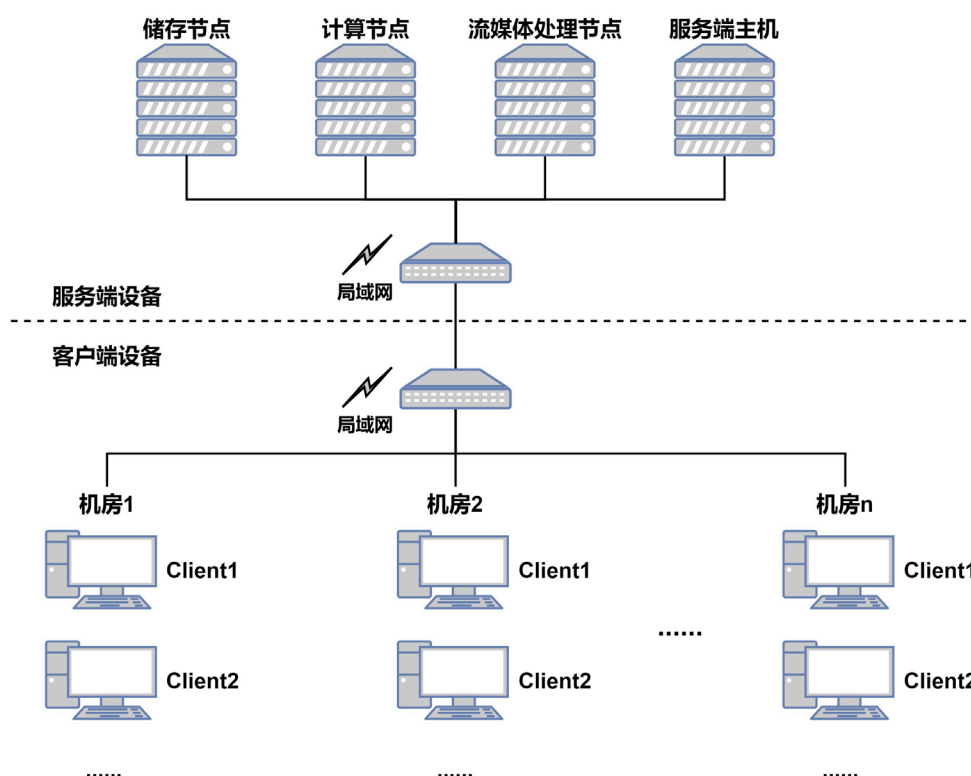


图 2.13 系统物理架构图

（3）流媒体处理节点

流媒体处理节点主要用于用户打开线上教学功能时，对音频流和视频流的处理和传输。服务器上编码器可以根据需要添加，一般编码器支持推 RTMP/UDP 流，如果集成 SRS 或 ZLMediaKit 流媒体方案后，则可以同时支持多种数据流协议，例如支持 RTMP、WebRTC、HLS、HTTP-FLV、SRT 和 GB28181 等协议。

（4）服务端主机

服务端主机主要用来运行服务端 Server 程序。另外，系统运行所需的配置文件和日志文件也会保存在服务端主机上。

上述四种不同类型的服务节点相互并不冲突。例如，一台服务器既可以作为计算节点的服务器也可以作为储存节点的服务器，还可以作为流媒体处理的服务器。只是为了保证系统的高可用性和可扩展性，建议将其划分为四组不同的服务器，以降低系统宕机的风险。并在设备需要扩容时，可以根据需要为各个集群增加服务器数量达到横向扩展的目的，以提高整个系统的承载能力。

2.8.3 系统各模块的分类

如图 2.14 所示，云桌面系统按照功能可分为：连接个人云桌面和线上直播教学两个主要功能。功能模块可划分为：用户管理模块、桌面管理模块、虚拟存储模块、直播教学模块四大类。

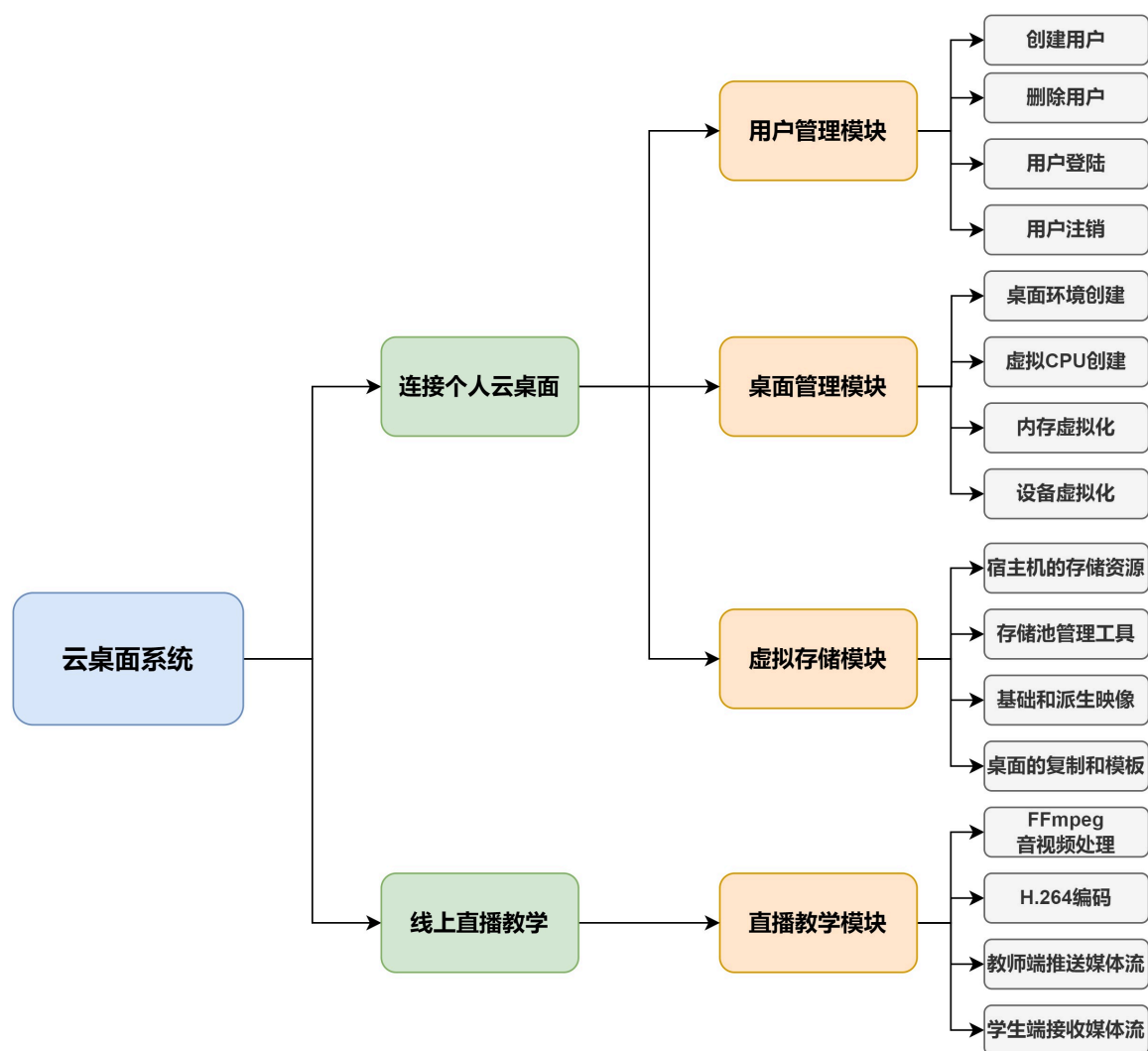


图 2.14 云桌面系统功能模块图

2.9 本章小结

本章首先对虚拟化技术进行了整体概述，介绍了虚拟化的概念和分类。先针对操作系统虚拟化的实现方式和虚拟化层 Hypervisor 的分类进行了具体的介绍，再对 kvm-qemu 虚拟化底层体系做进一步阐述。之后介绍了桌面传输协议 SPICE、VDI 虚拟桌面基础框架、FFmpeg 音视频处理框架以及它们的特点，并结合它们各

自的优势说明了本文设计的云桌面系统为什么采用这些技术。通过对这些技术的研究，为之后的系统设计和实现打下了理论基础。

再针对云桌面系统的主要业务场景进行了介绍，提出云桌面系统以高校云办公和云实验室为主要目标。其次按照不同用户的分类，分析了云桌面系统的各类需求，并对系统的业务逻辑进行抽象与分析。

之后提出基于 Proxmox VE 虚拟化环境的云桌面系统整体架构，包括系统逻辑架构和系统物理架构。最后将整个系统按照模块划分设计，第 3、4 章将针对上述各功能各模块，进行更详细的设计，解释其详细设计与实现。

第3章 系统线上直播教学功能的设计与应用

本文设计的云桌面系统加入了学生端和教师端的线上直播功能。该直播教学模块也弥补了国内云桌面厂商在线上直播教学功能上的空白。教育部指出网络教育更应当注重信息传递和资源传递的优势，实现能够在使用云上直播教学时，也可以充分地将优质教学资源同现代化教育平台相结合。云桌面系统的设计初衷是将互联网与高校教学办公相结合，实现校园内智慧云办公和线上直播教学等功能。

学生可以实时获取教师授课信息，教师进行课件演示、软件实践教学等。用户使用终端设备上的客户端时，通过输入用户名和密码等信息接入到虚拟主机上的个人桌面，或者直接输入房间号和密码进入线上直播间。这样的云上拓展方式，激发了学生的学习兴趣，也有效地提高了数据安全性和资源利用率。直播教学模块功能示意图如图 3.1 所示。

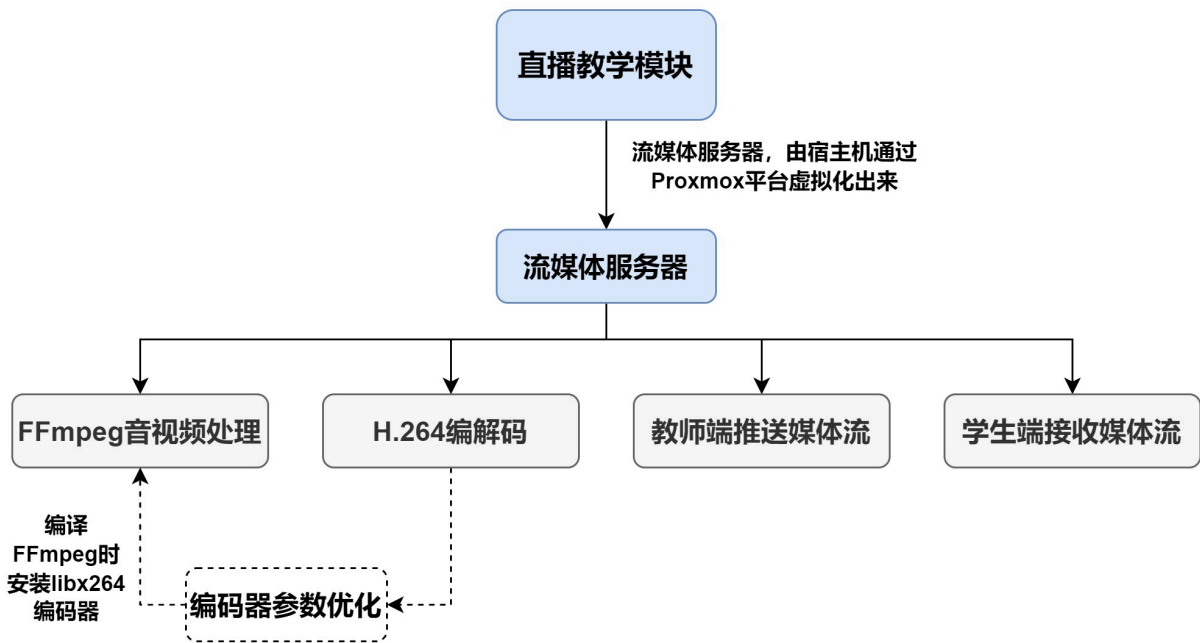


图 3.1 直播教学模块功能示意图

在进行线上直播的过程中，通常是一个老师面对多个学生进行授课。在这种情况下，流媒体服务器应保持足够的数据发送处理能力。因此本项目选择将流媒体服务部署在由 Proxmox 创建的 Ubuntu 16.04 系统上，作为云桌面系统线上直播功能的流媒体服务器。在流媒体服务器中搭建 SRS3.0 流媒体服务组件、FFmpeg 音视频框架，并通过对框架和编码器的优化，缩短直播中的延迟，增加传输效率。

3.1 H.264 编码技术

鉴于 H.26x 系列中的 H.264 视频编解码标准常用于实时流媒体视频直播系统，所以在云桌面系统的线上直播模块中采用 H.264 视频编解码标准对推流端的音视频原始数据进行编码，收流端再将数据解码并通过设备进行播放。

H.264 编码标准和之前的编解码标准一样都使用了基于块匹配的混合编码技术，但它在分层划分方面比早期编码标准有了显著的改进。图像层、GOB 层、宏块层和块层是 H.261 到 H.263 标准码流结构中从上到下的四层。而 H.264 标准则针对视频数据在网络上传输的需要在功能上分为两层，即视频编码层（VCL）和网络抽象层（NAL），如图 3.2 所示。

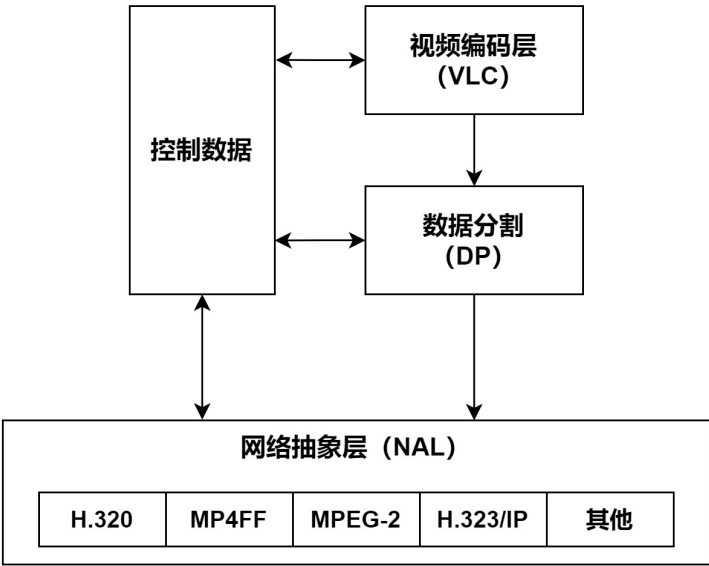


图 3.2 H.264 结构层次图

VCL 层，也被称为视频编码层，在 VLC 中使用各种编码方法，包括以块为基本单位的运动补偿等混合编码技术，对视频数据进行压缩和编码。

由于编码后的数据按照 NAL 标准规定的方式包含在 NAL 单元中，因此可以在网络传输中统一化数据格式，使其更容易与网络协议连接和传输。一个 NAL 单元由 NAL 协议头 Header 部分和包含编码后视频数据的原始 data 段（Raw Byte Sequence Payload，原始字节序列有效载荷，简称 RBSP）两部分组成。Header 头部一般保存 RBSP 部分的简略解析信息和整个包的初始化信息，包括帧数据信息、信道的信令和时间戳信息等。

3.2 x264 编码器系统优化

x264 编码器是一种实现了 H.264 编码标准的开源编码器。由于其开源特性，它拥有很多维护群体。自 2003 年以来，它经历了多次修订，并不断升级和改进。由于其优越的编码性能和丰富的参考资料，它已发展成为最受欢迎的编码器。H.264 的下一代编码标准是 HEVC，它理论性能上优于 H.264，但由于其开源项目上工程性能较差，在视频播放和流媒体传输领域广泛使用的还是以 x264 编码器为代表的 H.264 编码标准。

x264 作为性能最好的 H.264 标准编码器，实现了标准中的主要编码算法。随着 x264 的应用越来越广泛，其优化方向也越来越多。有的使用汇编语言在 x264 中编写比较耗时的功能模块，有的将 x264 移植到不同类型的硬件平台上，利用硬件加速的优势实现编码性能的提升。

3.2.1 编码器的编码流程

x264 编码器是从程序 main 入口开始执行。编码器在进行编码前，通过设置命令行参数，给程序传递了一些编码器所需的控制参数。首先初始化编码器控制参数 x264_param_t，然后使用 X264_param_default(& param) 函数为控制参数分配默认值。

然后调用 Parse 函数处理命令行中输入的参数，设置参数后进入编码函数 ret=encode(& param, &opt) 中。编码函数的返回值 ret 用于指明编码是否成功。在这个函数中首先根据编码器的输入参数来进行初始化，并对要编码帧的数量进行计算。

使用 x264_picture_alloc 函数创建内存空间，用于存放视频文件每一帧画面的 YUV 数据，其原始数据格式为 YUV420。在完成这些操作之后，确定要编码的总帧数 i_frame_total，并且以此作为控制条件，在编码帧数小于 i_frame_total 的状态下对原始数据格式图像的各帧进行编码。除了编码操作之外，还另外使用了 NAL 包格式来封装每一帧的编码数据，以便帧数据通过网络传输。图 3.3 为 x264 编码器的执行流程图。

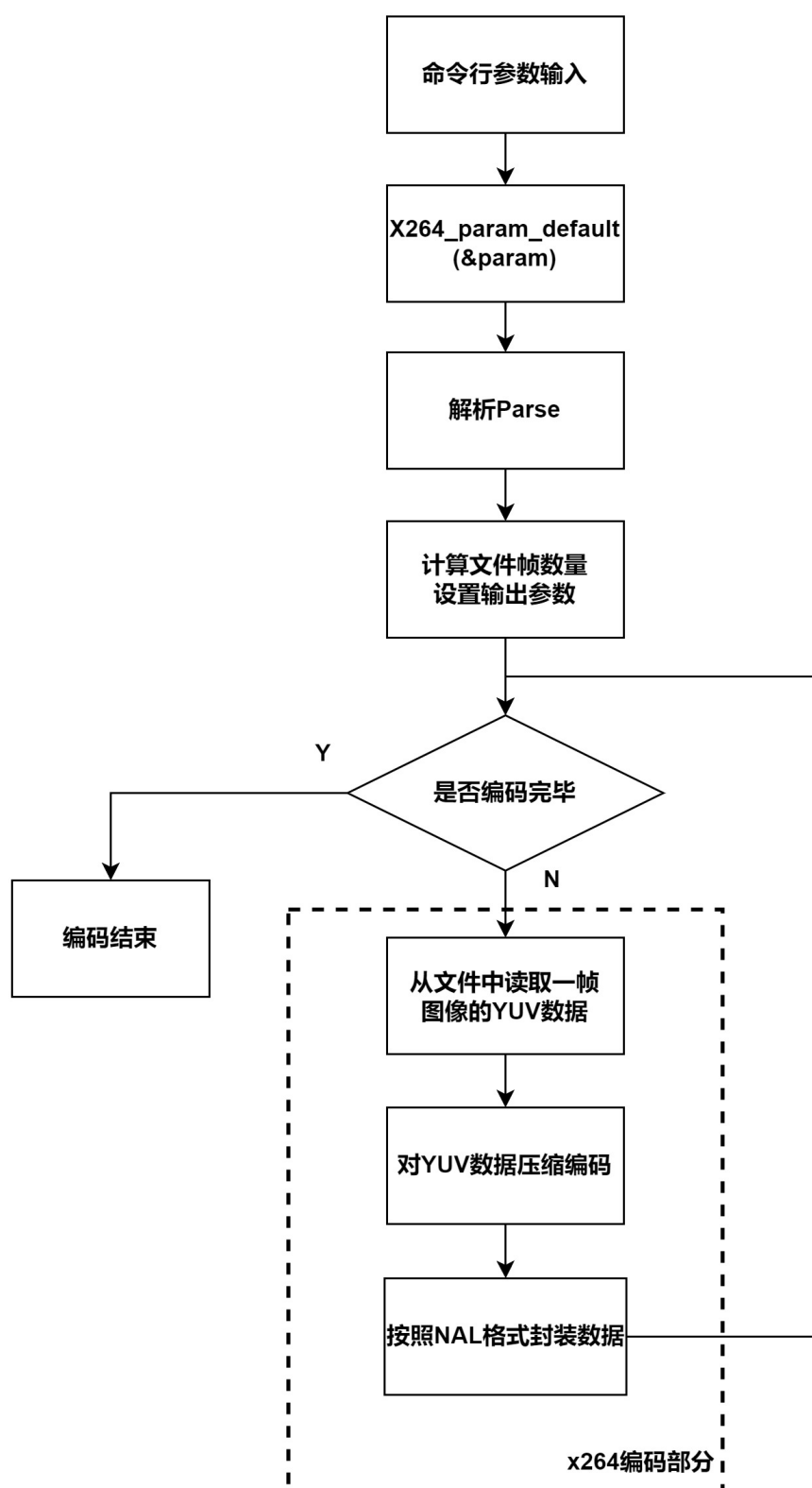


图 3.3 x264 编码流程图

3.2.2 编码器的算法可定制模块

H.264 编码标准是不同编码技术的组合,共同实现高效的编码过程,包括场景切换检测、图像组调整、去噪预处理、变换与量化、逆量化与逆变换、环路滤波、运动预测、运动补偿等一系列其他编码技术。虽然有些技术在视频编码标准中有具体的规定,但只描述了一些编码技术传输的数据流的语法内容以及编码和解码过程。这使得用户可以根据实时编解码的要求,为给定的编码技术选择最佳模式,以满足不同的应用需求。正因为编码参数有不同选项才有了编码技术的不同模式。根据编码参数的作用,视频编解码可以划分为不同的算法定制模块。如图 3.4 所示,其中标黄部分是可以根据应用场景和项目修改的算法模块。

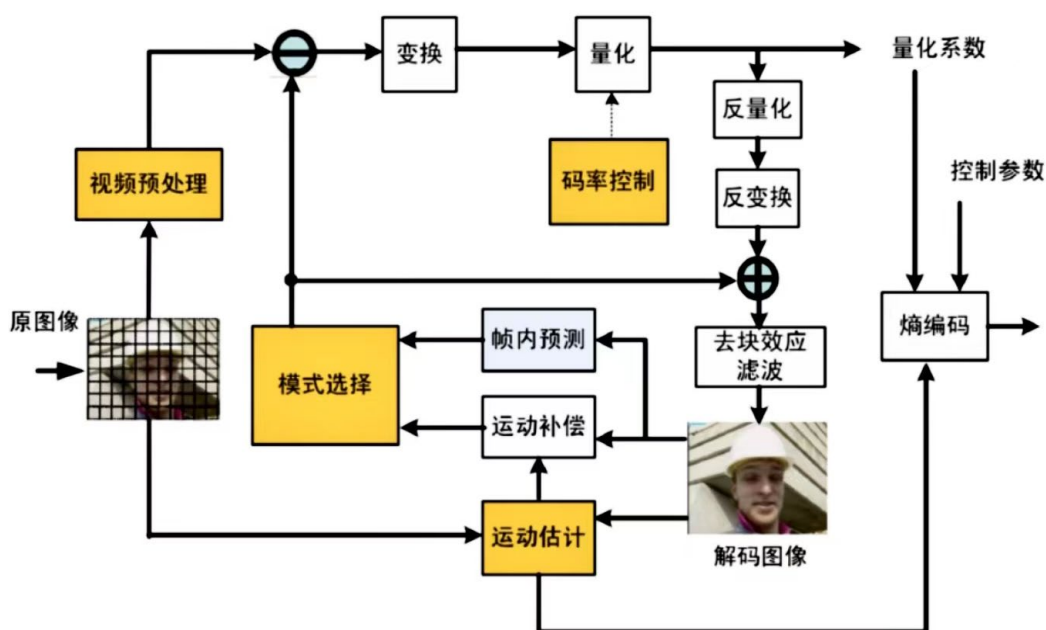


图 3.4 x264 编码器的算法模式流程图

（1）视频预处理模块

在编码器端收到原始数字视频图像后对视频画面进行处理,并分析视频的复杂性、编码效果和主观视频质量;

（2）码率控制模块

主要功能是为编码和量化模块提供量化步骤,控制输出流的码率,在某些允许的条件下优化编码率失真的性能,并显著提高视频压缩的质量;

（3）运动估计模块

鉴于视频信号的空间和时间维度之间的联系,运动预测算法对视频编码尤为关键。运动预测的效率直接受到运动矢量预测准确性的影响,因此运动估计模块必须找到解决这个问题的方法。

(4) 模式选择模块

在 H.264 和 AVS 等编码标准中，帧内和帧间预测过程中使用各种编码模式，这些模式都对编码结果有影响。提高编码效率的关键是基于不同的视频序列的特征选择不同的模式。模式选择模块在这其中发挥了作用。下一小节将探讨对于本文设计的直播教学功能，在编码器参数、运动估计算法等方面的优化和定制。

3.2.3 编码器的参数优化

由于其强大的功能和兼容性，x264 编码器具有广泛的应用。VLC、FFmpeg 和其他软件都使用了 x264 编码器。目前在视频监控、视频会议和其他实时编码场景中，x264 都是最受欢迎的编码器。

编码器的关键性能指标是 PSNR（峰值信噪比，可以反应视频的质量）、码率、帧率和编码速度。通过采用不同的控制参数，可以提高编码器在不同指标上的性能。例如，为了提高视频的 PSNR 指标，可以选择不同的运动估计算法或不同的编码控制参数，但这可能会导致视频码率的增加。换句话说，一个指标的提升可能会导致其他指标的表现更差，所以需要达到一个平衡。而在实际应用中，根据不同的运用场景，对不同的性能指标有着不同的要求。此可以通过改变不同的编码控制参数，实现优化编码效率的目的。

本文根据自身的需求，通过选择合适的 x264 编解码参数，改进 FFmpeg 框架的功能设置，以此降低直播中推送流和接收流的延迟。由于本文设计的直播需求为实时编码，且压缩的音视频分辨率为 720P，所以使用了 Main Profile 的压缩档次。为了在保持图像质量的同时，找到最佳的控制参数来提高系统性能，对量化步长参数 Qstep、参考帧数目、搜索模式和动态预测进行了研究和测试。进行优化研究的测试序列和平台环境如表 3.1 所示。

表 3.1 测试环境

测试序列	总编码帧数	序列分辨率	操作系统	测试编码器	CPU
Pigeon.yuv	30	1280×720	Windows 11	X264	AMD Ryzen7 5800H

3.2.3.1 量化步长研究

在量化过程中实际用于计算的参数是 Qstep 量化步长，其范围为 0 到 51。该数字越小，图像质量越好。通常是在保证可接受视频质量的前提下选择一个最大的值，为了找到一个更适用于直播实时编码的 Qstep 值，本文利用命令行预先设置

编码器的量化步长，以下对 Qstep 设置不同的值，通过峰值信噪比 PSNR 和码率等指标来查看对系统性能的影响，测试结果如表 3.2 所示。

表 3.2 Qstep 测试结果

Qstep	PSNR/dB	码率/ (kb/s)
1	62	101,565
10	51	56,325
20	43	23,317
30	35	7,855
40	29	2,552
50	23	760

图 3.5 中的六张图像分别对应 Qstep 设置不同参数值时的编码图像。



(a) Qstep 为 1 的编码图像



(b) Qstep 为 10 的编码图像



(c) Qstep 为 20 的编码图像



(d) Qstep 为 30 的编码图像



(e) Qstep 为 40 的编码图像



(f) Qstep 为 50 的编码图像

图 3.5 测试 Qstep 时编码图像

PSNR 值和视频编码的码率可以从一定程度上反映画质的好坏，由表 3.2 统计出的数据可以得出，对于此视频序列，随着 Qstep 的增大 PSNR 值逐步减小，码率也相应降低。但由于码率降低，编码速度有相应提升。但是当取值在 30 以上时，视频的 PSNR 值下降得很快，这表明画面质量下降很明显。通过测试可以得出结论，Qstep 的配置对本文研究对象的实时视频画面有显著影响。在保证视频质量的前提下，可以通过设置适当的 Qstep 值提升编码速度。经过多次测试分析，本文将量化步长 Qstep 值设为 27。

3.2.3.2 参考帧数目研究

H.264 标准为帧间预测创建了两个参考帧队列，用来保存用于预测的解码帧。当使用较少的参考帧时，所需的计算量也较少，但预测精度会降低，影响画质和码率。在 x264 编码器中对于它的测试结果，如表 3.3 所示。

表 3.3 参考帧数目测试结果

参考帧数目	PSNR/dB	码率/(kb/s)
1	36.086	8,285.91
4	36.105	7,895.71
7	36.094	7,808.22
10	36.096	7,799.91
13	36.092	7,789.64

由表 3.3 中的数据可以看出，当参考帧的数目逐渐增加时，码率会下降，但是当参考帧数目超过一定值时，码率下降的效果就不明显了，同时对 PSNR 的影响几乎没有。结合这一测试的结果，将本文项目中的参考帧数目设为 4。

3.2.3.3 运动补偿和运动估计算法研究

因为运动是连续的,所以具有相同时间间隔的相邻帧中的大多数元素具有相同的特性。由于各帧的时间关联性,就可以实现对图像的压缩。当一个物体移动时,它通常会从图像中的一个点移动到另一个点。当移动范围很小时,图像背景将具有高度的相似性。可以利用这一特性,选择参考帧并建立坐标系,并使用该属性对参考帧和预测帧进行比较,运动量可以通过运动物体在参考帧和预测帧中的坐标计算出来,并且当前图像也可以通过运动量与参考帧的信息组合来表示。

这样的视频压缩算法过程被称为运动补偿。这样的算法通常包括几个步骤:运动估计(即搜索运动向量)、基于运动补偿的帧间预测、帧间差值的导出。其中运动估计是比较重要的一步,它决定了系统计算开销的大小。对运动矢量的搜索实际上是一个对应点问题,或者说是帧之间的匹配问题。

如果将运动向量限制水平和竖直上的位移在 $[-p, p]$ 的范围内,就可以得到一个大小为 $(2p+1) \times (2p+1)$ 的搜索网格窗口,窗口中则共有 $(2p+1)^2$ 个像素点。宏块的中心点 (x_0, y_0) 可以位于这个 $(2p+1) \times (2p+1)$ 网格中的任意一个点上。用 (x, y) 表示目标帧中某个宏块左上角像素点在一个画面帧中的坐标,用 (i, j) 表示运动向量,这个运动向量就是从目标宏块到参考宏块的位移。将目标宏块中的像素点用 $C(x+k, y+l)$ 表示,则在参考帧中通过运动向量的位移求出的对应像素点为 $R(x+i+k, y+j+l)$ 。其中 k 和 l 分别为宏块中的水平索引和数值索引。

运动向量 (i, j) 的平均绝对值差(MAD值)可用式(4-1)计算:

$$MAD(i, j) = \frac{1}{N^2} \sum_{k=0}^{N-1} \sum_{l=0}^{N-1} |C(x+k, y+l) - R(x+i+k, y+j+l)| \quad (4-1)$$

N 是宏块的大小,实际上运动向量搜索的问题可以转化为一个最优化问题:

$$(u, v) = [(i, j) | \min MAD(i, j), i \in [-p, p], j \in [-p, p]] \quad (4-2)$$

$[-p, p]$ 是运动向量的搜索窗口,在上述分析中,用了平均绝对差值这个指标来衡量运动向量的好坏。

同样,不同的搜索策略将对运动估计的计算规模产生影响,进而对视频压缩精度和处理速度产生影响。菱形搜索、正六边形搜索、可变半径六边形搜索和全局搜索是x264中可以指定的四个参数值,可以在x264中进行配置。由表3.4的数据可以看出,对于测试序列,使用全局搜索算法时运动估计的计算量最大,编码速度最低。采用菱形搜索时编码速度最快,而视频的PSNR值和码率没有明显的

降低。因此在本文项目中使用菱形搜索算法,可以保证视频画面质量的同时,获得较快的编码速度。

表 3.4 搜索算法测试结果

搜索算法	菱形搜索	正六边形搜索	可变半径六边形搜索	全局搜索
PSNR/dB	36.026	36.041	36.012	36.086
码率/(kb/s)	7887.42	7895.69	7828.67	7815.78
编码速度 (Frames Per Second)	94.71	87.93	78.95	65.63

从前面的测试中可以看出,量化步长、参考帧数目和运动搜索算法都对编码性能有很大影响。PSNR 在很大程度上受量化步长的影响。本测试的主要目标是在保持视频画面质量的同时,提高系统的编码速度和码率。通过考虑各种因素对系统性能的影响,并根据研究测试的标准适当地调整参数,可以提高视频编码性能。

3.3 编译 FFmpeg 时安装 x264 编码器

因为 FFmpeg 具有强大的可扩展性,可以手动添加第三方音频和视频编解码器。所以可以在流媒体服务器编译安装 FFmpeg 时,添加上一节已经修改好参数的 x264 编码器。要求编译 ffmpeg 时添加参数: `--enable-gpl --enable-libx264`。

将 x264 编码器源码放置 FFmpeg 工作目录 `ffmpeg_sources` 下,操作路径及命令如下:

```
cd ~/ffmpeg_sources
cd x264 && \
PATH="$HOME/bin:$PATH" PKG_CONFIG_PATH="$HOME/ffmpeg_build/lib
/pkgconfig" ./configure
--prefix="$HOME/ffmpeg_build" --bindir="$HOME/bin" --enable-static \
--enable-pic && \
PATH="$HOME/bin:$PATH" make && \
make install
安装完成后即可返回。
```

3.4 本章小结

本章针对云桌面系统的直播教学模块进行了详细设计和实现。主要包括 H.264 编解码技术的介绍、编码器可定制模块的分析、提出了针对本文项目的 x264 编码器参数优化方案和云桌面系统线上直播教学功能的一些技术探讨和模块优化。

第4章 系统连接个人桌面功能的设计与应用

为了实现用户登陆、连接和使用个人桌面功能，本章结合用户管理模块、桌面管理模块、虚拟存储模块设计了云桌面系统中的个人桌面功能。

4.1 用户管理模块设计与实现

用户管理模块用于管理所有类型的系统用户，包括创建用户、删除用户、登录用户和注销用户。用户管理模块的功能示意图如图 4.1 所示。

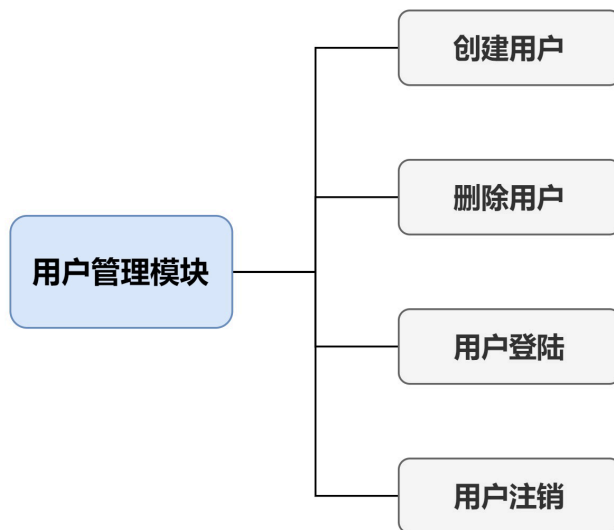


图 4.1 用户管理模块功能示意图

4.1.1 用户的创建与删除

用户管理模块的基本功能之一是创建和删除用户，这可以由管理员连接到 Web 端的云桌面控制台来完成。由于多用户中，学生身份的用户创建删除和相关的权限变更比较大，因此以学生用户为例，注册及创建学生用户的流程图，如图 4.2 所示。

首先，管理员连接到 Web 控制台页面。之后导航到用户管理页面，在那里可以手动或使用 XML 文件添加用户信息。一旦管理员提交用户创建请求，网络组件通过 HTTP 请求将用户信息传递给服务器端 Server 程序，服务端 Server 程序将会对其进行处理。

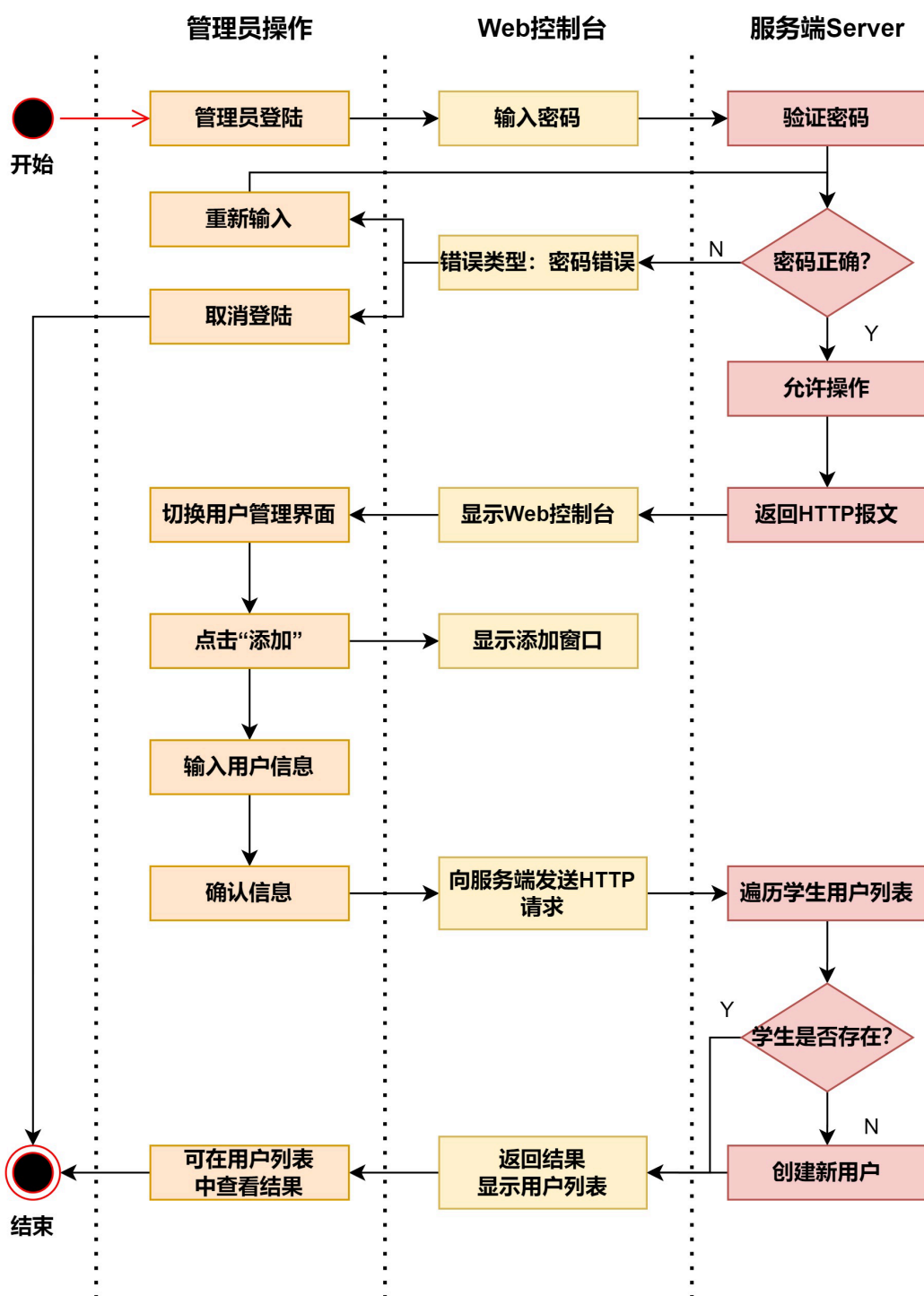


图 4.2 注册及创建用户流程图

如果学生或教师用户不存在，Server 程序就会遍历服务器主机中的用户 conf 配置文件里的用户列表，并向配置文件中添加新的用户数据。当服务端 Server 程序成功发送一个 HTTP 响应消息给 Web 组件，并且 Web 控制台向管理员显示成功

操作的结果后，用户创建的过程最终完成。管理员操作窗口如图 4.3 所示，操作结果如图 4.4 所示。

添加: 用户

用户名:

st1

领域:

Proxmox VE authenticat

密码:

.....

确认密码:

.....

组:

有效期至:

never

已启用:

☒

备注:

名:

student

姓:

Cloud-Desktop

E-Mail:

xxx@qq.com

高级 ☐

添加

图 4.3 添加用户窗口

数据中心

添加 | 编辑 | 删除 | 密码 | TFA | 权限

用户名 ↑	领域 ↑	已启用	有效期至	名称	TFA	备注
root	pam	是	永不		否	
st1	pve	是	永不	student Cloud-Desk...	否	

图 4.4 添加用户操作结果

删除用户的过程与创建用户相类似。如图 4.5 所示，管理员选中用户列表中的某个学生用户，点击顶部编辑栏中的“删除”按钮，即可将该选中的学生用户删除。在顶部菜单栏中还可以进行修改学生用户密码和权限的操作。

管理员用户、教师用户的创建和删除方式与学生用户大体类似，本小节以学生用户为例，对用户的创建和删除进行了介绍。



图 4.5 删除用户窗口

4.1.2 用户的登陆与注销

与用户的创建和删除需要管理员在 Web 控制台上操作不同的是，用户的登陆和注销主要在客户端软件上完成。

如图 4.6 所示，这是学生或教师在使用云桌面客户端软件时需要完成的第一个步骤。用户可以使用客户端进行在线登录。对于在线登录的用户，其密码会被加密并保存在该用户指定的配置文件中。

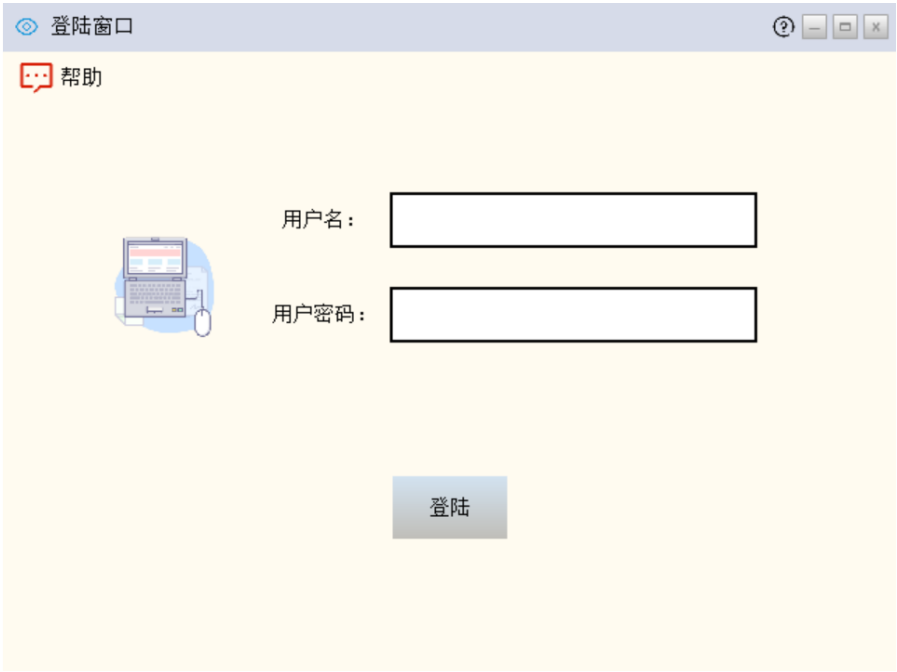


图 4.6 客户端登陆页面

如图 4.7 所示，是用户进行登陆活动的流程图。

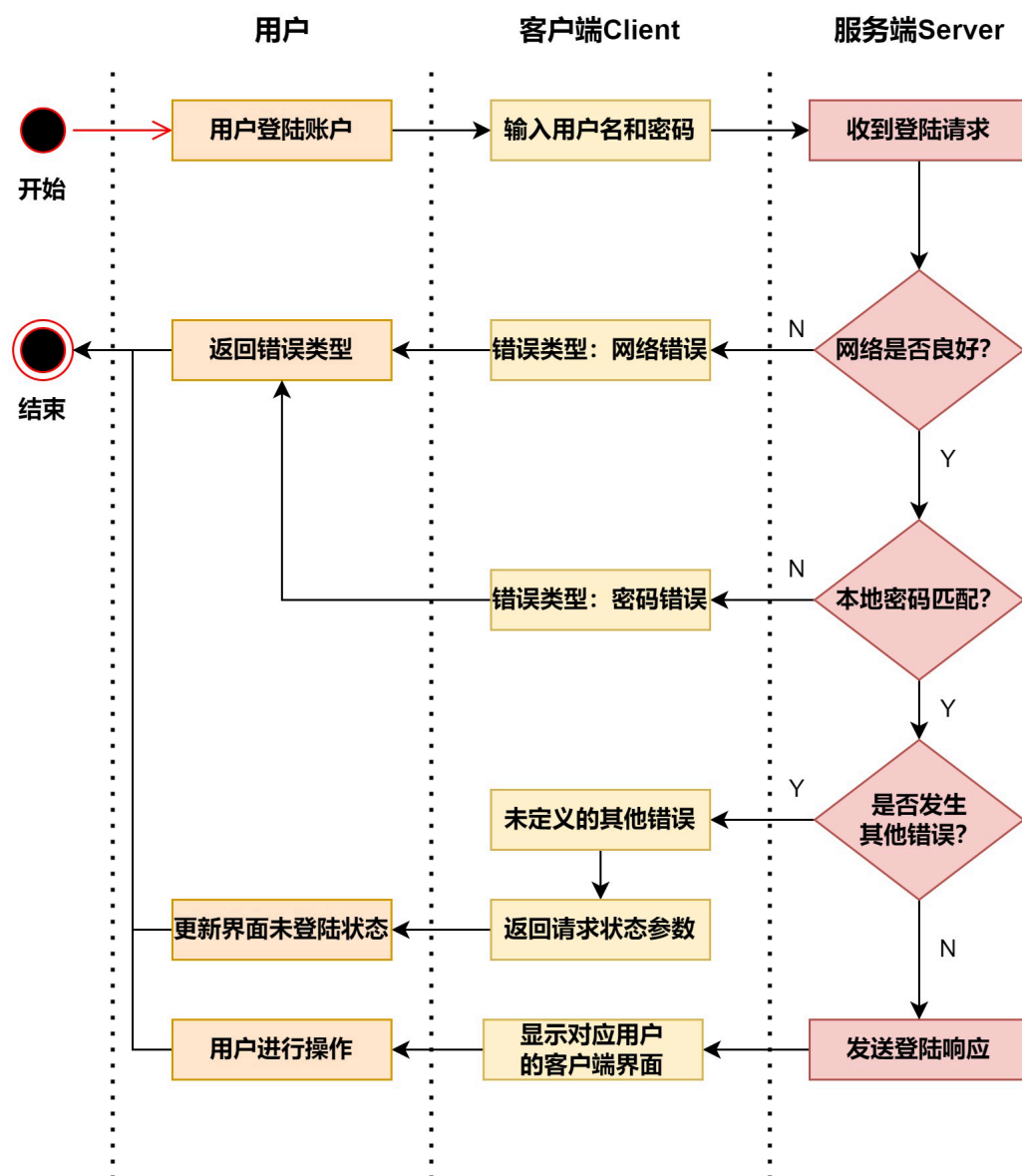


图 4.7 用户登陆流程图

关闭客户端或点击注销返回到登录界面时，都发起用户注销事件。执行注销事件之前，系统会确认当前用户的云桌面没有处于工作状态，并且没有外围设备挂载在云桌面系统上。如果这两个条件都成立，将会执行注销程序。

如图 4.8 所示，是用户进行注销活动的流程图。

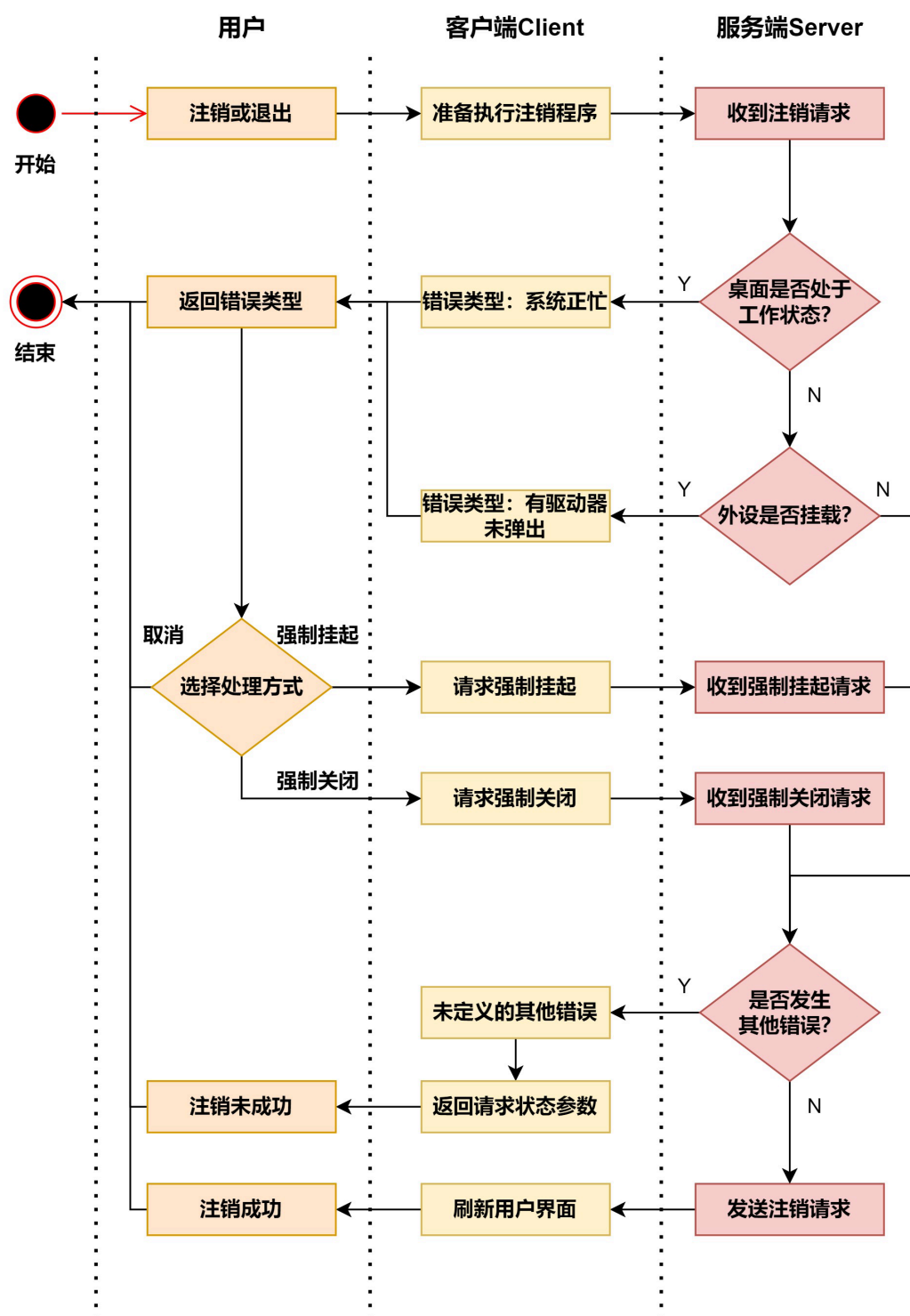


图 4.8 用户注销流程图

4.2 桌面管理模块设计与实现

桌面管理模块是云桌面系统最核心的部分，也是服务端 Server 程序最重要的部分。本系统根据高校云实验室和云办公场景下的部署情况和功能需求，在服务端 Server 程序中对 Proxmox VE 虚拟化环境的某些组件进行了重写和优化，并通过暴露的接口与底层 Proxmox VE 相互通信。涉及到的功能组件有桌面环境的创建、QEMU 侧 CPU 的创建、KVM 侧 vCPU 的创建、内存虚拟化和设备虚拟化组件等。桌面管理模块的功能示意图如图 4.9 所示。

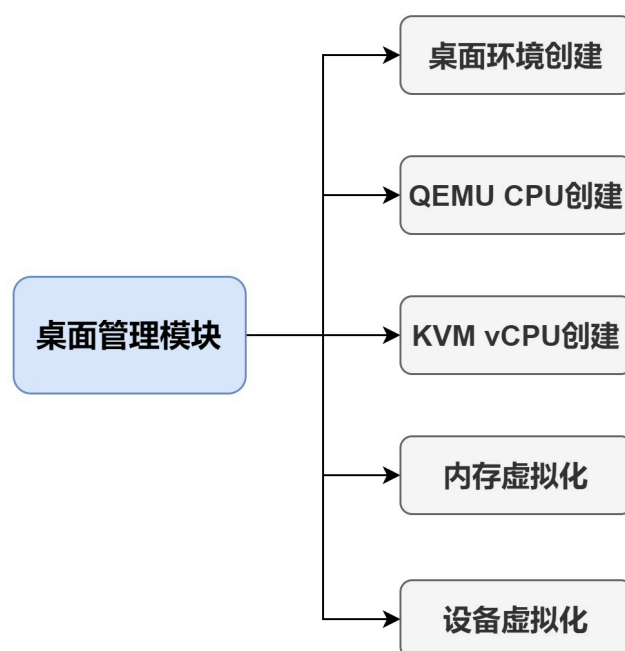


图 4.9 桌面管理模块功能示意图

4.2.1 桌面环境的创建

如果需要利用 KVM 模块建立一个虚拟桌面，需要先在用户态的 QEMU 侧发起请求，下面从 QEMU 和 KVM 两个方面，对桌面环境的创建进行说明。

4.2.1.1 QEMU 侧桌面环境的创建

当从 QEMU 侧发起创建请求时，解析消息函数会进入下列 case 分支：

```
case QEMU_OPTION_enable_kvm:  
    olist = qemu_find_opts("machine");  
    qemu_opts_parse_noisily(olist, "accel=kvm", false);  
    break;
```

通过 case 分支, 将 olist 这个参数添加了一个 accel=kvm 的二级参数, 之后 QEMU 侧的主线程将会调用 configure_accelerator 函数, 该函数会从 machine 的参数列表 olist 中取出 accel 的值, 确定其所属的类型, 最后再调用 accel_init_machine 函数, 进行初始化阶段。accel_init_machine 函数中, 对 accel 初始化的关键语句如下:

```
AccelState *accel = ACCEL(object_new(cname));
int ret;
ms->accelerator = accel;
ret = acc->init_machine(ms);
```

accel_init_machine 函数根据 accel 值的类型调用 object_new, 继而创建一个 AccelState 结构体, 该结构体其实就是 KVM 组件中的 KVMState 结构体。ret = acc->init_machine 语句将继续调用与 accel 类型对应的 init_machine 回调函数。该回调函数在 KVM 组件中就是 kvm_init 的变体, 这是通过类别初始化函数 kvm_accel_class_init 对 TYPE_KVM_ACCEL 进行设置的。

kvm_init 中关键语句如下:

```
MachineClass *mc = MACHINE_GET_CLASS(ms);
KVMState *s;
s = KVM_STATE(ms->accelerator);
s->fd = qemu_open("/dev/kvm", O_RDWR);
s->nr_slots = kvm_check_extension(s, KVM_CAP_NR_MEMSLOTS);
do {
    ret = kvm_ioctl(s, KVM_CREATE_VM, type);
} while (ret == -EINTR);
ret = kvm_arch_init(ms, s);
```

在 QEMU 侧, KVMState 结构体用来表示 KVM 相关的数据类型和方法。kvm_init 首先打开 “/dev/kvm” 中的某一个设备, 得到文件句柄 fd, 并且将其保存到类型为 KVMState 的结构体变量 s 的 fd 成员变量中。继续调用 kvm_ioctl 接口在 KVM 侧创建一个虚拟桌面环境。最后调用 kvm_arch_init 完成与计算机架构, 例如 x86 架构有关的初始化工作。

4.2.1.2 KVM 侧桌面环境的创建

kvm_init 最关键的作用就是调用 “/dev/kvm” 设备的得 ioctl 接口。使用 ioctl (KVM_CREATE_VM) 函数起到在 KVM 组件中创建一个桌面虚拟机的目的。一个

桌面虚拟机本质上就是一个 QEMU 进程，在 KVM 侧则用结构体变量 `kvm` 表示一台虚拟桌面。对于 `KVM_CREATE_VM`，其处理函数是 `kvm_dev_ioctl_create_vm`，关键语句如下：

```
int r;
struct kvm *kvm;
kvm = kvm_create_vm(type);
r = kvm_coalesced_mmio_init(kvm);
r = anon_inode_getfd("kvm-vm-desktop", &kvm_vm_fops, kvm, O_RDWR|
O_CLOEXEC);
```

该函数的主要任务是调用 `kvm_create_vm` 创建虚拟桌面实例，每一个虚拟桌面实例用一个 `kvm` 结构体变量表示，`kvm_coalesced_mmio_init` 用来对合并 MMIO 进行初始化，主要就是分配一个内存页，合并 MMIO 指的是将 MMIO 的写请求放到一个环中，等到其他事件产生或者是环满了并产生 `VMExit` 时，再对 MMIO 进行处理。

`anon_inode_getfd` 函数创建了一个匿名 file，其返回值用变量 `r` 接收。该匿名的 file 对应的 fd（即变量 `r`）返回给用户态的 QEMU 侧，代表这是一台虚拟桌面，之后 QEMU 就可以通过该 fd 对虚拟桌面环境进行操作了。

4.2.2 QEMU 侧 CPU 的创建

CPU 模型的继承结构，如图 4.10 所示。因为 QEMU 能够模拟各种 CPU 模型，所以它需要一套继承结构来表示 CPU 对象。QEMU 除了定义了实际存在的 CPU，例如 `pentium` 和 `Haswell` 外，还会定义一些并没有真实物理 CPU 的虚拟 CPU 模型，如 `qemu-x86_64-cpu` 或 `kvm-x86_64-cpu`。因为具有相同架构的 CPU 具有类似的功能，只有一些小的特征差异，所以大部分的功能都是在 `TYPE_X86_CPU` 这个类型的对象中实现的。

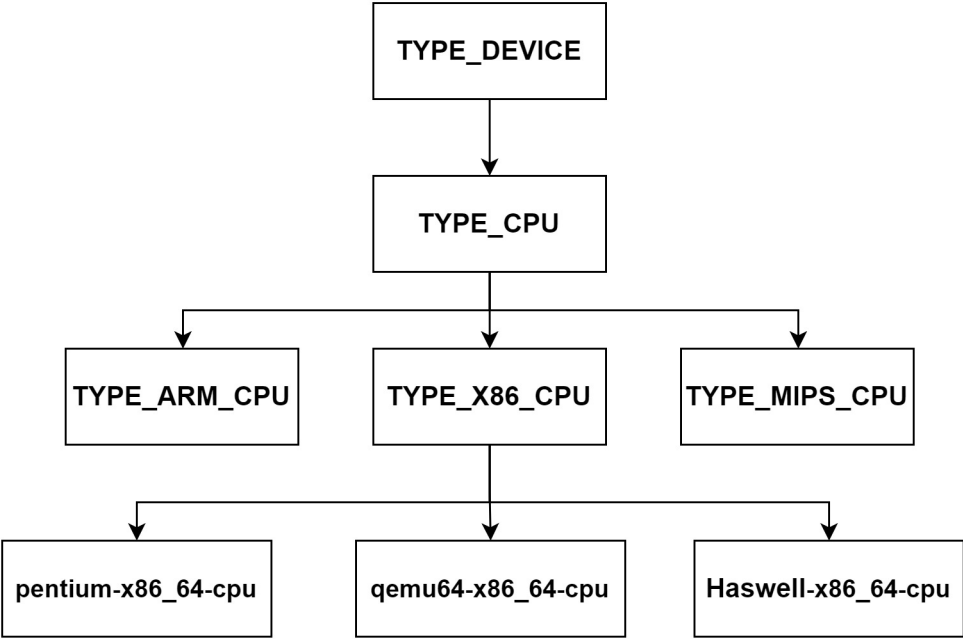


图 4.10 CPU 模型的继承结构

QEMU 所支持的所有 x86_64 架构的 CPU 都定义在一个数组内，该数组由类型为 X86CPUDefinition 的结构体变量组成，该结构体的定义如表 4.1 所示。

表 4.1 X86CPUDefinition 结构体成员变量

成员变量名	变量含义
name	CPU 的名字
level	CPUID 指令支持的最大功能号
xlevel	CPUID 扩展质量支持的最大功能号
vendor、family、model、stepping	CPU 的基本信息
features	记录 CPU 特性的数组
model_id	CPU 的品牌名称或信息

由于真实世界中有很多种 CPU，所以 builtin_x86_defs 是一个很大的数组。它的每一个元素都表示一种模拟的 CPU 模型。QEMU 既模拟了一些真实存在的 CPU，比如 pentium、SandyBridge 和 Haswell，也模拟了一些虚拟的 CPU 类型，比如 qemu64、kvm64、qemu32 和 kvm32 等。

函数 x86_register_cpufdef_type 从一个个的 X86CPUDefinition 结构中构造出 TypeInfo 结构，并调用 type_register 函数进行 CPU 类型的注册。TYPE_X86_CPU 的对象实例由 X86CPU 结构体表示。一个 X86CPU 结构体表示一个 x86 架构的虚

拟 CPU，其保存了有关 CPU 的各种信息。X86CPU 结构体的成员变量如表 4.2 所示。

表 4.2 X86 结构体成员变量

成员变量名	变量含义
CPUState	所有 CPU 都具有的通用数据
CPUX86State	仅限于 X86CPU 类型的数据
check_cpuid	QEMU 命令行参数指定用来指示是否需要检测宿主机的 CPUID
filtered_features	被过滤掉的特性
apic_id	该 CPU 对应的 LAPIC 的 id
socket_id、core_id、thread_id	该虚拟 CPU 在系统上的拓扑信息

在/etc/pve/qemu-server 路径保存的云桌面配置文件中，CPU 的参数一般是：
-cpu model, +feature, -feature, feature=foo

对于+feature 参数，把其加到 plus_features 链表中；对于-feature 参数，把其加到 minus_features 中，在之后的 CPU 具象化的过程中会把相应的 feature 对应的属性设置为 true 或者 false；对于 feature=foo，则是设置 CPU 的其他特性，它会设置 CPU 的对应属性。

在进行虚拟桌面硬件初始化的时候，会调用 pc_new_cpu 函数。它是硬件虚拟化时最重要的步骤，通过 pc_new_cpu 函数来创建一个类型为 X86CPU 结构体的 CPU 实例，实例的个数是解析/etc/pve/qemu-server/xxx.conf 配置文件中 CPU 参数得到的，pc_new_cpu 中最关键的语句如下：

```
cpu = X86_CPU( object_new(typename) );
```

pc_new_cpu 函数通过 object_new 创建一个 CPU 实例，通过将结构体具象化，完成 CPU 实例的具象化和初始化，完成 QEMU 侧 CPU 的创建。

4.2.3 KVM 侧 vCPU 的创建

4.2.1.2 节提到的 kvm_dev_ioctl_create_vm 函数中会创建一个匿名的文件 fd，用来表示一台虚拟机并将 fd 返回到用户态，QEMU 使用该 fd 来控制虚拟桌面。该匿名文件的 file_operations 定义中主要的操作函数是 kvm_vm_ioctl。

kvm_vm_ioctl 函数中主要操作也是 ioctl 对象，这个函数是处理虚拟桌面类别的 ioctl 的入口，入口中最上层对应的函数是 kvm_vm_ioctl_create_vcpu 函数。由

于整个 vCPU 的创建过程涉及到的函数比较多，其整个流程可以简化为图 4.11 所示。vCPU 创建的主要过程如下：

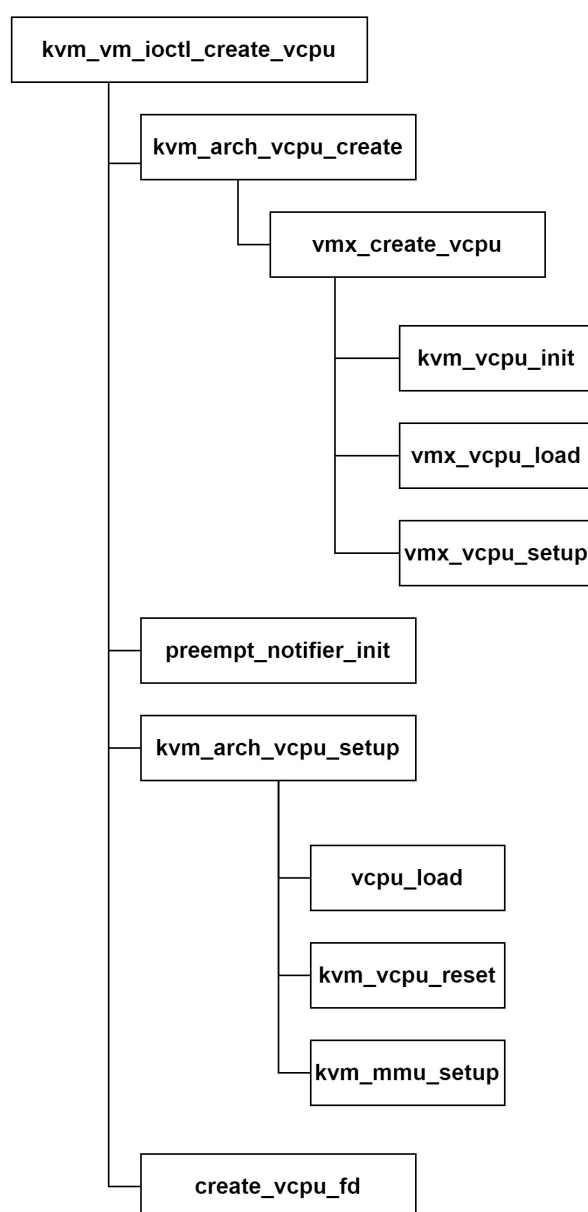


图 4.11 KVM 侧 vCPU 创建过程调用图

- (1) `kvm_vm_ioctl_create_vcpu` 首先调用 `kvm_arch_vcpu_create`，在该函数内调用了 `vmx` 实现的 `vmx_create_vcpu` 函数；
- (2) 接着调用 `preempt_notifier_init` 将 vCPU 的 `preempt_notifier` 进行初始化；
- (3) 然后调用 `kvm_arch_vcpu_setup` 进行 vCPU 的一些初始化；

(4) 最后, `kvm_vm_ioctl_create_vcpu` 调用 `create_vcpu_fd` 创建一个 vCPU 的句柄 `fd`, 并且返回到应用层。应用层 QEMU 侧可以通过该 `fd` 对 vCPU 进行操作。所有 vCPU 都保存在 `VCPUS` 结构体变量中。

4.2.4 内存虚拟化

由于内存是计算机的组成部件之一, 所以每一种形式的虚拟化解决方案都必须处理好内存虚拟化的问题。从 CPU 的角度来看, 物理内存是一条从 0 开始的连续可用的线性储存空间。在虚拟化中, 每个虚拟机也都需要这个从 0 开始的连续可用的地址。为此, 在建立虚拟机时必须为其创造出一段这样的地址空间。

如果直接将 QEMU 进程的虚拟地址作为这样一段空间提供给虚拟机, 会造成 CPU 访问时实际使用的是虚拟地址, 而虚拟地址必须通过 MMU 硬件进行转换, 将虚拟地址转换成物理地址之后才能访问到实际的物理地址。该过程如图 4.12 (a) 所示。但是在虚拟化场景下, 虚拟机内部也有自己的保护模式, 所以当虚拟设备内部的虚拟 CPU 进行内存寻址的时候, 其使用的也是虚拟设备自身的虚拟地址。

如果想让 CPU 访问到物理宿主机上的某段真实的物理内存地址, 就必须先将目标地址转换成虚拟设备的物理地址, 然后将它转换成 QEMU 的虚拟地址, 最后将 QEMU 的虚拟地址转换成物理机上的物理地址, 才能访问到数据。该过程如图 4.12 (b) 所示。这显然需要虚拟化软件给出一种方法来进行这种的转换, 这被称为 MMU 虚拟化。

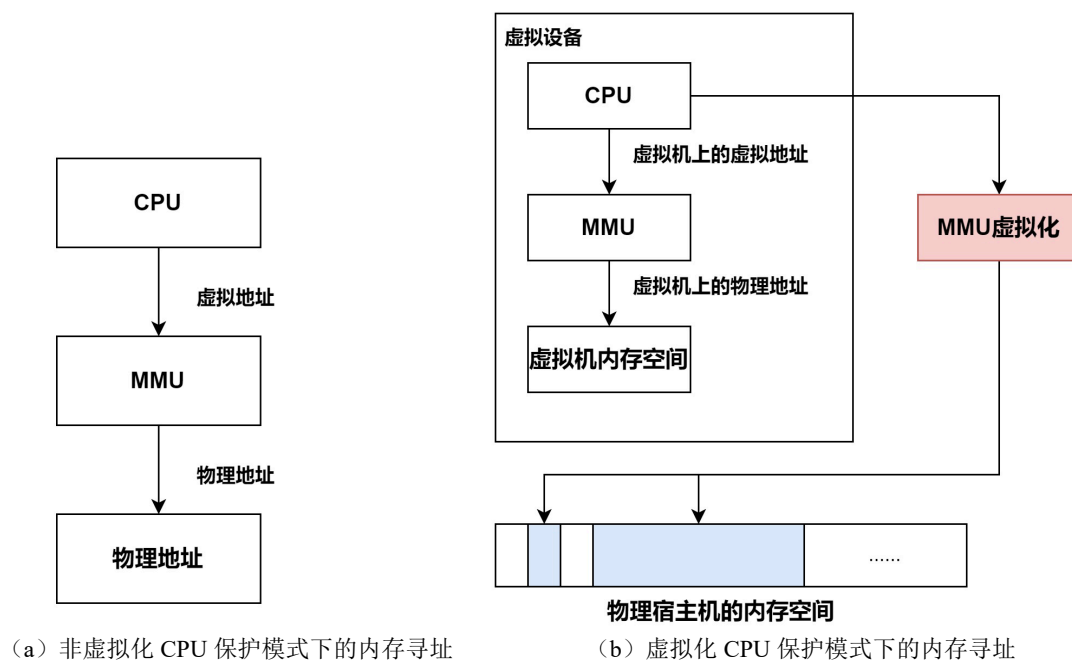


图 4.12 CPU 保护模式下的内存寻址过程

在 CPU 没有支持 Extended Page Table 技术之前，是通过所谓的影子页表来实现这个功能。但影子页表技术使用纯软件来模拟一个 MMU，效率低下，所以本文引入 VMX 架构中的拓展页表技术，即 Extended Page Table，简称 EPT 方案。在 EPT 方案中，CPU 的寻址模式在 VM non-root operation 下会发生变化，其会使用两个页表。其基本原理如图 4.13 所示。

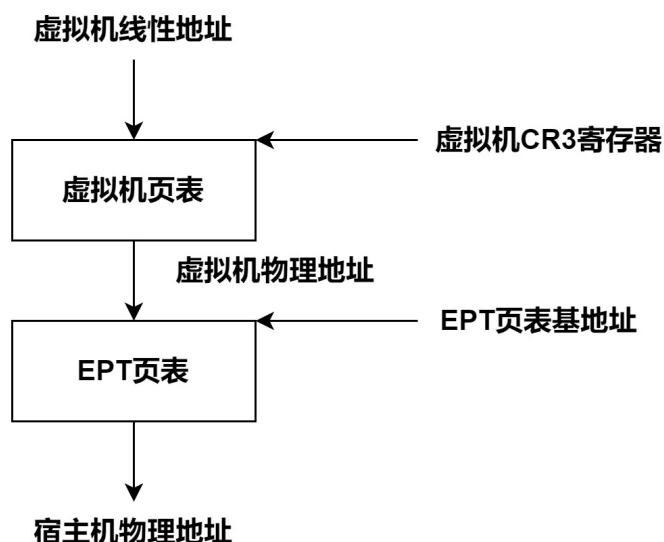


图 4.13 EPT 基本原理

当 CPU 在启用 EPT 的情况下并执行一个 VM Entry 时，CPU 使用 EPT 功能，并且虚拟机可以完全控制它自己的内部页表。CPU 首先将虚拟机内部的虚拟地址转换为虚拟机的物理地址，然后将此地址转换为宿主机的物理地址，然后搜索 EPT 页表，从而查找虚拟机的内部页表。当 CPU 执行 VM Exit 时，则关闭 EPT，此时 CPU 再次使用单页表在主机上寻址。虚拟机中的页表功能类似于物理机的系统页表，不同之处在于宿主机负责维护 EPT 页表，里面记录着虚拟机物理地址到宿主机的物理地址的转换。而虚拟机中的页表保存虚拟机的物理地址，由自己维护。

尽管物理地址是 48 位，但 EPT 通常使用 IA-32e 分页模型。页表总共有四层，后 12 位表示页内（4KB）的偏移量，前 9 位用于物理地址定位。图 4.14 展示了 EPT 的分页模式。

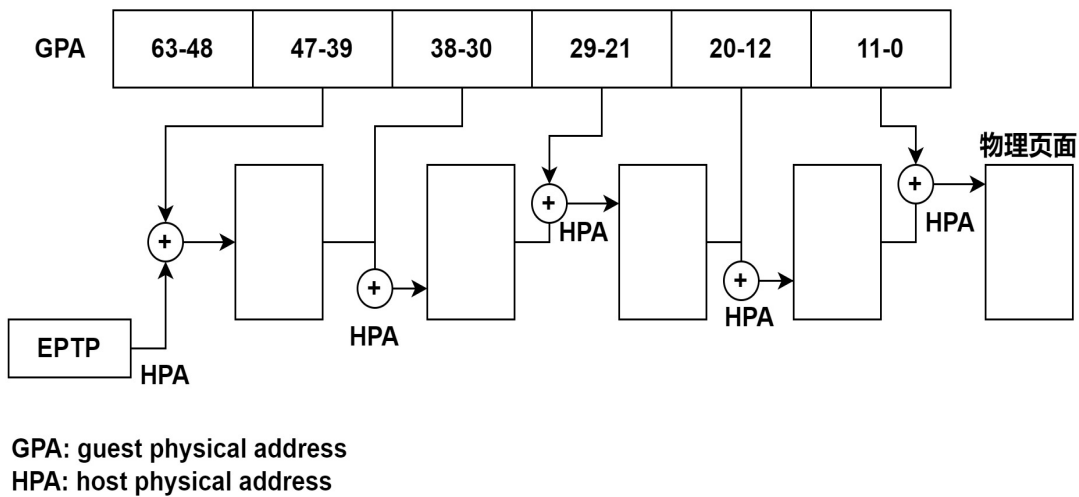


图 4.14 EPT 分页模式

虚拟机在 EPT 分页基址下进行访问内存会导致一系列 GPA 到 HPA 转换。图 4.14 展示了 EPT 的访问过程，EPT 页表的基地址保存在 VMCS 结构内的 EPTP 指针中。当执行内存访问时，虚拟机首先将虚拟机内存转换为虚拟机的物理内存地址。在此期间，这将查询虚拟机的页表，如果相应的页表存在且有效，将读取该信息。然后，虚拟机将读取下一级页表，并计算出需要访问的 GPA。最后，这个 GPA 将执行另一个页表访问。最终找到物理机上的数据。如果在这个操作过程中，虚拟机发生了页面故障，虚拟机将修复自己的页面表。如果发生 EPT 错误，虚拟机将退出到 KVM 侧来建立 EPT 页表。

4.2.5 设备虚拟化

4.2.5.1 总线数据类型和创建

一台完整的计算机离不开各种外设，所以虚拟化也要模拟出各种各样的设备。随着云计算的快速发展，对于性能的追求也使得一些常用的设备在硬件层面支持虚拟化。QEMU 的设备模拟也逐渐发展，经历了由传统的纯软件模拟的纯虚拟化向半虚拟化 virtio 转变，再向设备直通转变的历程。

本文设计的系统中，总线类型用 TYPE_BUS 表示，它是所有总线设备的基类，所有具体的总线设备类型的父类都是 TYPE_BUS，例如 PCI_BUS、ISA_BUS、SCSI_BUS 等。

与总线有关的类是 BusClass，其定义和成员变量含义如表 4.3 所示。

表 4.3 BusClass 类成员变量

成员变量名	变量含义
print_dev	打印总线上的一個设备
get_dev_path	得到设备路径以及在 firmware 中的路径
realize	表示 Bus 进行 realize 的回调函数
unrealize	销毁时的回调函数
max_dev	表示该 Bus 上允许的最大设备

表示 Bus 对象的结构体是 BusState，其定义和成员变量如表 4.4 所示。

表 4.4 BusState 结构体成员变量

成员变量名	变量含义
parent	表示总线所在的设备
hotplug_handler	指向一个处理热插拔的处理器
max_index	表示插在该总线上的设备个数
sibling	用来连接在一条总线上的设备

介绍完总线的对象和父类之后，接下来分析 TYPE_BUS 的类型信息。TYPE_BUS 信息封装在一个类型为 TYPE_Info 的结构体常量中。一个 TYPE_Info 结构体实例代表一种总线设备。TYPE_Info 类型结构体定义如表 4.5 所示。

表 4.5 TYPE_Info 结构体成员变量

成员变量名	变量含义
qbus_initfn	为 Bus 实例增加了一个 link 属性
qbus_finalize	总线删除函数
bus_class_init	Bus 类型在初始化的时候调用

qbus_initfn 在创建总线 Bus 实例的时候调用，它为 Bus 实例增加了一个 link 属性，其所要连接的对象是 TYPE_HOTPLUG_HANDLER，目的地址是 bus->hotplug_handler；qbus_finalize 是总线删除函数；bus_class_init 设置了 Object 基类的 unparent 函数，这个回调函数在子类解除父类引用时被调用，另一个是 BusClass 的 get_fw_dev_path，这是默认函数。如果子类没有重写这个函数，那么当查找固件路径的时候就会用这个函数。

4.2.5.2 设备数据类型和创建

本文所设计的系统中外围设备类型用 DeviceClass 表示,其定义如表 4.6 所示。

表 4.6 DeviceClass 结构体成员变量

成员变量名	变量含义
categories	表示设备的种类,如显卡、网卡等
fw_name	用于生产设备在固件中的路径
desc	用于描述设备
props	表示设备的属性
hotpluggable	表示设备是否能进行热插拔
reset、realize 和 unrealize	表示设备重置、具象化、反具象化时调用的函数
vmsd	表示设备的状态,在虚拟机进行热迁移的时候,需要记录设备的状态以便在目的主机上恢复
bus_type	表示该设备挂载的总线类型

DeviceClass 用来表示设备具有的共性,而 DeviceState 表示一个具体的设备实例,其定义如表 4.7 所示。

表 4.7 DeviceState 结构体成员变量

成员变量名	变量含义
id	表示设备名
realized	表示设备是否已经被具象化
pending_deleted_event	设备实例销毁时,判断设备是否已经具象化
opts	表示设备对应的参数
hotplugged	表示设备是否通过热插拔进入到系统中
parent_bus	表示该设备挂载的总线
gprios	相当于设备的输入输出,用来模拟硬件的 pin
child_bus	用来连接该设备下面的所有总线
num_child_bus	表示总线的序号

DeviceState 表示的是 TYPE_DEVICE 的实例,表示所有设备都会有的特性。TYPE_DEVICE 是一个抽象类型,每一个具体的设备类型都需要设置其父设备为 TYPE_DEVICE。

设备的创建有两种方式：第一种是跟随主板初始化一起创建，典型的设备包括南北桥、一些传统的 ISA 设备、默认的显卡设备等，这类设备是通过 `qdev_create` 函数创建；第二种方法是在虚拟桌面对应的 `conf` 文件中通过添加 `-device` 参数添加的，这类设备通过 `qdev_device_add` 函数创建。两种创建方式本质上都调用了 `object_new` 创建对应设备的实例化对象。

4.3 虚拟存储模块维护与管理

虚拟存储是可以提供给虚拟机的存储空间，是宿主机物理存储的一部分，是通过模拟或半虚拟化的块设备驱动程序分配给虚拟机的存储。在云桌面系统设计中，需要一个存储后端为虚拟机提供存储。如果是由 Proxmox VE 或 libvirt 等虚拟化环境创建和管理的存储，被称为托管式的存储，他是用存储池、存储卷等技术高效便捷的管理存储系统。如果由管理员手动创建的存储区，被称为非托管式的存储，它适用于体积比较小的测试环境。

4.3.1 宿主机的存储资源

可以分配给虚拟桌面的存储资源有很多种，常用的有映像文件、LVM 卷、宿主机设备和分布式存储系统。下面将对齐一一进行介绍。

（1）映像文件

映像文件只能存储在宿主机文件系统中，例如本地文件系统或网络文件系统。管理员如果用 SSH 远程连接到宿主机，可以用 Proxmox 已经安装好的 `qemu-img` 或 `pvesm` 命令行工具创建、管理映像文件。虚拟桌面最常见的磁盘映像格式是 `raw` 和 `qcow2`。`raw` 格式的映像文件只包含数据内容，不包括其他元数据。当对磁盘 I/O 性能要求高且很少通过网络传输映像文件时，推荐使用 `raw` 格式的文件。因为在映像文件中仅仅分配了数据的扇区，所以在网络传输情景下，`qcow2` 格式相对 `raw` 格式则更有优势。

（2）LVM 卷

由 LVM 创建的逻辑卷比映像文件有更好的性能，因为它们可以直接分配给宿主机上的虚拟桌面。可以通过控制 LVM 卷，方便管理宿主机上虚拟桌面的磁盘资源。

（3）宿主机设备

可以利用“硬件直通”功能。将宿主机上的设备例如显卡、磁盘、CD-ROM 等直接分配给宿主机上的虚拟桌面，从而获得更好的硬件体验。

（4）Ceph 分布式存储系统

Proxmox VE 目前支持 Ceph 分布式存储系统。它将数据存储分散在众多独立的节点上，每个节点都统一地对外提供存储服务。其可扩展的系统架构增强了系统的可靠性、可用性和访问效率。

4.3.2 qemu-img 和 pvesm 命令的使用

qemu-img 是一个管理虚拟磁盘映像文件的命令行工具。它属于非托管式的存储管理方式，需要手动挂载到虚拟桌面中。使用它可以创建、格式化、修改与维护多种格式的映像文件。例如：

```
# qemu-img --help | grep Supported
```

```
Supported formats: alloc-track backup-dump-drive blkdebug blklogwrites
blkverify bochs cloop compress copy-on-read dmg file ftp ftps gluster host_cdrom
host_device http https iscsi iser luks nbd null-aio null-co nvme pbs preallocate qcow
qcow2 qed quorum raw rbd replication throttle vdi vhdx vmdk vpc vvfat zeroinit
```

上述命令列出了该版本的 qemu-img 所支持的所有文件格式。其他常用的子命令如表 4.8 所示。

表 4.8 qemu-img 常用子命令

子命令	描述
create	创建新的映像文件
check	检查现有的映像文件是否有错误
convert	转换映像文件格式
info	显示映像文件信息
snapshot	管理映像文件快照
rebase	调整派生映像文件与基础映像文件的位置与名称
commit	将保存在派生映像文件的数据提交到基础映像文件中
resize	调整映像文件的大小

使用 qemu-img 命令时，需要指定映像文件格式，这里介绍本系统主要使用的两种磁盘映像格式。

(1) raw：默认的原始磁盘映像格式，它是最快的映像文件的格式，如果文件系统支持 ext2、ext3、ext4 或者 xfs，则只有在写入数据的时候才会分配空间。

(2) qcow2：QEMU 映像文件格式，支持 AES 加密，基于 zlib 压缩，支持快照，但这些功能是以性能开销为代价的。

创建新的磁盘映像文件语法格式如下：

*qemu-img create [-f format] [-o options] filename [size]*

create 命令通过-f 参数来创建指定格式的映像文件，如不指定-f 参数，默认以 raw 格式创建文件。

pvesm 命令则是 Proxmox VE 虚拟化环境提供的存储管理工具，管理员除了使用命令行进行管理，也可以在 Web 管理控制台中进行可视化的操作和管理。该种方式属于托管式存储管理方式。常用的 pvesm 命令行如表 4.9 所示。

表 4.9 pvesm 常用命令行

pvesm 命令	含义
pvesm add <type> <storage> [OPTIONS]	用于创建映像文件
pvesm list <storage> [OPTIONS]	用于显示列表中的存储内容

pvesm add 命令用于创建映像文件，<type>参数支持如下选项：<cephfs | cifs | dir | drbd | glusterfs | iscsi | iscsidirect | lvm | lvmthin | nfs | pbs | rbd | zfs | zfspool>; <storage>参数表示存储标识符。

pvesm list 命令用于显示列表中的存储内容，<storage>参数也是存储标识符。例如，输入 pvesm list local 可打印出宿主机内映像文件列表，结果如图 4.15 所示：

Valid	Format	Type	Size	VMID
local:iso/CentOS-7-x86_64-DVD-2207-02.iso	iso	iso	4746903552	
local:iso/cn_windows_10_business_editions_version_x64_dvd.iso	iso	iso	5311711232	
local:iso/cn_windows_7_enterprise_x64_dvd_x15-70741.iso	iso	iso	3203516416	
local:iso/Fedora-Workstation-Live-x86_64-35-1.2.iso	iso	iso	2009333760	
local:iso/spice-guest-tools-0.164.4.iso	iso	iso	144150528	
local:iso/ubuntu-16.04.7-desktop-amd64.iso	iso	iso	1697906688	
local:iso/ubuntu-20.04.3-desktop-amd64.iso	iso	iso	3071934464	
local:iso/virtio-win-0.1.229.iso	iso	iso	534818816	
local:vztmpl/ubuntu-20.04-standard_20.04-1_amd64.tar.gz	tgz	vztmpl	214203058	

图 4.15 pvesm list local 命令结果图

输入 pvesm list local-lvm 可打印出宿主机内 lvm 类型逻辑卷的列表。lvm 逻辑卷由 Proxmox VE 直接分配至虚拟桌面，作为虚拟桌面的磁盘资源。并在管理员创建虚拟机时，自动创建并挂载在虚拟桌面上。结果如图 4.16 所示：

Valid	Format	Type	Size	VMID
local-lvm:vm-102-disk-0	raw	images	137438953472	102
local-lvm:vm-103-disk-0	raw	images	137438953472	103

图 4.16 pvesm list local-lvm 命令结果图

同样，对于 pvesm 命令行的操作结果，管理员也可以在 Web 管理控制台中可视化操作。首先，管理员连接到 Web 控制台页面。之后导航到存储管理页面，点

击 `local` 和 `local-lvm` 存储池可以查看和管理存储池中的文件，如图 4.17 和 4.18 所示。

存储'local'在节点'pve'上

 概要  备份  ISO镜像  CT模板  权限	上传 从URL下载 删除 <table><tr><th>名称</th><th>日期</th><th>格式</th><th>大小</th></tr><tr><td>CentOS-7-x86_64-DVD-2207-02.iso</td><td>2023-02-18 14:47:18</td><td>iso</td><td>4.75 GB</td></tr><tr><td>Fedora-Workstation-Live-x86_64-35-1.2.iso</td><td>2022-01-04 20:02:04</td><td>iso</td><td>2.01 GB</td></tr><tr><td>cn_windows_10_business_editions_version_x64_dvd.iso</td><td>2023-02-19 14:38:02</td><td>iso</td><td>5.31 GB</td></tr><tr><td>cn_windows_7_enterprise_x64_dvd_x15-70741.iso</td><td>2021-12-20 13:06:46</td><td>iso</td><td>3.20 GB</td></tr><tr><td>spice-guest-tools-0.164.4.iso</td><td>2023-02-18 14:36:37</td><td>iso</td><td>144.15 MB</td></tr><tr><td>ubuntu-16.04.7-desktop-amd64.iso</td><td>2022-12-06 18:49:09</td><td>iso</td><td>1.70 GB</td></tr><tr><td>ubuntu-20.04.3-desktop-amd64.iso</td><td>2021-09-20 17:57:58</td><td>iso</td><td>3.07 GB</td></tr><tr><td>virtio-win-0.1.229.iso</td><td>2023-02-18 14:37:28</td><td>iso</td><td>534.82 MB</td></tr></table>	名称	日期	格式	大小	CentOS-7-x86_64-DVD-2207-02.iso	2023-02-18 14:47:18	iso	4.75 GB	Fedora-Workstation-Live-x86_64-35-1.2.iso	2022-01-04 20:02:04	iso	2.01 GB	cn_windows_10_business_editions_version_x64_dvd.iso	2023-02-19 14:38:02	iso	5.31 GB	cn_windows_7_enterprise_x64_dvd_x15-70741.iso	2021-12-20 13:06:46	iso	3.20 GB	spice-guest-tools-0.164.4.iso	2023-02-18 14:36:37	iso	144.15 MB	ubuntu-16.04.7-desktop-amd64.iso	2022-12-06 18:49:09	iso	1.70 GB	ubuntu-20.04.3-desktop-amd64.iso	2021-09-20 17:57:58	iso	3.07 GB	virtio-win-0.1.229.iso	2023-02-18 14:37:28	iso	534.82 MB
名称	日期	格式	大小																																		
CentOS-7-x86_64-DVD-2207-02.iso	2023-02-18 14:47:18	iso	4.75 GB																																		
Fedora-Workstation-Live-x86_64-35-1.2.iso	2022-01-04 20:02:04	iso	2.01 GB																																		
cn_windows_10_business_editions_version_x64_dvd.iso	2023-02-19 14:38:02	iso	5.31 GB																																		
cn_windows_7_enterprise_x64_dvd_x15-70741.iso	2021-12-20 13:06:46	iso	3.20 GB																																		
spice-guest-tools-0.164.4.iso	2023-02-18 14:36:37	iso	144.15 MB																																		
ubuntu-16.04.7-desktop-amd64.iso	2022-12-06 18:49:09	iso	1.70 GB																																		
ubuntu-20.04.3-desktop-amd64.iso	2021-09-20 17:57:58	iso	3.07 GB																																		
virtio-win-0.1.229.iso	2023-02-18 14:37:28	iso	534.82 MB																																		

图 4.17 local 存储池文件

存储'local-lvm'在节点'pve'上

概要	删除			
VM磁盘	名称	日期	格式	大小
CT卷	vm-102-disk-0	2023-02-19 14:48:19	raw	137.44 GB
权限	vm-103-disk-0	2022-11-30 15:25:42	raw	137.44 GB

图 4.18 local-lvm 存储池文件

4.3.3 基础映像和派生映像

在高校云实验室中使用的桌面虚拟化系统，桌面环境中的许多虚拟桌面都运行了相同的操作系统和软件版本。在这种应用场景下，就可以使用基础映像桌面和派生映像桌面来减少存储空间的消耗。先创建一台虚拟桌面，安装操作系统和应用程序后，将这台虚拟桌面关机。其磁盘映像文件就可以作为基础映像（也称“模板”）。基于模板创建新的虚拟桌面时，新桌面的磁盘映像只是模板的派生映像。派生映像文件仅仅保存与模板之间的差异，虚拟桌面会自动将派生映像文件和基础模板拼接后的结果，当做自己的磁盘进行读取。如图 4.19 所示。

存储'local-lvm'在节点'pve'上

📄 概要	<button>删除</button>			
🗄️ VM磁盘	名称	日期	格式	大小
🗄️ CT卷	base-103-disk-0	2022-11-30 15:25:42	raw	137.44 GB
🔒 权限	vm-102-disk-0	2023-02-19 14:48:19	raw	137.44 GB
	vm-104-disk-0	2023-02-19 15:33:26	raw	137.44 GB

图 4.19 利用模板创建新桌面后 local-lvm 存储池文件

管理员将 vm-103 虚拟桌面设置为模板之后，创建其他虚拟桌面时只需要在 vm-103 的基础上，进行链接复制，派生映像文件的格式必须为 qcow2 格式。新创建的 vm-104 虚拟桌面，将 base-103-disk 和 qcow2 格式的派生映像进行合并，将合并之后的 vm-104-disk 作为自己的硬盘挂载。由于 base-103-disk 是公共部分，无论后续新建多少虚拟桌面，其磁盘文件都在 base-103-disk 上“拓展”，这样的创建方式在保证虚拟桌面磁盘容量大小的情况下，极大地减少了宿主机的硬件压力。

4.4 本章小结

本章主要对云桌面系统的个人桌面功能进行了详细设计和实现。主要包括用户账户的管理、虚拟桌面的创建与管理、虚拟存储池的维护。针对每个模块的功能和其实现方式也在本章中做了相关的说明，对于一些重要部分的原理过程，给出了重点函数和流程框图进行梳理。

第5章 系统的构建、运行和验证

5.1 系统的运行环境和部署情况

本节主要介绍云桌面系统服务端和客户端的运行环境，包括服务端开发环境、客户端开发环境、虚拟化平台和服务端 Server 程序的搭建和部署。客户端程序的部署以及客户端硬件与软件环境。

5.1.1 服务端和客户端程序开发环境

服务端程序是一个 C/C++ 项目，遵循 C11 标准，利用 `cmake` 工具组织项目并生成构建文档。使用 Clion 远程 SSH 连接到服务端对整个程序进行开发。

- C/C++ 语言标准：C11
- `cmake` 版本：`cmake 3.5.1`
- 编译器版本：`gcc 7.5.0`、`g++ 5.4.0`
- IDE 版本：`Clion 2022.1.3`

客户端程序是一个基于 Qt 5.10 开发的 C++ GUI 项目，项目遵循 C11 标准，利用 Qt 自带的 `qmake` 工具组织项目并生成构建文档。Qt 项目可以使用 Qt Creator 进行开发，其旨在将视图界面与逻辑代码解耦，以获得更多的便利性和直观性，可以快速而直观地开发软件界面。

- C/C++ 语言标准：C11
- `qmake` 版本：3.1
- Qt 版本：5.10.1
- IDE 版本：Qt Creator 4.5.1

5.1.2 服务端和客户端设备硬件配置

云桌面系统服务端设备主要分为三类：服务端主机和计算节点为同一台物理机，存储节点为一台物理机，流媒体处理节点为计算节点虚拟化出来的一台 Linux 主机。三类服务器硬件配置如表 5.1 所示。

本文设计的云桌面系统采用 VDI 架构，云桌面全部都运行在服务端设备中，并通过 SPICE 协议传输，因此本地客户端设备可以是瘦客户端，可以不具备太强的计算能力。客户端设备运行 Windows 操作系统，云桌面系统客户端软件在 Windows 系统平台中进行开发和运行。客户端设备硬件配置如表 5.2 所示。

表 5.1 系统服务端设备

设备类型	系统配置	参数
服务端主机、计算节点	操作系统	PVE 7.0-11
	内核版本	Linux 5.11.22-4
	CPU	Intel Xeon E3-1220
	处理器架构	x86_64
	内存	32GB
	硬盘	500GB
存储节点	操作系统	PVE 7.0-11
	内核版本	Linux 5.11.22-4
	CPU	Intel Xeon E3-1220
	处理器架构	x86_64
	内存	16GB
	硬盘	1TB
流媒体处理节点 (流媒体服务器)	操作系统	Ubuntu 16.04
	内核版本	Linux 4.4.0-210
	CPU	kvm64 vCPU
	处理器架构	x86_64
	内存	2GB
	硬盘	40GB

表 5.2 系统客户端设备

设备类型	系统配置	参数
客户端主机	操作系统	Windows 10
	CPU	Intel Celeron N5105
	处理器架构	x86_64
	内存	8GB
	硬盘	128GB

5.1.3 Proxmox VE 虚拟化环境的部署

如图 5.1 所示，将 Proxmox VE 直接安装在服务器裸机上。



图 5.1 Proxmox VE 安装界面

在安装好 Proxmox VE 之后，主界面会打印如图 5.2 所示信息：

```
-----
Welcome to the Proxmox Virtual Environment. Please use your web browser to
configure this server - connect to:

https://192.168.6.128:8006/
-----

pve login: root
Password:
Linux pve 5.11.22-4-pve #1 SMP PVE 5.11.22-8 (Fri, 27 Aug 2021 11:51:34 +0200) x86_64

The programs included with the Debian GNU/Linux system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.
Last login: Fri Feb 17 21:10:35 CST 2023 from 192.168.6.1 on pts/1
root@pve:~# _
```

图 5.2 主界面信息

管理员除了可以本地通过命令行管理服务端，还可以通过远程 SSH 或者是登陆指定 URL 的 Web 管理控制台可视化对集群进行管理。当 QEMU 进程启动时，就会调用 SPICE 协议中的 Server 动态链接库，利用 SPICE 协议和客户端 Client 进行连接。

5.1.4 服务端 Server 程序和客户端 Client 程序的部署

服务端 Server 程序是整个云桌面系统的服务端程序，在 Proxmox VE 虚拟化环境上做二次开发，由于其开源特性，允许开发人员根据部署情况和功能需求更改部分组件的功能和性能。Server 程序使用 C/C++ 语言编写，编译完成后通过暴露的 API 接口与下层 Proxmox VE 相互通信，以处理客户端和 Web 网络端业务需求。

客户端 Client 程序在编译完成后,生成 exe 可执行文件,运行在 Windows 平台上,对 Windows 操作系统的版本要求为 7.0 及以上。本文选定的客户端设备操作系统为 Windows 10,为确保 Qt 库中动态或静态链接库对操作系统的兼容性,推荐云桌面客户端 Client 程序运行在 Windows 10 及以下的操作系统中。

本文所设计的云桌面系统相对于文献[39]中提到的云桌面解决方案更加具有通用性和兼容性。云桌面客户端程序独立于特定的固件设备或硬件终端,它更像是 Windows 终端上的一个 "应用程序"。只要是符合 Windows 操作系统要求的瘦客户机或迷你终端都可以运行云桌面客户端程序。

5.2 系统的功能验证

本节将根据系统总体设计列举的总体架构,部署云桌面系统,执行服务端应用程序,并测试各功能模块。

5.2.1 用户创建、删除等管理功能

连接服务端主机的 8006 端口,例如 <https://192.168.6.128:8006/>可进入可视化的 Web 管理控制台。完整的管理界面如图 5.3 所示。在图 5.3 所示的控制台主界面中,从左至右依次是,服务器视图、功能菜单和主窗口。

服务器视图主要显示已经创建好的云桌面资源、虚拟存储池和云桌面模板。服务器视图右侧的是功能菜单,可以对系统进行相关的设置,包括系统概要、存储、备份快照、云桌面设置等功能。主窗口负责显示和操作。在页面底栏是系统日志,单击系统日志边栏,会显示云桌面系统的操作日志和集群日志。

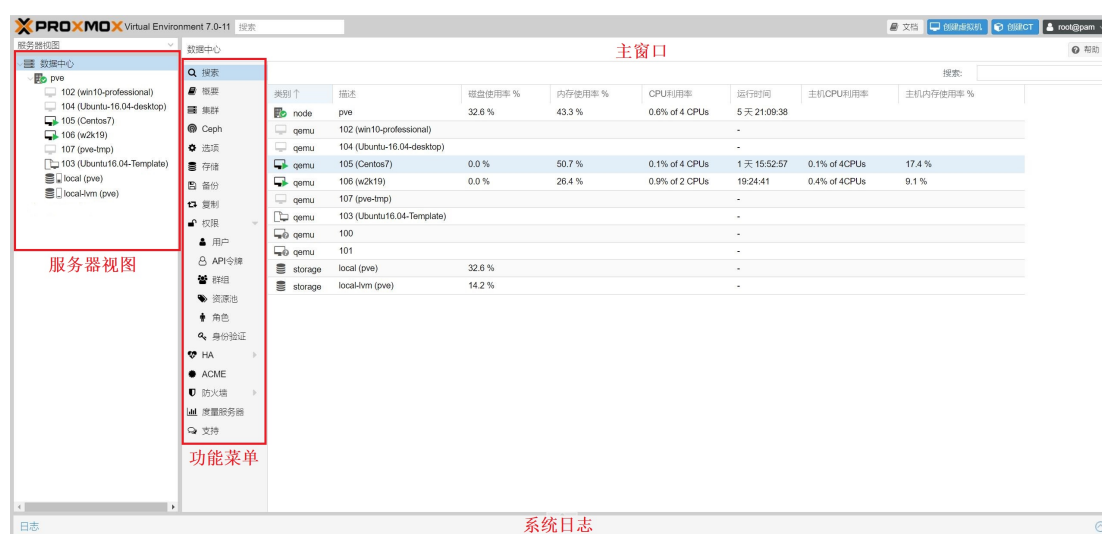
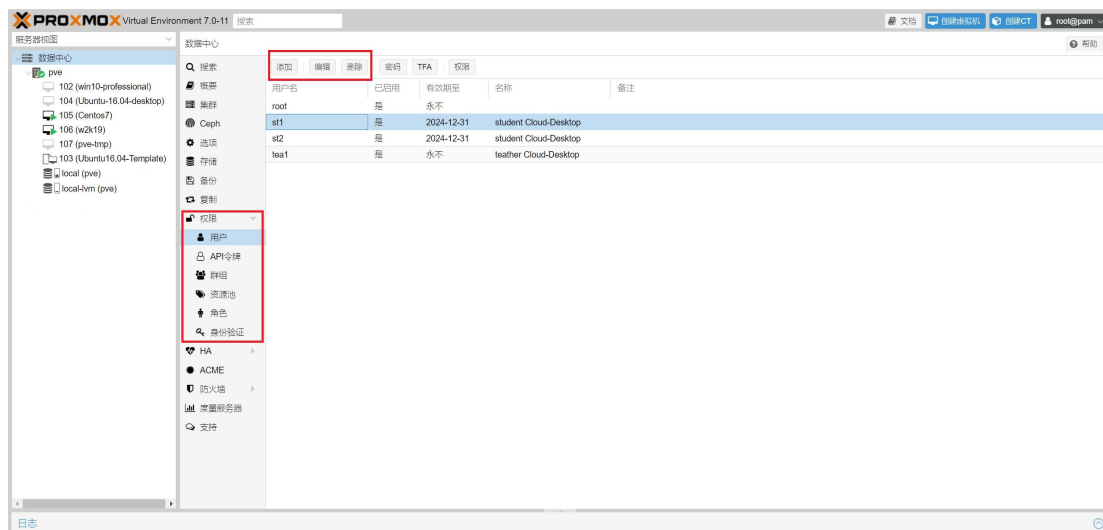


图 5.3 Web 管理控制台主界面

如图 5.4 (a) 所示, 在左侧的功能菜单中, 选择“权限-用户”选项, 管理员可以在这里对用户的添加和删除进行可视化操作。创建用户窗口如图 5.4 (b) 所示。这里以学生用户为例, 创建教师或管理员用户的过程与其基本相似。



(a) 用户列表

添加: 用户

用户名: 202030605212 名: 学生用户

领域: Proxmox VE authenticat 姓: 学生用户

密码: E-Mail:

确认密码:

组: group1

有效期至: never

已启用: ☒

备注:

高级 ☐ 添加

(b) 添加用户功能窗口

图 5.4 用户管理功能界面

同样, 通过可视化操作的过程也可以用 SSH 命令行复现。对于用户管理功能, 管理员只要修改/etc/pve/user.cfg 文件, 即可加入或删除用户, 如图 5.5 所示。

```
user:root@pam:1:0:::812044391@qq.com:::
user:st1@pve:1:1735574400:student:Cloud-Desktop:xxx@qq.com:::
user:st2@pve:1:1735574400:student Cloud-Desktop:::::
user:tea1@pve:1:0:teacher Cloud-Desktop:::::

group:group1:::

acl:1:/vms:@group1:PVEVMUser:
user.cfg (END)
```

图 5.5 user.cfg 文件

5.2.2 云桌面的创建和运行

云桌面系统的核心功能就是生成和运行云桌面。服务器应用程序在接收到用户请求后就会构建一个云桌面，该请求最初由管理员使用 Web 管理控制台发出。最后，学生或教师用户通过登录本地主机上的云桌面客户端软件访问云桌面环境。首先管理员在 Web 面板服务器视图中，向服务器主机上传用于创建云桌面的 ISO 镜像文件，如图 5.6 所示。

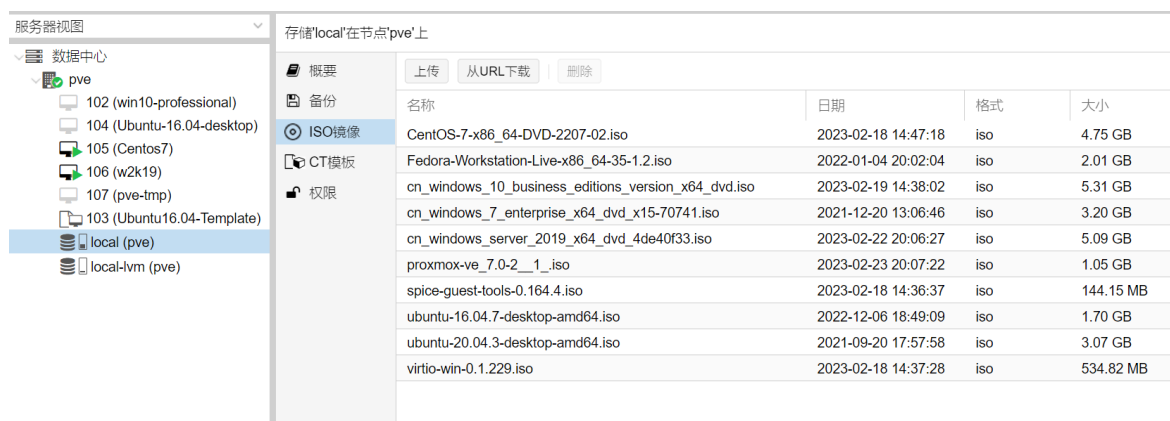
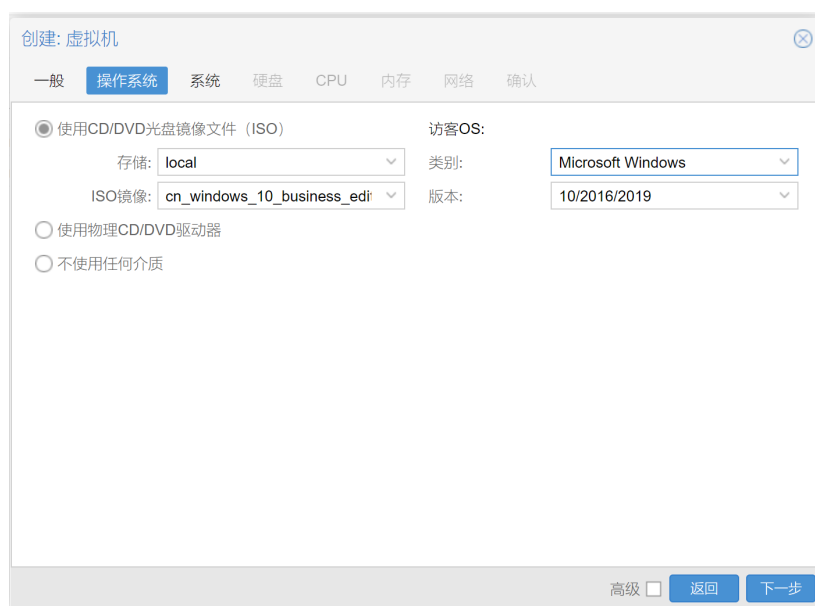
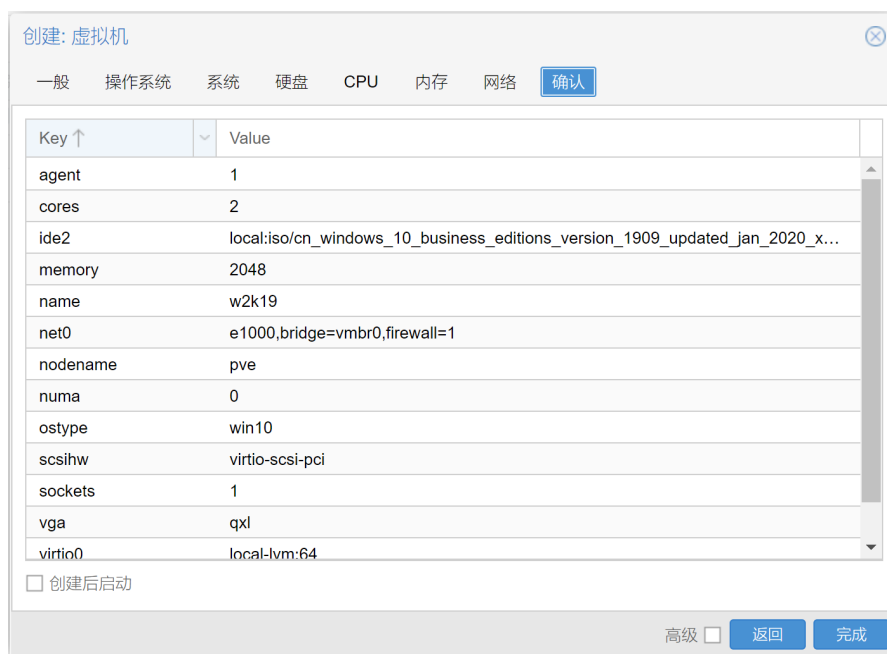


图 5.6 local 存储池内 ISO 镜像文件

随后，管理员开始创建云桌面，向服务器端发起请求，在创建云桌面窗口中，选择对应的配置参数，按照提示完成，设置完成后，即向服务端发送创建云桌面请求。等待后台回复即可。创建云桌面窗口如图 5.7（a）和 5.7（b）所示。



（a）选择虚拟桌面操作系统



(b) 确认虚拟桌面硬件配置

图 5.7 创建虚拟桌面窗口

管理员创建好虚拟桌面之后，需要在用户权限中，将每个用户与能够运行的资源做一个匹配。管理员可以规定，例如学生 st1 用户只能打开 VM102 号云桌面，教师用户可以管理整个学生用户组 group1 的云桌面资源。添加界面如图 5.8 所示。



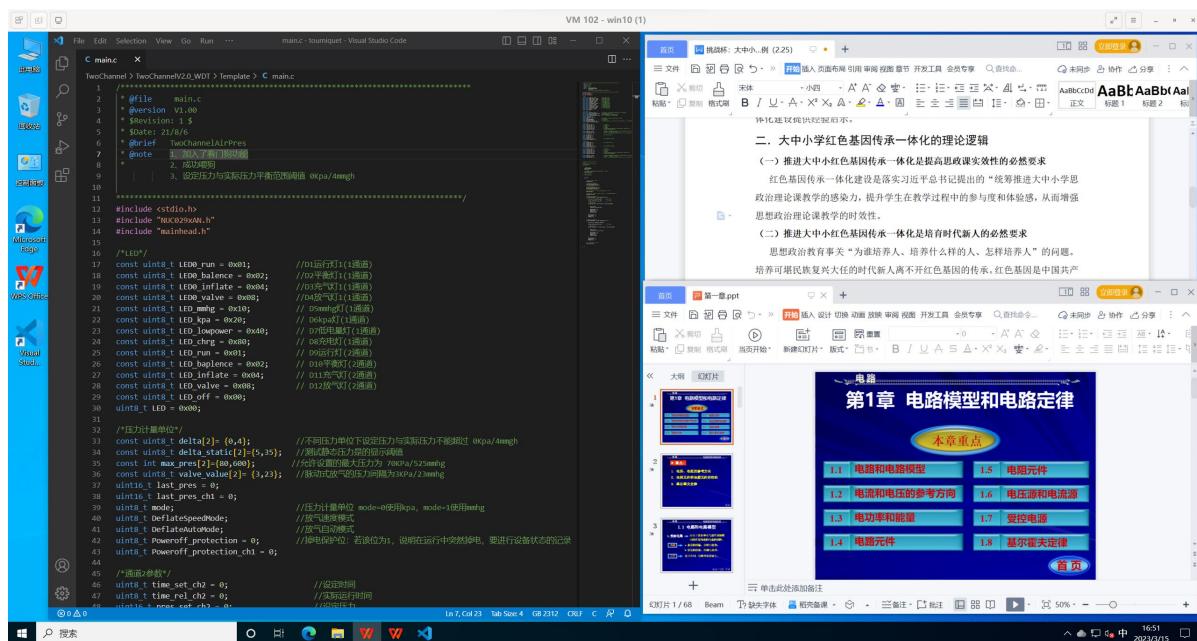
图 5.8 添加用户权限

至此，由管理员负责操作的创建云桌面功能已经完成。管理员向 VM102 号云桌面注册的权限和账户密码等配对信息，已经写入服务端/etc/pve/qemu-server/102.

conf 文件中。接下来，用户可以在自己的桌面客户端，使用用户名和密码即可进行连接。账户验证如图 5.9（a）所示，连接成功后，如图 5.9（b）所示。



（a）用户验证界面



（b）云桌面使用界面

图 5.9 云桌面登陆连接画面

5.2.3 直播教学功能验证

云桌面系统的直播教学功能现已使用在学院的《电路原理》课程中，用户在进行登陆后，如果不进行云桌面连接操作，可以在后台中进行挂起，系统提示用

户处于空闲状态。此时教师用户可以将学生用户添加到自己的用户直播间中，加入方式可以由学生用户主动选择，也可由教师用户发出邀请请求，学生用户的客户端在收到请求后，会进入线上直播进程，进入直播间后显示界面如图 5.10 所示。

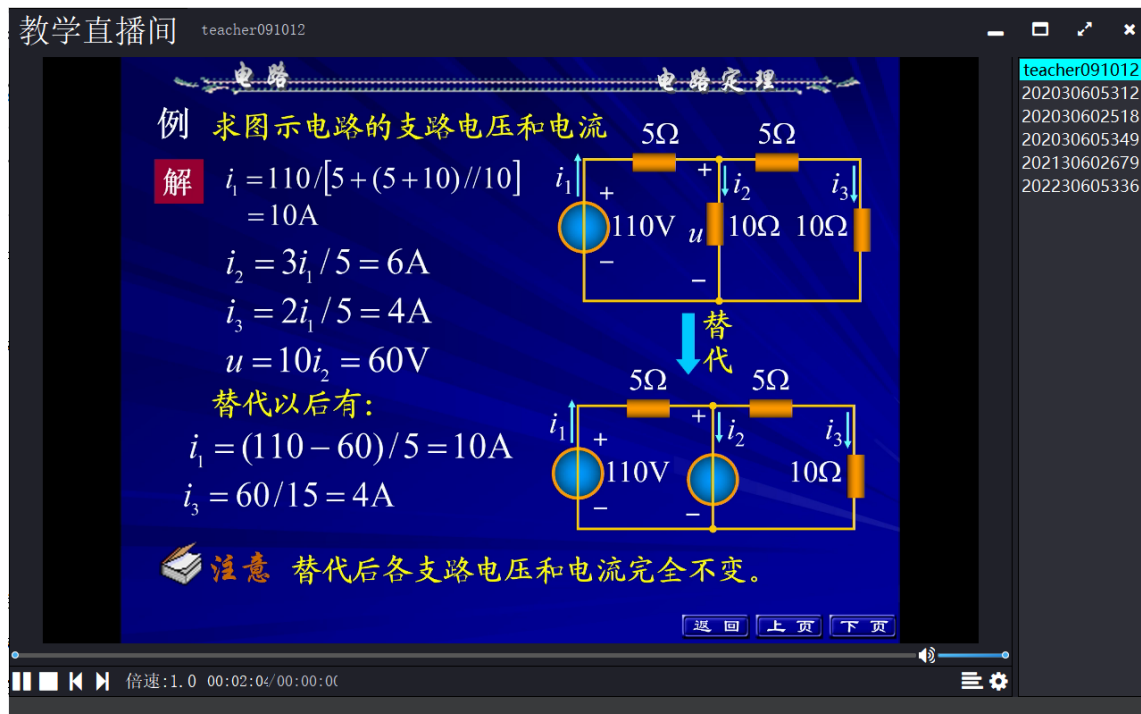


图 5.10 线上直播间界面

右侧是已经进入直播间的教师和学生用户列表，界面中央是教师用户共享的屏幕或摄像头捕捉数据。如果教师端推送的是 RTMP 或 RTSP 类型的实时媒体流，学生用户的客户端只支持暂停或开始功能。不能对收到的媒体流进行倍速、快进后退操作。如果教师端推送的是 MP4、flv 等本地视频，学生用户在缓冲完成后，即可对视频进行倍速播放、快进、后退等操作。

5.3 系统的性能验证

5.3.1 云桌面运行时性能验证

为了确保云桌面系统可以在承担一定业务压力的条件下仍正常工作，对系统进行了稳定性测试，在主服务器上开启一定数量的云桌面资源用来模拟业务压力，对系统稳定性进行测试。测试方法为云桌面后台服务器集群上，同时开启 Windows 10、Windows 7、Ubuntu、CentOS 云桌面，连续工作 12 小时。图 5.11 为连续运行

的系统 IO 响应延迟, 这个指标的好坏影响着用户利用客户端和主服务器进行通信时的流畅程度。

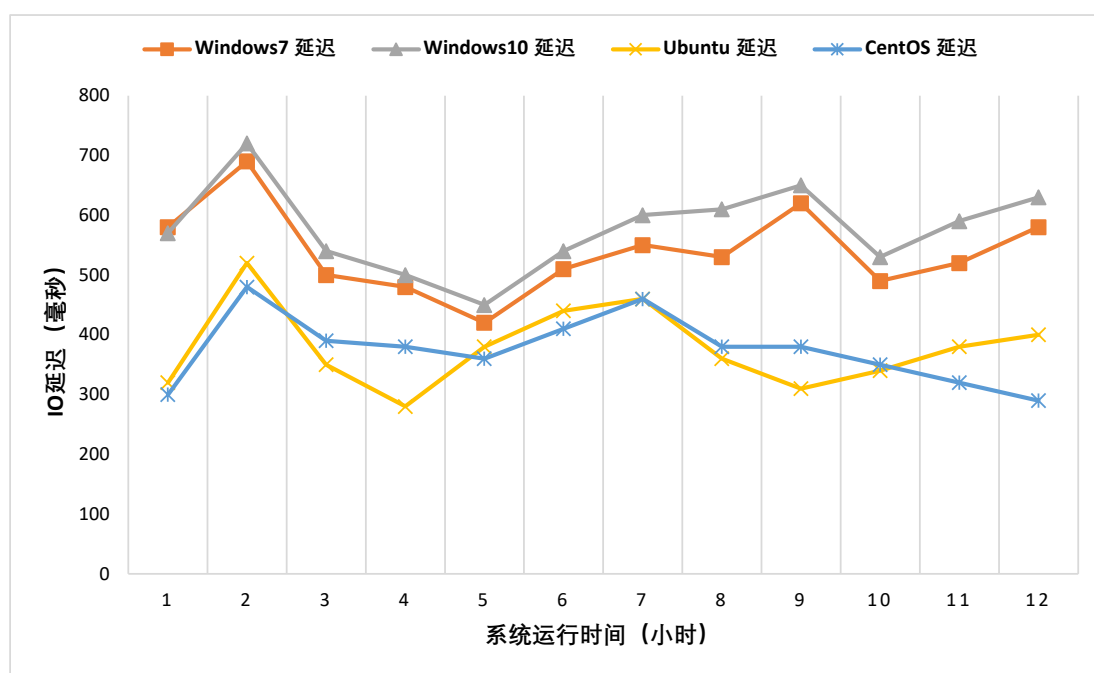


图 5.11 系统 I/O 响应延迟

当用户在连接云桌面资源时, 云桌面后台能够将系统 I/O 延迟稳定在 2 秒以内, 这能够满足用户对云桌面资源的正常使用需求。

同时本节将对比文献[39]中提到的武汉职业技术学院近年来为打造全国首批信创云机房应用的云桌面解决方案。从 CPU 占用率、网络带宽流量的角度, 在相同的使用场景下, 进行性能指标上的对比。测试场景基于云桌面的两种典型应用场景: 文本处理和浏览网页。在每个测试场景中打开 Word 软件和 Edge 浏览器, 并使用相同的用户输入模拟软件, 以确保用户和外界输入没有变化。文献[39]终端和本文设计的云桌面客户端都安装相同的 TrafficMonitor 监控软件, 进行带宽和 CPU 的监控。在每个场景中进行多组测试以确认测试数据的准确性, 并通过去除极值, 计算平均值来生成最终的测试数据。

5.3.1.1 文字处理场景下的性能对比

使用 Word 软件时, 测得武职院应用的云桌面客户端和本文设计的云桌面客户端在相同网络条件下, 使用相同主频 CPU 时的 CPU 占用率如表 5.3 所示。

表 5.3 使用 Word 时 CPU 占用率对比

设备	网络情况	CPU 占用率
文献[39]中的云桌面终端	局域网	2.01%
	广域网	3.06%
本文设计的云桌面客户端	局域网	1.68%
	广域网	2.50%

表 5.3 展示了在云桌面系统文字编辑处理场景中,两种云桌面解决方案的 CPU 占用率对比。本文设计的云桌面客户端 CPU 占用率低于文献[39]提出的方案。在文字处理场景中,两种方案的客户端 CPU 占用率都比较低,这主要是因为在该场景下,不需要复杂的压缩处理,场景中的图像较为单一,颜色程度适中,不需要 CPU 有较多的图形渲染。但由于广域网的存在,广域网的整体 CPU 消耗要比在局域网下时高。

在使用 Word 软件时,网络带宽的流量如表 5.4 所示。

表 5.4 使用 Word 时网络带宽流量对比

设备	网络情况	网络带宽流量 (Kb/s)
文献[39]中的云桌面终端	局域网	35.02
	广域网	36.82
本文设计的云桌面客户端	局域网	23.68
	广域网	24.53

表 5.4 展示了在云桌面系统文字编辑处理场景中,两种云桌面解决方案的网络流量对比。本文设计的云桌面客户端网络流量场景低于文献[39]时对网络流量的消耗。

5.3.1.2 浏览网页场景下的性能对比

打开 Edge 浏览器浏览网页时,测得两种客户端的 CPU 占用率如表 5.5 所示,网络带宽流量消耗情况如表 5.6 所示。

表 5.5 使用 Edge 浏览器时 CPU 占用率对比

设备	网络情况	CPU 占用率
文献[39]中的云桌面终端	局域网	6.28%
	广域网	6.32%
本文设计的云桌面客户端	局域网	4.33%
	广域网	5.01%

表 5.6 使用 Edge 浏览器时带宽流量对比

设备	网络情况	网络带宽流量 (Kb/s)
文献[39]中的云桌面终端	局域网	42.65
	广域网	46.38
本文设计的云桌面客户端	局域网	36.22
	广域网	39.08

在使用 Edge 浏览器浏览网页时, 本文设计的云桌面客户端在 CPU 占用率和网络带宽流量消耗上都优于文献[39]提出的方案。

5.3.2 直播教学性能验证

对于云桌面系统的线上直播模块延迟测试, 教师客户端作为直播系统的推流端, 学生客户端作为收流端。测试的客户端硬件为 5.1 节所列的云桌面系统客户端设备, 测试的视频分辨率为 1280×720 。为了检测视频播放时的延迟时间, 对线上直播模块进行了延迟测试, 测试结果如图 5.12 所示。

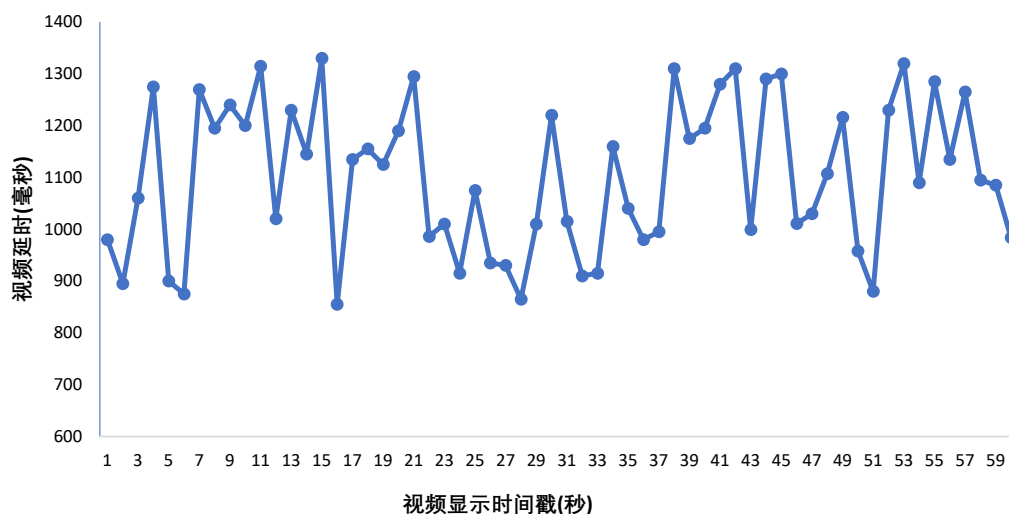


图 5.12 视频播放延时

一般越小的视频延迟用户的体验程度也越好。从图 5.12 的实验结果可以看出,选取 60 秒的直播视频进行测试,获取从推流端到收流端的流数据延迟,视频播放的延迟均在 1500ms 以内,总体延迟区间在 800ms 到 1500ms 之间,可以满足用户在云桌面系统上正常使用直播功能的需求。如表 5.7 所示,应用在教育市场的主流直播系统的延迟均在 3000ms 以上。本文所设计的线上直播功能在相同分辨率下,延时均在 1500ms 以下,延迟时间较低。

表 5.7 直播延迟对比

系统类别	720P 视频下的直播延迟时间
高清实时流媒体直播系统 ^[65]	6000ms
基于互动课堂的屏幕直播系统 ^[66]	1000ms~3000ms
本文设计的线上直播系统	800~1500ms

5.4 本章小结

本章首先介绍了服务端和客户端程序的开发环境、服务端和客户端设备的硬件配置,其次介绍了相关虚拟化平台在服务器上的部署,最后对云桌面平台用户管理、桌面创建与运行和直播教学等相关的功能和性能做了相对应的测试。通过相关实验可以得出结论,系统的功能需求和性能需求达到了要求。

第6章 总结与展望

6.1 工作总结

经过多年的发展,云计算在基础技术方面逐步得到推广,已经成为信息社会基础设施的重要组成部分,在推动社会进步方面起着重要支撑作用。作为云计算领域的重要实践,云桌面基于虚拟化技术,将其中的计算、存储、网络等各类资源抽象并池化,构建桌面资源池,实现了云桌面环境与物理设备的解耦。因此,用户可以通过云桌面客户端方便地获取桌面环境,为个人桌面环境提供管理和维护上的高效和便利。

本文所做具体工作如下:

(1) 基于服务器虚拟化技术和 Proxmox VE 虚拟化环境,提出 VDI 架构的云桌面系统。本文所设计的云桌面系统,包括 Proxmox 平台、服务端 Server 程序、客户端 Client 程序三大部分。所有数据由服务器统一存储运行,在性能上弹性化,不依赖本地终端。通过 VDI 架构的设计,降低了桌面环境的管理复杂性,并实现了对云桌面的按需资源分配,包括 CPU、内存、存储和其他资源,提高了桌面环境拓展和部署的灵活性。

(2) 针对云桌面的用户场景和需求进行了说明。明确本文所设计的云桌面系统的使用场景是高校云实验室和云办公,主要功能就是给学生和教师用户提供一个桌面环境,满足其实验以及办公需求。

(3) 为了更好地发挥网络教育在信息传递和资源传递中的优势,在本文设计的系统中加入了线上教育直播系统。设计并实现了直播教学模块,该模块的设计初衷是用户在使用云上直播教学时,也可以充分地将优质教学资源同现代化教育平台相结合。能够将互联网与高校教学办公相结合,实现校园内智慧云办公和教学。本文通过优化 x264 编码器以改进 FFmpeg 框架的功能设置,以此减少直播中推流拉流的延迟,并且研究了各种因素对系统性能的影响。

(4) 设计并实现了用户管理模块,用于处理所有类型的系统用户,包括添加新用户、删除现有用户、用户登录和注销。用户管理模块的基本功能之一是创建和删除用户,这可以由管理员来完成。管理员除了能添加删除用户,还可以对每个用户进行权限的制定,绑定用户各自的桌面资源。

(5) 设计并实现了桌面管理模块,该模块是云桌面系统最核心的部分,也是服务端 Server 程序最重要的部分。根据云实验室、高校云办公场景的部署和功能

需求,在服务器端 Server 程序中对 Proxmox VE 虚拟化环境的部分组件进行重写和优化,通过暴露的接口与底层 Proxmox VE 进行通信。涉及的功能组件包括桌面环境的创建、QEMU 侧 CPU 的创建、KVM 侧 vCPU 的创建、内存虚拟化和设备虚拟化组件等。

(6) 为虚拟桌面提供存储空间,创建的存储空间一般分为托管式存储和非托管式存储。托管式存储是利用存储池、存储卷进行统一的管理,非托管式存储是由管理员手动创建,用于测试环境。本文所设计的系统,用 `qemu-img` 命令对非托管式存储空间进行管理,用 `pvesm` 命令对 Proxmox 平台进行创建的托管式存储空间进行管理。除了以上两种方式,管理员还可以通过 Web 管理控制台进行可视化操作,更加快捷高效。另外,为了解决在实验室中对同一批相同环境下的云桌面一一创建时效率低下和资源碎片化严重的问题,提出基础映像和派生映像的解决办法,大大降低了对宿主机物理存储空间的压力。

完成以上设计之后,在实际的物理环境中部署了云桌面系统,并测试验证了系统各个功能的运行以及相关功能的性能情况。验证结果表明,本文设计的云桌面系统各模块功能和性能指标正常,可以实现云桌面资源分配和线上直播教学等功能。

6.2 后续展望

在现有工作的基础上,展望未来的研究路径,有必要在以下领域对这一主题进行更深入的研究和应用。

(1) 本文所设计的云桌面系统采用了 VDI 架构,但是没有一种架构模式可以处理所有的问题,每一种云桌面系统架构都有自己的实际适用场景。例如,VDI 与 IDV 架构就适用于不同的业务场景。IDV 架构的优势在于“集中管理,本地运行”,在 IDV 架构下,虚拟桌面被放到了本地终端中运行,这样大大减轻了宿主机的负载负担,最大限度地减少了网络的传输和对网络带宽的要求。服务端只需负责桌面环境的统一配置和维护即可。但缺点就是终端设备需要计算能力强的客户机,将 VDI 架构中对宿主机服务器和网络传输的高需求分摊到了 IDV 架构中的每一台客户机上。在今后的研究中,可以整合 IDV 和 VDI 架构各自的优势,研究各种场景下的融合云桌面架构模型,并增强实际用户体验。

(2) 虽然本文对直播教学功能中视频网络延迟进行了改进,但考虑到直播教育系统的复杂性,仍可进行更多研究。比如,可以为流媒体服务器提供保存流媒体数据的能力,这样学生就可以回看教师的讲课内容;为了防止恶意信息流入,

可以加入推流鉴权功能，还可以对推流数据进行加密，防止推流信息被不法分子盗用。

(3) 本文设计的云桌面系统的客户端仅在 Windows 平台上进行了开发，暂时没有 Linux 平台和 arm 平台版本。今后应解决客户端多平台版本的问题。

参考文献

- [1] 游钊栗, 熊卫华, 应繁. 基于服务器虚拟化的智慧云桌面系统设计与应用[J]. 软件工程, 2022, 25(10): 54-58.
- [2] Anthony F, Julie K, A. P M. Using Online Class Preparedness Tools to Improve Student Performance: The Benefit of “All-In” Engagement [J]. Journal of Management Education, 2021, 45(4): 558-578.
- [3] 张军, 吴荻, 周海芳, 等. 基于虚拟桌面计算机实验室实训教学平台建设[J]. 实验室研究与探索, 2021, 40(08): 234-238.
- [4] 庄严. 超融合架构平台在发射监控系统中的应用[J]. 广播与电视技术, 2020, 47(12): 102-108.
- [5] 陶奎印. 基于 FFmpeg 的教育直播系统设计与实现[D]. 大连: 大连理工大学, 2021: 6-15.
- [6] Algarni S A, Ikbal M R, Alroobaea R, et al. Performance Evaluation of Xen, KVM, and Proxmox Hypervisors[J]. International Journal of Open Source Software and Processes (IJOSSP), 2018, 9(2): 39-54.
- [7] 张沛昊. 高校超融合云平台的设计与实现[J]. 现代信息科技, 2021, 5(04): 20-23.
- [8] 敖青云, 蒋文蓉. 基于 KVM 和 QEMU 的虚拟桌面系统的实现与应用[J]. 计算机应用与软件, 2012, 29(11): 217-219.
- [9] Hai Y, Jing G, Lei W, et al. Application of Server Virtualization Technology in Power Information Construction [J]. Journal of Physics: Conference Series, 2021, 1744 (2): 1-8.
- [10] Jia L, Zhi L, Tianhong Z. Research on the Practical Application of Server Virtualization Technology in Computer Laboratory [J]. Journal of Physics: Conference Series, 2021, 1744 (2): 1-7.
- [11] 吴金坛, 陈路路, 李智鑫. 虚拟机和容器超融合试验研究[J]. 计算机应用与软件, 2021, 38(09): 10-5+59.
- [12] 孙义刚. 高校超融合平台分布式存储模式探索与实践[J]. 科技风, 2021, (01): 105-6.
- [13] 毕红棋, 陈露. 基于超融合架构的智慧校园双活数据中心建设研究[J]. 教书育人(高教论坛), 2021, (06): 26-7.
- [14] 郑凯, 刘一鸣. 云桌面在勘测设计与办公系统中的应用研究[J]. 人民黄河, 2019, 41(S2): 62-63+74.
- [15] 杨雄, 王泽世. 基于人脸识别的云桌面身份认证系统[J]. 制造业自动化, 2020, 42(04): 1-4+23.
- [16] 张殷希. 存储虚拟化在无锡广电新媒体中的应用[J]. 电视技术, 2016, 40(12): 73-76+124.
- [17] 王星魁, 彭新光, 郝瑞. Xen 虚拟机存储方式 I/O 性能研究[J]. 太原理工大学学报, 2013, 44(05): 608-611.

- [18] Abu L M, Chung T, Huh E N. Adaptive Desktop Delivery Scheme for Provisioning Quality of Experience in Cloud Desktop as a Service[J]. Computer Journal, 2016, 59 (2): 260-274.
- [19] 李艳, 吕鹏, 李珑. 虚拟云桌面为高校图书馆服务和管理带来的革新——以中南民族大学图书馆为例[J]. 现代情报, 2015, 35(06): 58-63.
- [20] Zhang K. Application of Desktop Computing Technology Based on Cloud Computing[J]. International Journal of Information Technologies and Systems Approach, 2021, 14(2): 1-19.
- [21] 骆慧勇. 基于云桌面实现网络安全隔离的应用[J]. 计算机应用与软件, 2020, 37(02): 15-17+38.
- [22] 王斌, 李伟民, 盛津芳, 肖斯诺. TCCL:安全高效的拓展云桌面架构[J]. 通信学报, 2017, 38(S1): 9-18.
- [23] Choi J, Lee T, Shin S. Application Methods for the Desktop-as-a-Service (DaaS) on the Virtual Desktop Infrastructure (VDI) of the National Defense Cloud System using the Desktop Experience (DeX) based on Secure Mobile Networks[J]. KIISE Transactions on Computing Practices, 2022, 28(7): 387-392.
- [24] 邓意. 基于 noVNC 的容器化的云桌面设计与实现[D]. 成都: 电子科技大学, 2021: 2-4.
- [25] ZHUMA M E, BRITO CASANOVA O J, TUBAY V J, et al. DYNAMIC ANALYSIS OF MALWARE IN A VIRTUALIZED NETWORK ENVIRONMENT[J]. Revista Conrado, 2021, 17(78): 113-20.
- [26] Algarni S A, Ikbai M R, Alroobaea R, et al. Performance Evaluation of Xen, KVM, and Proxmox Hypervisors[J]. International Journal of Open Source Software and Processes (IJOSSP), 2018, 9(2): 39-54.
- [27] Casanova L, Marcel, Kristianto E. Comparing RDP and PCoIP Protocols for Desktop Virtualization in VMware Environment[J]. Advanced Science Letters, 2018, 24(1): 592-595.
- [28] 张源泉. 基于 Citrix 技术的连锁药店结账系统远程共享的设计和实现[J]. 现代计算机: 上下旬, 2016, 71-76.
- [29] Kumar V S V, Malathi K. Managing the Stateful Applications for High Availability using Novel Kubernetes based Microservice Architecture over Cloud based Azure Virtualization Architecture[J]. Journal of Pharmaceutical Negative Results, 2022, 13: 1536-1547.
- [30] 卢天池, 张国有. 公有云服务商竞争力的中美比较[J]. 科技管理研究, 2020, 40(04): 138-145.
- [31] Regola N, Chawla N V. Storing and using health data in a virtual private cloud[J]. Journal of medical Internet research, 2013, 15(3): 63-63.
- [32] 黄建峰. 积极融入数字中国建设 加快档案信息化战略转型[J]. 中国档案, 2022(10): 32-33.
- [33] 允升. 2022 信创云桌面企业排行[J]. 互联网周刊, 2022(18): 28-29.
- [34] 陈鹏. 数智金融的行业趋势前瞻[J]. 中国金融, 2022(S1): 40-42.
- [35] 刘志文, 钱震. “互联网+”时代“准智慧”教室的设计与应用研究[J]. 现代教育技术, 2019, 29(12): 68-74.

- [36] 高志强, 袁敏, 张杰. 面向用户需求的云桌面推荐方法研究[J]. 华东师范大学学报(自然科学版), 2019(04): 97-110.
- [37] 韩友前. 基于 IDV 技术的智能云教室部署与优化[J]. 实验技术与管理, 2021, 38(02): 237-241+245.
- [38] 梁聪. 华为桌面云技术白皮书英译项目报告[D]. 南京: 南京师范大学, 2018: 24-69.
- [39] 苏传宇. 基于 KVM 与 IDV 架构的桌面云服务端设计与实现[D]. 广州: 华南理工大学, 2020: 7-20.
- [40] 陈涛. 虚拟化 KVM 极速入门[M]. 北京: 清华大学出版社, 2022: 2-12.
- [41] Popek G J, Goldberg R P. Formal requirements for virtualizable third generation architectures[J]. Communications of the ACM, 1974, 17(7): 412-421.
- [42] 朱琛, 王剑, 高翔, 等. 龙芯 KVM 虚拟机 I/O 中断子系统的优化[J]. 高技术通讯, 2020, 30(09): 893-900.
- [43] 张寒冰, 毛明, 史国振, 等. 基于 QEMU-KVM 的密码计算资源虚拟化方法[J]. 计算机应用与软件, 2019, 36(11): 315-321.
- [44] 张健, 蔡长亮, 宫良一, 等. 基于 KVM 虚拟化环境的异常行为检测技术研究[J]. 信息安全, 2017(11): 1-6.
- [45] Segalini A, Pacheco D L, Urvoy-Keller G, et al. Hy-FiX: Fast In-Place Upgrades of KVM Hypervisors[J]. Ieee Transactions on Cloud Computing, 2022, 10(4): 2679-2690.
- [46] Li C, Guo R, Tian X, et al. KHV: KVM-Based Heterogeneous Virtualization[J]. Electronics, 2022, 11(16): 1-14.
- [47] Wojtowicz R, Kowalik R, Rasolomampionona D D, et al. Virtualization of Protection Systems-Tests Performed on a Large Environment Based on Data Center Solutions[J]. Ieee Transactions on Power Delivery, 2022, 37(4): 3401-3411.
- [48] Yang C T, Liu J C, Chen S T, et al. Virtual machine management system based on the power saying algorithm in cloud[J]. Journal of Network and Computer Applications, 2017, 80: 165-180.
- [49] 刘子杰, 文成玉, 薛霁. 基于 SPICE 协议的虚拟桌面技术[J]. 计算机系统应用, 2018, 27(04): 100-3.
- [50] 赖孙荣. 虚拟桌面框架 Spice 剖析及其客户端的设计与实现[D]. 广州: 华南理工大学, 2012: 28-36.
- [51] 翟莹昕, 历颖, 赵新倩. 一种基于本地域的虚拟桌面架构设计与性能测试[J]. 中国测试, 2020, 46(06): 116-120.
- [52] 何新彪. 基于云终端算力提升 VDI 云桌面性能的融合架构[J]. 科学技术创新, 2022(31): 70-73.
- [53] 祝成. 基于 VDI 技术虚拟桌面在档案信息系统的应用[J]. 中国档案, 2017(02): 62-63.
- [54] 阿不都克里木·玉素甫, 王亮亮. 基于自主可控平台的 FFmpeg 在线视频转换系统[J]. 计算机与现代化, 2020(01): 81-84+116.

- [55] 辛长春, 娄小平, 吕乃光. 基于 FFmpeg 的远程视频监控系统编解码[J]. 电子技术, 2013, 40(1): 3-5.
- [56] 黄松涛, 夏挺. 基于 FFmpeg 的远程监控调试系统[J]. 电子世界, 2019(1): 177-178.
- [57] 雷霄骅, 姜秀华, 王彩虹. 基于 RTMP 协议的流媒体技术的原理与应用[J]. 中国传媒大学学报(自然科学版), 2013(6): 59-64.
- [58] Wales G S, Smith J M, Lacey D S, et al. Multimedia stream hashing: A forensic method for content verification[J]. Journal of Forensic Sciences, 2023, 68(1): 289-300.
- [59] 邓正良. 基于 FFmpeg 和 SDL 的视频流播放存储研究综述[J]. 现代计算机, 2019(22): 47-50.
- [60] 郝朝, 刘升护. 基于 FFmpeg 和 SDL 的遥测视频解析技术[J]. 计算机技术与发展, 2019, 29(4): 191-194.
- [61] Zhao H, Zhou C-L, Jin B-Z. Design and implementation of streaming media server cluster based on FFMpeg[J]. The Scientific World Journal, 2015, 2015: 963083.
- [62] 任衡. 基于互动课堂的屏幕直播系统延迟优化与设计实现[D]. 北京: 北京工业大学, 2019: 7-12.
- [63] 李闻斌, 庞璐宁, 黄晟. 实时流媒体传输系统中关键技术的研究与实现[J]. 信息通信, 2020(12): 159-161.
- [64] 杨明. 在线教育直播平台设计与实现[D]. 北京: 北京交通大学, 2019: 7-11.
- [65] 葛学仕. 高清实时流媒体直播系统研究[D]. 上海: 上海交通大学, 2014.
- [66] 任衡. 基于互动课堂的屏幕直播系统延迟优化与设计实现[D]. 北京: 北京工业大学, 2019.